



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Informatyki

Filia w Gdańsku

Bartosz Bohatyrewicz

Nr albumu s26860

Nazwa specjalizacji: Sztuczna Inteligencja

Łukasz Korycki

Nr albumu s26972

Nazwa specjalizacji: Cyberbezpieczeństwo

Kamil Maliński

Nr albumu s26984

Nazwa specjalizacji: Cyberbezpieczeństwo

Jan Szydłowski

Nr albumu s26978

Nazwa specjalizacji: Cyberbezpieczeństwo

Igor Wojciechowski

Nr albumu s27106

Nazwa specjalizacji: Aplikacje Internetowe

Wdrożenie systemu weryfikacji uprawnień według rezerwacji z planu zajęć w Gdańskim oddziale PJATK w budynku wydziału Informatyki

Rodzaj pracy

inżynierska

Imię i nazwisko promotora

dr hab. Marek Bednarczyk

Gdańsk, Luty, 2026

Streszczenie: Celem pracy inżynierskiej było zaprojektowanie i stworzenie systemu [EkeyPJATK](#), który umożliwia weryfikację uprawnień użytkowników na podstawie rezerwacji pochodzących z planu zajęć systemu uczelnianego [PJATK](#). Projekt nie stanowi bezpośredniej kontynuacji systemu [Cyfrowe Klucze](#), lecz został opracowany jako rozwiązanie uzupełniające, współpracujące z istniejącą infrastrukturą oraz z aplikacją [Cyfrowe Klucze Mobile](#).

W ramach projektu powstał moduł odpowiedzialny za pobieranie i przetwarzanie danych z systemu [GAKKO](#), mechanizmy weryfikacji dostępu do sal, moduł raportowy oparty na logach z [modułów sprzętowych](#) oraz rozbudowany panel zarządzania dla dziekanatu, administracji i zespołu IT/[BSS](#). Opracowano również nową wersję [modułu sprzętowego](#) z obsługą [PoE](#) oraz funkcjonalność generowania kodów [QR](#) wykorzystywanych do uwierzytelniania użytkowników przy użyciu aplikacji mobilnej [Cyfrowe Klucze Mobile](#).

Ze względu na ograniczenia techniczne i organizacyjne po stronie systemów zewnętrznych, pełna integracja z [Cyfrowe Klucze](#) nie była możliwa. System został jednak przygotowany w formie kompletnego prototypu, który może zostać rozwinęty i zintegrowany w kolejnych etapach wdrożenia.

Rezultatem pracy jest spójny, działający system prototypowy, który zwiększa efektywność zarządzania dostępem do pomieszczeń uczelnianych oraz stanowi podstawę do dalszego rozwoju i wdrożenia w środowisku produkcyjnym.

Słowa kluczowe: czytniki, systemy kontroli dostępu, bezpieczeństwo, [karty magnetyczne](#), integracja systemów



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: Wdrożenie systemu weryfikacji uprawnień według rezerwacji z planu zajęć w Gdańskim oddziale PJATK w budynku wydziału Informatyki Temat projektu po angielsku: Implementation of a system for verifying access rights based on reservations from the class schedule at the Gdańsk branch of PJATK in the Computer Science Department building.	Akronim: EkeyPJATK Data ustalenia tematu 2025-04-01
Promotor: dr hab. Marek Bednarczyk	Konsultanci: 1. Antoni Ulenberg
Cele projektu: Stworzenie systemu weryfikacji uprawnień według rezerwacji z planu zajęć zintegrowanego z systemem Cyfrowe Klucze i aplikacją Cyfrowe Klucze Mobile .	
Rezultaty projektu: Moduł weryfikujący uprawnienia według rezerwacji z planu zajęć i przekazującego wynik do systemu Cyfrowe Klucze . Moduł generujący kody QR wyświetlanym na czytniku. Aplikacja internetowa dla administracji i dziekanatu. Dokumentacja techniczna i użytkowa. Przekazywanie wyniku weryfikacji uprawnień do sali na podstawie rezerwacji z planu zajęć do systemu Cyfrowe Klucze . Pośredniczenie pomiędzy aplikacją Cyfrowe Klucze Mobile , a systemem Cyfrowe Klucze .	
Miary sukcesu: Integracja systemu EkeyPJATK z Cyfrowe Klucze i Cyfrowe Klucze Mobile , umożliwiając automatyczne zarządzanie dostępem do pomieszczeń na podstawie planu zajęć.	
Ograniczenia: Współpraca z istniejącym systemem uczelnianym PJATK . Wiele osób pośrednich wymaganych do realizacji projektu.	

Wykonawcy	Numer albumu	Specjalizacja	Tryb studiów
Bartosz Bohatyrewicz	s26860	Sztuczna Inteligencja	Stacjonarny
Łukasz Korycki	s26972	Cyberbezpieczeństwo	Stacjonarny
Kamil Maliński	s26984	Cyberbezpieczeństwo	Stacjonarny
Jan Szydłowski	s26978	Cyberbezpieczeństwo	Stacjonarny
Igor Wojciechowski	s27106	Aplikacje Internetowe	Stacjonarny

Data ukończenia projektu: 17 stycznia 2026	Recenzent: dr. Bartosz Marcinkowski
--	---

Spis treści

1	Wstęp	7
2	Słownik pojęć	8
3	Opis problemu	15
3.1	Analiza stanu obecnego	15
3.1.1	Zidentyfikowane problemy	15
3.1.2	Analiza problemów	16
3.2	Analiza oryginalnego projektu Cyfrowe Klucze (CK)	17
3.3	Analiza oryginalnego projektu Cyfrowe Klucze Mobile (CKM)	18
3.4	Analiza konkurencji	19
3.4.1	Tritech	19
3.4.2	Logitech Tap Scheduler	20
3.4.3	Vidikom	21
3.5	Propozycja rozwiązania	23
4	Konteksty projektu	25
4.1	Kontekst systemowy	25
4.1.1	Zakres i granice systemu EkeyPJATK	26
4.1.2	Aktorzy i systemy zewnętrzne	27
4.1.3	Scenariusze interakcji użytkowników z systemem	28
4.1.4	Różnice względem prototypu Cyfrowe Klucze	29
4.1.5	Wymogi BHP i bezpieczeństwa ewakuacji	29
4.2	Udziałowcy	30
4.3	Kontekst biznesowy	36
4.3.1	Wstęp	36
4.3.2	Zastosowanie Business Model Canvas	36
4.3.3	Propozycja wartości i segmenty klientów	36
4.3.4	Kanały i relacje	36
4.3.5	Strumienie przychodów / korzyści	36
4.3.6	Kluczowe zasoby	37

4.3.7	Kluczowe działania	37
4.3.8	Kluczowe partnerstwa	37
4.3.9	Struktura kosztów	37
4.4	Kontekst społeczny	37
4.4.1	Interesariusze i role	37
4.4.2	Zmiany i spodziewane skutki	38
4.4.3	Transparentność i komunikacja	38
4.4.4	Akceptacja społeczna i partycypacja użytkowników	39
4.4.5	Miernik wpływu społecznego	39
4.4.6	Ryzyka społeczne i środki zaradcze	39
4.5	Kontekst etyczny	39
4.5.1	Założenia i wartości	40
4.5.2	Prywatność i minimalizacja danych	40
4.5.3	Proporcjonalność i celowość	40
4.5.4	Sprawiedliwość i niedyskryminacja	41
4.5.5	Transparentność i rozliczalność	41
4.5.6	Bezpieczeństwo informacji	41
4.5.7	Dostępność i inkluzywność	41
4.5.8	Zarządzanie ryzykiem i nadzór etyczny	42
4.5.9	Metryki etyczne	42
4.5.10	Podsumowanie	42
5	Planowanie projektu	43
5.1	Metodyka pracy	43
5.2	Infrastruktura	45
5.2.1	Kontrola Wersji i Organizacja Repozytoriów	45
5.2.2	Komunikacja Zespołowa i Interesariusze	45
5.2.3	Zarządzanie Wiedzą i Projektowanie	46
5.2.4	Ewolucja Narzędzi Zarządzania	47
5.3	Podział ról	48
5.4	Ryzyka i ograniczenia	48
5.4.1	Podsumowanie ryzyk z perspektywy systemu EkeyPJATK	53
5.5	Proces planowania	54
6	Analiza wymagań	56
6.1	Wymagania ogólne i dziedzinowe	56
6.2	Wymagania funkcjonalne	59
6.3	Wymagania zewnętrzne	64

6.4	Interfejs z otoczeniem	68
6.5	Przypadki użycia	73
6.5.1	Podstawowi użytkownicy	74
6.5.2	Użytkownicy z uprawnieniami do sal	74
6.5.3	Dziekanat	75
6.5.4	Ochrona	77
6.5.5	IT/BSS	78
6.5.6	Administracja	78
7	Projektowanie systemu	80
7.1	Architektura Systemu	80
7.1.1	Integracja z infrastrukturą Cyfrowych Kluczy	81
7.1.2	Integracja z Cyfrowymi Kluczami Mobile	81
7.1.3	Połączenie z Gakko	82
7.1.4	Bazy danych EkeyPJATK	87
7.2	Technologie i ewolucja stosu technologicznego	89
7.2.1	Modularność i struktura repozytoriów	89
7.2.2	Pierwotny stos technologiczny	90
7.2.3	Problemy z pierwotnym podejściem	90
7.2.4	Migracja na Go i unifikacja technologii	90
7.2.5	Zmiana architektury: od MVC do architektury heksagonalnej	91
7.2.6	Ewolucja technologii baz danych	92
7.3	Projekt interfejsu użytkownika (UI)	92
7.3.1	Moduł zarządzania rezerwacjami	93
7.3.2	Obsługa zdarzeń i zgłoszeń	93
7.3.3	Dziennik zdarzeń (Logi)	94
7.3.4	Generator raportów	95
7.3.5	Ewolucja projektu	96
7.4	Implementacja interfejsu (Hi-Fi)	96
7.4.1	Struktura nawigacyjna i układ widoku	97
7.4.2	Komunikacja z użytkownikiem i obsługa błędów	98
7.4.3	Przejsie od prototypu do implementacji	99
7.5	Projekt modułu sprzętowego	99
7.5.1	Architektura elektroniki	99
7.5.2	Projekt interfejsu użytkownika (UI)	100
7.5.3	Backend	101
7.5.4	Projekt obudowy	101

8	Opis realizacji	102
8.1	Semestr 6.	102
8.1.1	Marzec - Czerwiec 2025	102
8.2	Wakacje	102
8.2.1	Lipiec - Sierpień 2025	102
8.2.2	Wrzesień 2025	103
8.3	Semestr 7.	103
8.3.1	Październik 2025	103
8.3.2	Listopad 2025	104
8.3.3	Grudzień 2025	104
8.3.4	Styczeń 2026	105
9	Szczegóły implementacji	106
9.1	Informacje techniczne i architektura systemu	106
9.1.1	Główny moduł backendowy (digikey-DB)	106
9.1.2	Moduł raportów (digikey-reports)	107
9.1.3	Moduł QR (digikey-QR)	107
9.1.4	Frontend	108
9.1.5	Moduł Sprzętowy (door-panel)	108
9.2	Działanie elementów systemu	109
9.2.1	Moduł Sprzętowy(door-panel)	109
9.2.2	Moduł DB (digikey-DB)	109
9.2.3	Moduł raportów (digikey-reports)	119
9.2.4	Frontend	127
9.3	Moduł QR	131
10	Testy	140
10.1	Testy jednostkowe	141
10.2	Testy akceptacyjne	142
10.3	Testy systemowe	143
10.4	Testy penetracyjne	144
10.4.1	Rekonesans i analiza powierzchni ataku	144
10.4.2	Weryfikacja siły haseł usług pomocniczych	145
10.4.3	Testy obciążeniowe i wykrycie podatności typu Denial of Service	148
10.4.4	Weryfikacja bezpieczeństwa warstwy danych	150
10.4.5	Weryfikacja odporności na wstrzykiwanie kodu SQL	151
10.4.6	Podsumowanie testów penetracyjnych	153

10.5 Testy manualne	153
10.5.1 Testy modułu sprzętowego i kontroli dostępu	153
10.5.2 Testy frontendu i panelu dziekanatu	156
10.5.3 Testy integracyjne i awaryjne	159
11 Prezentacja rozwiązań	163
12 Nakład pracy	164
12.1 Ogólny nakład pracy	165
12.2 Bartosz Bohatyrewicz	166
12.3 Łukasz Korycki	166
12.4 Kamil Maliński	168
12.5 Jan Szydłowski	169
12.6 Igor Wojciechowski	169
13 Podsumowanie	170
13.1 Zrealizowane zadania	170
13.1.1 Moduł raportowy	170
13.1.2 Moduł sprzętowy	172
13.1.3 Moduł QR	173
13.1.4 Panel zarządzania	173
13.2 Niezrealizowane zadania	174
13.2.1 Integracja z systemem Cyfrowe Klucze (CK)	174
13.2.2 Integracja z Gakko	176
13.3 Doświadczenia i wyzwania projektu	177
13.3.1 Kontynuowanie projektu inżynierskiego innego zespołu	177
13.3.2 Współpraca z zewnętrznymi zespołami	178
13.3.3 Wyzwania i doświadczenia z wytworzeniem własnych modu- łów sprzętowych	179
13.4 Plany na przyszłość	179
13.5 Podsumowanie końcowe	180
14 Bibliografia i materiały dodatkowe	182
14.1 Tabele	182
14.2 Rysunki	183
14.3 Kod	185
14.4 Bibliografia	186

Rozdział 1

Wstęp

Dokument przedstawia proces tworzenia systemu [EkeyPJATK](#), którego celem jest weryfikacja uprawnień użytkowników na podstawie rezerwacji pochodzących z planu zajęć. System nie stanowi bezpośredniej kontynuacji projektu [Cyfrowe Klucze](#), lecz został zaprojektowany jako rozwiązanie uzupełniające, współpracujące z istniejącą infrastrukturą uczelnianą oraz z aplikacją [Cyfrowe Klucze Mobile](#).

W ramach projektu opracowano mechanizmy umożliwiające weryfikację dostępu do sal na podstawie danych z systemu [GAKKO](#), a także odświeżono i rozbudowano panele zarządzania dla Dziekanatu, Administracji oraz zespołu IT/[BSS](#). Ze względu na prototypowy charakter oryginalnego systemu [Cyfrowe Klucze](#), konieczne było również stworzenie nowej wersji modułu sprzętowego, dostosowanej do potrzeb [SCD](#).

W wyniku współpracy z zespołem rozwijającym [Cyfrowe Klucze Mobile](#) system [EkeyPJATK](#) został rozszerzony o funkcjonalności umożliwiające uwierzytelnianie użytkowników przy użyciu kodów [QR](#). Integracja ta, mimo ograniczeń wynikających z architektury zewnętrznych systemów, pozwoliła na stworzenie alternatywnego mechanizmu dostępu, który może być rozwijany w kolejnych etapach projektu.

Rozdział 2

Słownik pojęć

Pojęcia

AES-GCM (Advanced Encryption Standard w trybie Galois/Counter Mode) to algorytm szyfrowania symetrycznego, który zapewnia zarówno poufność, jak i integralność danych. Jest szeroko stosowany w zabezpieczaniu komunikacji sieciowej i przechowywaniu danych.

Agile Filozofia zarządzania projektami i tworzenia oprogramowania, oparta na iteracyjnym podejściu, współpracy z klientem, elastyczności i ciągłym doskonaleniu produktu

Android Otwartoźródłowy system operacyjny oparty na jądrze Linuksa, stworzony głównie dla urządzeń mobilnych, takich jak smartfony i tablety. Rozwijany przez Google, umożliwia obsługę aplikacji i usług

API Interfejs programistyczny aplikacji; umożliwia komunikację pomiędzy modulem raportowym a panelami użytkowników oraz systemem kontroli dostępu.

artefakt deweloperski Dokumentacja, kod źródłowy, diagramy lub inne materiały tworzone podczas procesu rozwoju oprogramowania, które wspierają zespół deweloperski w realizacji projektu, pozostawione w aplikacji po jej wdrożeniu.

AutoCAD Oprogramowanie do projektowania wspomaganego komputerowo (CAD) używane do tworzenia precyzyjnych rysunków technicznych i modeli 3D.

boolean-based blind SQL Injection Technika ataku SQL Injection, w której atakujący manipuluje zapytaniami SQL, aby uzyskać odpowiedzi typu praw-

da/fałsz, co pozwala na wyciąganie informacji z bazy danych bez bezpośredniego dostępu do wyników zapytań.

brute force Metoda ataku polegająca na systematycznym próbowaniu wszystkich możliwych kombinacji haseł lub kluczy w celu uzyskania nieautoryzowanego dostępu do systemu.

ciąg obecności Koncept weryfikacji obecności wykładowcy, zakładający, że jeśli prowadzi on kilka bloków zajęć z rzędu w tej samej sali, fizyczne otwarcie drzwi jest wymagane tylko przed pierwszym blokiem. Kolejne zajęcia są uznawane za zrealizowane, o ile zachowana jest ciągłość przebywania w sali.

Cyfrowe Klucze System kontroli dostępu do sal w Gdańskim oddziale [PJATK](#), w budynku wydziału Informatyki. Stworzony przez zespół Antoniego Ulenberga, Marka Kudły i Kingi Marszałkowskiej.

Cyfrowe Klucze Mobile Aplikacja mobilna stworzona przez Magdę Megan Kornacką, rozszerzający [System Cyfrowego Dostępu PJATK](#) o weryfikację poprzez kody [QR](#).

Digikey Nazwa zespołu odpowiedzialnego za tworzenie projektu [EkeyPJATK](#)

DOS (Denial of Service) to atak mający na celu uniemożliwienie dostępu do usługi lub zasobu poprzez przeciążenie systemu nadmierną ilością żądań.

EkeyPJATK System zarządzania dostępem rozszerzający i poprawiający system [Cyfrowe Klucze](#), zintegrowany w [Systemie Cyfrowego Dostępu PJATK](#).

Elektrozwoza Urządzenie służące do mechanicznego blokowania i odblokowywania drzwi w systemie kontroli dostępu.

error-based SQL Injection Technika ataku SQL Injection, w której atakujący wykorzystuje komunikaty błędów generowane przez bazę danych do uzyskania informacji o strukturze bazy i danych.

false positive Sytuacja, w której system bezpieczeństwa lub narzędzie do analizy błędnie identyfikuje legalne działanie jako zagrożenie lub błąd.

GAKKO System zarządzania uczelnią, z którego pobierane są dane dotyczące planu zajęć i uprawnień dostępu.

GCM (Galois/Counter Mode) to tryb operacji szyfrowania blokowego, który łączy szyfrowanie w trybie licznika (CTR) z uwierzytelnianiem danych za pomocą kodu MAC. Zapewnia zarówno poufność, jak i integralność danych.

Heksagonalna Architektura oprogramowania, która oddziela logikę biznesową od zewnętrznych interfejsów poprzez definiowanie portów (interfejsów) i adapterów (implementacji). Umożliwia łatwą wymianę komponentów i integrację z różnymi technologiami bez wpływu na rdzeń aplikacji.

Hydra Narzędzie do przeprowadzania ataków siłowych na różne protokoły uwierzytelniania, umożliwiające szybkie testowanie haseł i identyfikatorów użytkowników.

IP (Internet Protocol) to sposób przesyłania danych w sieciach komputerowych z wykorzystaniem protokołu internetowego. Polega na dzieleniu informacji na pakiety, które są adresowane i przesyłane między urządzeniami w sieci, umożliwiając wymianę danych w internecie i sieciach lokalnych.

IPS Technologia stosowana w wyświetlaczach **LCD**, w której ciekłe kryształy ułożone są równolegle do powierzchni ekranu. Dzięki temu ekrany IPS oferują szerokie kąty widzenia i lepsze odwzorowanie kolorów niż tradycyjne matryce **LCD**.

Karta magnetyczna Nośnik identyfikacyjny wykorzystywany przez studentów, wykładowców i gości do autoryzacji dostępu do pomieszczeń uczelni.

LAN (Local Area Network) to lokalna sieć komputerowa, która łączy urządzenia znajdujące się blisko siebie. Umożliwia szybkie przesyłanie danych i współdzielenie zasobów.

LCD Technologia wyświetlania obrazu, w której ciekłe kryształy modulują światło. Jest szeroko stosowana w ekranach urządzeń elektronicznych dzięki niskiej wadze i energooszczędności.

LED Dioda elektroluminescencyjna, która emituje światło, gdy przepływa przez nią prąd elektryczny. Jest energooszczędna, trwała i szeroko stosowana w oświetleniu, wyświetlaczach oraz urządzeniach elektronicznych.

Mock Sztuczna implementacja zależności używana w testach, która imituje zachowanie prawdziwego komponentu. Pozwala kontrolować odpowiedzi i symulować różne scenariusze bez uruchamiania realnych usług.

Moduł Sprzętowy Fizyczny komponent systemu montowany przy salach, odpowiedzialny za odblokowywanie drzwi na podstawie weryfikacji uprawnień. Obsługuje karty [magnetyczne](#) oraz kody [QR](#), pełniąc kluczową rolę w kontroli dostępu w systemie [Cyfrowego Dostępu](#)

MVC (Model-View-Controller) to wzorzec architektoniczny oprogramowania, który dzieli aplikację na trzy główne komponenty: Model (logika biznesowa i dane), Widok (interfejs użytkownika) oraz Kontroler (pośrednik między Modelem a Widokiem). Ułatwia to zarządzanie kodem i rozwój aplikacji.

Nmap Narzędzie do skanowania sieci komputerowych, służące do odkrywania hostów i usług w sieci oraz identyfikacji potencjalnych luk bezpieczeństwa.

nonce (Number Used Once) to unikalna wartość używana tylko raz w procesie szyfrowania lub uwierzytelniania, zapobiegająca powtórным atakom i zapewniająca bezpieczeństwo transmisji danych.

PCB (Printed Circuit Board) to płytką drukowaną, na której montuje się i łączy elementy elektroniczne za pomocą ścieżek przewodzących. Stanowi podstawę większości urządzeń elektronicznych, zapewniając kompaktowe i trwałe połączenia między komponentami.

PJATK Polsko-Japońska Akademia Technik Komputerowych.

PoE (Power over Ethernet) to technologia oparta na kilku standardach przesyłu energii elektrycznej za pomocą skrętki do urządzeń peryferyjnych będących elementami sieci Ethernet. Pozwala dostarczyć napięcie 48V przez port Ethernet.

Prepared Statements Mechanizm w bazach danych, który pozwala na przygotowanie i skompilowanie zapytania SQL z parametrami wejściowymi, oddzielając strukturę zapytania od danych. Zwiększa bezpieczeństwo i wydajność, chroniąc przed atakami SQL Injection.

producer consumer pattern Wzorzec projektowy, w którym jeden lub więcej producentów generuje dane lub zadania, które są następnie konsumowane przez jeden lub więcej konsumentów. Umożliwia asynchroniczne przetwarzanie i efektywne zarządzanie przepływem danych między różnymi komponentami systemu.

Programowanie ekstremalne (Extreme Programming), [zwinna metodyka](#) tworzenia oprogramowania, która stawia na prostotę, częstą komunikację, informację zwrotną i szybkie dostarczanie wartości w krótkich iteracjach.

przetwarzanie wsadowe Technika przetwarzania danych, w której zadania są grupowane i wykonywane jako jedna jednostka (wsad), zamiast przetwarzania pojedynczych elementów jednocześnie. Umożliwia efektywne zarządzanie zasobami i optymalizację wydajności.

QR Dwuwymiarowy kod graficzny w formie kwadratu, który można zeskanować telefonem lub skanerem. Umożliwia szybki dostęp do zapisanych informacji.

RabbitMQ System kolejkowania wiadomości typu open-source, który umożliwia komunikację między różnymi aplikacjami poprzez przesyłanie wiadomości w sposób asynchroniczny.

race condition Sytuacja w programowaniu współbieżnym, w której wynik działania programu zależy od nieprzewidywalnego porządku wykonywania operacji przez różne wątki. Może prowadzić do błędów i niespójności danych, jeśli nie zostaną zastosowane odpowiednie mechanizmy synchronizacji.

Redis Szybka baza danych w pamięci operacyjnej typu klucz-wartość, używana głównie jako cache. Zapewnia bardzo niskie opóźnienia i wysoką wydajność.

REST API Interfejs programistyczny oparty na architekturze REST, umożliwiający komunikację między modulem integracyjnym a systemem [GAKKO](#).

RPC (Remote Procedure Call) to protokół komunikacyjny umożliwiający wywoływanie procedur lub funkcji na zdalnym serwerze tak, jakby były lokalne.

Scrum [Zwinna metodyka](#) zarządzania projektami. Oparta na iteracyjnej pracy w krótkich cyklach zwanych sprintami.

separation of concerns Paradygmat projektowania oprogramowania, który polega na podziale systemu na odrębne moduły lub komponenty, z których każdy odpowiada za konkretną funkcjonalność lub aspekt działania aplikacji. Ułatwia to zarządzanie kodem, jego rozwój i utrzymanie.

snapshot Nieodwracalny zrzut stanu danych w określonym momencie czasu, używany do wersjonowania i przechowywania historii zmian.

SQL Injection Technika ataku na aplikacje webowe, polegająca na wstrzykiwaniu złośliwych poleceń SQL do zapytań bazodanowych w celu uzyskania nieautoryzowanego dostępu do danych lub manipulacji nimi.

SvelteKit Framework do budowy aplikacji webowych oparty na Svelte, oferujący narzędzia do tworzenia wydajnych i skalowalnych aplikacji z obsługą routingu, SSR i innych funkcji.

Swagger Narzędzie do projektowania, budowania, dokumentowania i konsumowania RESTful APIs. Umożliwia tworzenie interaktywnych dokumentacji API.

System Cyfrowego Dostępu PJATK Całościowe rozwiązanie zarządzania dostępem do sal uczelnianych Gdańskiego oddziału [PJATK](#), w budynku wydziału Informatyki. Obejmuje trzy współpracujące podsystemy: [Cyfrowe Klucze](#), [Cyfrowe Klucze Mobile](#) oraz [EkeyPJATK](#). System integruje funkcje rezerwacji, uwierzytelniania i kontroli dostępu, zapewniając spójność danych oraz bezpieczeństwo działania

Table-driven Technika testowania w Go, w której przypadki testowe zapisuje się w formie tabeli z wejściami i oczekiwanymi wynikami. Ułatwia testowanie wielu scenariuszy w jednym teście.

TailwindCSS Framework CSS oparty na podejściu utility-first, umożliwiający szybkie tworzenie responsywnych i dostosowywalnych interfejsów użytkownika poprzez stosowanie gotowych klas CSS.

time-based blind SQL Injection Technika ataku SQL Injection, w której atakujący wykorzystuje opóźnienia w odpowiedziach serwera do wyciągania informacji z bazy danych, gdy bezpośrednie wyświetlanie wyników jest niemożliwe.

TLS (Transport Layer Security) to protokół kryptograficzny zapewniający bezpieczną komunikację w sieci komputerowej poprzez szyfrowanie danych przesyłanych między klientem a serwerem.

TTL (Time To Live) to mechanizm określający czas, przez jaki dana wartość może być przechowywana w systemie, po czym zostaje automatycznie usunięta. Służy do kontrolowania ważności danych i zapobiegania przechowywaniu nieaktualnych informacji.

UI Interfejs użytkownika; graficzne środowisko umożliwiające interakcję z aplikacją webową.

UNION query-based SQL Injection Technika ataku SQL Injection, w której atakujący wykorzystuje operator UNION do łączenia wyników złośliwych zapytań z oryginalnymi zapytaniem, co pozwala na wyciąganie danych z innych tabel w bazie danych.

VESA Standard określający rozstaw otworów montażowych na tylnej obudowie monitorów i telewizorów. Umożliwia łatwe i uniwersalne mocowanie ekranów do uchwytów, stojaków lub ramion zgodnych z tym standardem.

Waterfall Model prowadzenia projektów i tworzenia oprogramowania, w którym prace przebiegają sekwencyjnie. Od analizy wymagań, przez projekt, implementację, testy, aż po wdrożenie. Każdy etap musi zostać zakończony przed rozpoczęciem kolejnego.

Webhook Mechanizm, w którym system wysyła żądanie HTTP do wskazanego adresu URL po wystąpieniu określonego zdarzenia. Umożliwia natychmiastowe, zdarzeniowe przekazywanie danych.

WebSocket Protokół komunikacyjny umożliwiający dwukierunkową komunikację w czasie rzeczywistym między klientem a serwerem.

wzorzec Registry Wzorzec projektowy, który polega na centralnym rejestrowaniu i zarządzaniu obiektami lub usługami w aplikacji. Umożliwia dynamiczne wyszukiwanie i dostęp do zarejestrowanych komponentów bez konieczności bezpośredniego tworzenia instancji.

Akronimy

AV/UC Audio Video / Unified Communications

BMC Business Model Canvas

BSS Baza Sprzętowo-Systemowa

CAD Computer-Aided Design

SIS System Informacji o Salach

Rozdział 3

Opis problemu

3.1 Analiza stanu obecnego

W obecnym systemie zarządzania dostępem do pomieszczeń w budynku filii [PJATK](#) w Gdańsku stosowane są tradycyjne fizyczne klucze, którymi rozporządzają pracownicy ochrony po uprzedniej weryfikacji tożsamości dydaktyka. Osoba chcąc uzyskać dostęp do sali musi zgłosić się na recepcję, zostać zweryfikowana oraz wpisać się do dziennika, a następnie pobrać klucz. Po zakończeniu zajęć klucz należy zwrócić w to samo miejsce. Jak wskazuje dokumentacja projektu, *“w obecnym systemie zarządzania dostępem do pomieszczeń w budynku filii PJATK w Gdańsku występuje szereg niedogodności, które znacząco wpływają na efektywność i precyzję funkcjonowania ”* [[CK](#), rozdział 3].

Uzależnienie otwarcia sal laboratoryjnych od fizycznych kluczy powoduje konieczność pobierania i oddawania kluczy u pracowników ochrony przez wykładowców. Proces ten ogranicza elastyczność i efektywne wykorzystanie czasu między zajęciami, gdyż zazwyczaj musi odbywać się w przerwach, co często wymaga przejścia kilku pięter i dłuższego oczekiwania [[CK](#), rozdział 3].

Pracownicy uczelni, tacy jak wykładowcy, sprzątacze czy administracja budynku, również muszą każdorazowo pobierać klucze, co w praktyce oznacza konieczność częstych wizyt na recepcji lub pobierania kilku kluczy jednocześnie.

3.1.1 Zidentyfikowane problemy

Obecny system generuje szereg trudności organizacyjnych. Dydaktycy, chcąc uprościć sobie pracę, często przekazują klucze między sobą bez wpisywania się do dziennika, co prowadzi do rozbieżności między stanem faktycznym a zapisanym w do-

kumentacji. Istnieje również ryzyko utraty klucza lub przypadkowego zabrania go przez innego wykładowcę, co w rezultacie uniemożliwia dostęp do sali.

Kolejnym problemem jest weryfikacja odbytych zajęć. Jeżeli prowadzący nie wpisał się do dziennika, Dziekanat musi kontaktować się z nim bezpośrednio, aby potwierdzić, czy zajęcia rzeczywiście się odbyły. Proces ten jest czasochłonny, nieefektywny i podatny na błędy. Ponadto w obecnym systemie brakuje przystępnej informacji dotyczącej tego, kto aktualnie korzysta z laboratorium oraz ile czasu pozostało do zakończenia zajęć:

Co więcej, w obecnym systemie brakuje przystępnej informacji dotyczącej tego, kto aktualnie korzysta z laboratorium oraz ile czasu pozostało do zakończenia zajęć [CK, rozdział 3].

Trudności napotykają również pracownicy techniczni, tacy jak sprzątacze czy administracja budynku. Każdorazowe pobieranie kluczy ogranicza ich elastyczność w wykonywaniu obowiązków. Jeżeli decydują się na pobranie wielu kluczy jednocześnie, rośnie ryzyko ich zgubienia, co dodatkowo komplikuje organizację pracy.

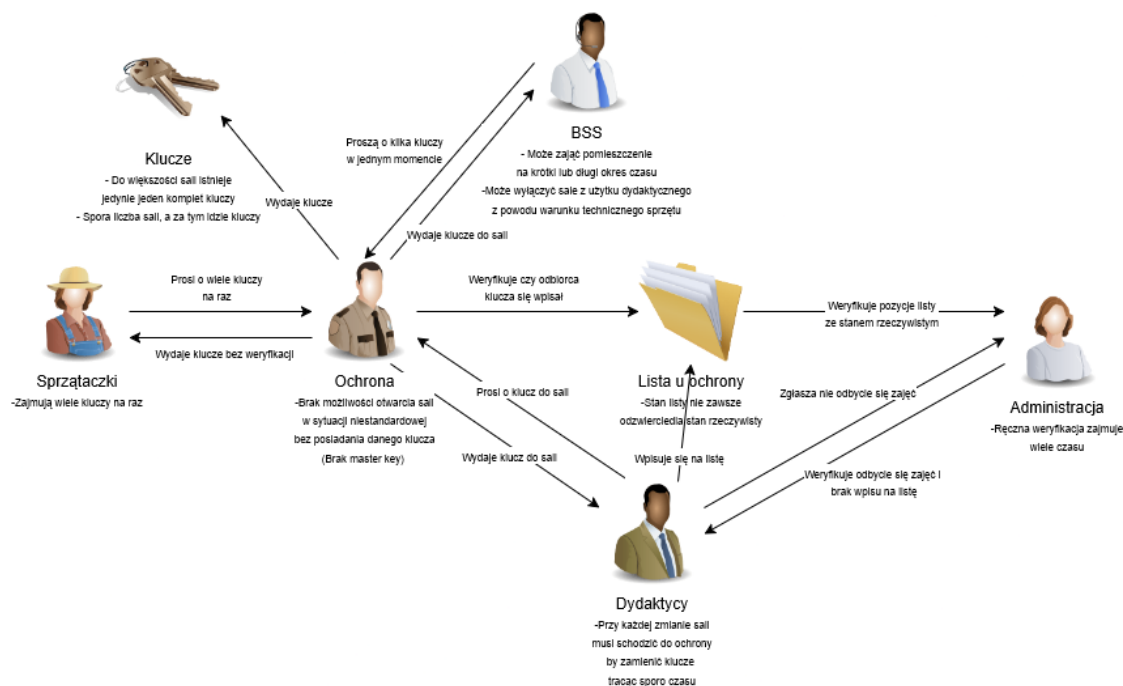
3.1.2 Analiza problemów

Analiza wskazuje, że obecny system oparty na fizycznych kluczach jest mało efektywny i generuje dodatkowe obciążenie organizacyjne. Brak pełnej kontroli nad obiegiem kluczy prowadzi do nieścisłości w dokumentacji i utrudnia rozliczanie odpowiedzialności za poszczególne sale. Obecna papierowa dokumentacja oraz manualnie prowadzone księgi raportowe nie są zintegrowane z używanymi przez uczelnię systemami rezerwacji, co utrudnia spójną i efektywną analizę danych dotyczących odbytych zajęć:

Obecna papierowa dokumentacja oraz manualnie prowadzone księgi raportowe nie są zintegrowane z używanymi przez uczelnię systemami rezerwacji, co utrudnia spójną i efektywną analizę danych dotyczących odbytych zajęć [CK, rozdział 3].

Aby umożliwić dokładną analizę, konieczne jest zdigitalizowanie papierowej dokumentacji i jej integracja z danymi z planu zajęć. Proces weryfikacji zajęć jest podatny na błędy ludzkie i wymaga dodatkowych działań administracyjnych, co spowalnia pracę Dziekanatu. Ograniczona elastyczność pracowników technicznych

wpływa negatywnie na sprawność obsługi budynku i może prowadzić do opóźnień w realizacji zadań.



Rysunek 3.1: Rich Picture ilustrujący obecny system zarządzania dostępem do pomieszczeń w budynku PJATK w Gdańsku.

3.2 Analiza oryginalnego projektu Cyfrowe Klucze (CK)

Projekt [Cyfrowe Klucze](#) stanowił prototypowy system kontroli dostępu do sal wykładowych, opracowany w ramach pracy inżynierskiej. Jego głównym celem było stworzenie zintegrowanego rozwiązania umożliwiającego przyznawanie dostępu do pomieszczeń na podstawie identyfikacji [kartą magnetyczną](#) oraz rezerwacji sal. Jak wskazano w dokumentacji:

Projekt cyfrowe klucze to zintegrowany system kontroli dostępu. W jego skład wchodzi: moduł sprzętowy (mikrokontroler, wyświetlacz dotykowy, czytnik kart), przeglądarkowa aplikacja dla administracji do zarządzania użytkownikami i rezerwacjami oraz panel ochroniarza służący do monitorowania i zdalnego otwierania drzwi w budynku. [CK, rozdział 4.1]

System zakładał również możliwość zdalnego sprawdzania stanu połączenia modułu sprzętowego, otwierania i zamykania drzwi oraz generowania raportów na podstawie danych o użytkowaniu sal. Wśród celów projektu wymieniono:

Stworzenie automatycznego systemu kontroli dostępu zintegrowanego z systemem rezerwacji w webowej aplikacji administracji

Zbieranie danych dotyczących użytkowania sal i generowanie raportów.

[CK, rozdział 4.2]

W podsumowaniu projektu wskazano konkretne kierunki dalszego rozwoju, które mogłyby zwiększyć potencjał wdrożeniowy systemu. Jednym z kluczowych postulatów była integracja z systemem GAKKO, odpowiedzialnym za plan zajęć uczelni. Jak zapisano:

Integracja aplikacji z systemem Gakko oraz planem zajęć umożliwiłaby synchronizowanie rezerwacji sal w proponowanym systemie z aktualnym planem zajęć uczelni. Dzięki temu administracja będzie miała jeden system do wprowadzania zajęć, co usprawni proces planowania, zapewni spójność danych i wyeliminuje potencjalne konflikty terminowe. [CK, rozdział 16.1.2]

3.3 Analiza oryginalnego projektu Cyfrowe Klucze Mobile (CKM)

Projekt [Cyfrowe Klucze Mobile](#) stanowi rozszerzenie systemu kontroli dostępu opracowanego w ramach wcześniejszego projektu [Cyfrowe Klucze](#). Tym razem nacisk położono na mobilność i wygodę użytkownika, w szczególności studentów i wykładowców. Jak wskazano w dokumentacji:

Projekt "Cyfrowe Klucze Mobile" to aplikacja mobilna rozbudowująca system kontroli dostępu o funkcjonalności:

- *Dostęp do sal poprzez skanowanie kodu QR,*
- *Wyświetlanie planu użytkownika z poziomu aplikacji,*
- *Wyświetlanie informacji o poszczególnych rezerwacjach.*

Rozwiązuje to potrzebę nadmierowej ilości kart dostępu oraz ogranicza ryzyko zgubienia karty. [CKM, rozdział 4.1]

Aplikacja mobilna wprowadza nowy kanał interakcji z systemem kontroli dostępu, dodając alternatywe do [kart magnetycznych](#). Zamiast tego użytkownik może zeskanować kod [QR](#), co upraszcza proces weryfikacji i zwiększa dostępność systemu. Dodatkowo, integracja planu zajęć bezpośrednio w aplikacji pozwala użytkownikom na szybki dostęp do informacji o rezerwacjach sal.

W dokumentacji projektu wskazano również potencjalne kierunki dalszego rozwoju aplikacji, które mogłyby zwiększyć jej funkcjonalność i użyteczność w środowisku akademickim. Jak zapisano:

Dalsza praca nad tą aplikacją wiązałaby się z pełną implementacją oraz wprowadzeniem jej w użytek na terenie uczelni, dla studentów oraz prowadzących zajęcia. Dodatkowo, można rozbudować aplikację o nowe funkcje, których nie udało się zaimplementować podczas obecnej pracy:

- *Moduł zgłaszania błędów aplikacji.*
- *Dodawanie rezerwacji do weryfikacji do systemu GAKKO.*

[CKM, rozdział 12.1]

Podobnie jak w przypadku pierwotnego projektu, integracja z systemem GAKKO pozostaje kluczowym elementem dalszego rozwoju. Umożliwienie dodawania rezerwacji do weryfikacji w systemie planowania zajęć pozwoliłoby na pełną synchronizację danych i usprawnienie procesu zarządzania dostępem do sal.

3.4 Analiza konkurencji

W celu określenia pozycji systemu EkeyPJATK na rynku dokonano analizy wybranych rozwiązań konkurencyjnych. Uwzględniono zarówno firmy oferujące kompleksowe systemy rezerwacji i integracji sal konferencyjnych, jak i producentów urządzeń wspierających proces zarządzania dostępem i rezerwacjami. Analiza obejmuje trzy przykładowe rozwiązania: ofertę firmy Tritech, urządzenie Logitech Tap Scheduler oraz system informacyjny Vidikom SIS.

3.4.1 Tritech

Firma Tritech New Technologies Sp. z o.o. z siedzibą w Warszawie działa na rynku od 2003 roku i specjalizuje się w projektowaniu oraz wdrażaniu systemów AV/UC, a także kompleksowych rozwiązań IT. Choć w swojej ofercie nie posiada bezpośrednio dedykowanego systemu kontroli dostępu do sal uczelnianych, jest otwarta na współpracę ze szkołami i uczelniami, dostosowując swoje rozwiązania do indywidualnych potrzeb klientów. Tritech kładzie duży nacisk na staranne przygotowanie projektów, wykorzystując narzędzia do modelowania 3D, takie jak AutoCAD, GstarCAD czy DraftSight. Na etapie projektowania tworzone są schematy blokowe, plany sieci, rozmieszczenie urządzeń oraz trasy kablowe. Firma prowadzi również konsultacje z klientem, architektem, inwestorem i integratorem, aby zapewnić spójność i funkcjonalność całego rozwiązania. W realizacjach wykorzystuje urządzenia renomowanych dostawców, takich jak Logitech, Cisco, Poly czy Yealink.

Zalety

Do głównych zalet oferty Tritech należy elastyczność i profesjonalizm w podejściu do klienta. Firma dostosowuje swoje projekty do budżetu oraz wymagań technicznych, co pozwala na tworzenie rozwiązań dopasowanych do specyfiki danej instytucji. Istotnym atutem jest również przygotowywanie pełnej dokumentacji projektowej w formacie **CAD**, co ułatwia późniejszą realizację i integrację systemów. Dzięki współpracy z wieloma dostawcami sprzętu Tritech może zaoferować szeroki wachlarz rozwiązań technologicznych, które odpowiadają różnym potrzebom klientów.

Wady

Do wad oferty Tritech można zaliczyć brak własnych, autorskich urządzeń, które mogłyby być w pełni dostosowane do indywidualnych wymagań klienta. Firma opiera się na sprzęcie dostarczonym przez zewnętrznych producentów, co oznacza, że rozwiązania te często bazują na zamkniętych systemach wymagających dodatkowych licencji lub subskrypcji. Może to generować dodatkowe koszty oraz ograniczać elastyczność wdrożenia w dłuższej perspektywie.



Rysunek 3.2: Strona główna firmy Tritech [1] .

3.4.2 Logitech Tap Scheduler

Firma Logitech, znana głównie z produkcji akcesoriów komputerowych, posiada w swojej ofercie również urządzenia do rezerwacji sal konferencyjnych. Jednym z nich jest **Logitech Tap Scheduler**, panel planistyczny montowany na ścianie,

wyposażony w 10,1 calowy dotykowy wyświetlacz [LCD](#). Urządzenie jest kompatybilne z wieloma systemami obsługi sal konferencyjnych, takimi jak Microsoft, Zoom, Robin czy też z autorskiego rozwiązania Logitech’a. Tap Scheduler został zaprojektowany z myślą o łatwej instalacji na szkle, futrynie lub ścianie, a kontrolki [LED](#) umożliwiają szybkie sprawdzenie statusu dostępności sali z daleka.

Urządzenie może być zarządzane z poziomu platformy **Logitech Sync**, która pozwala na monitorowanie stanu pomieszczeń, wdrażanie aktualizacji oraz konfigurację ustawień. Każdy produkt zakupiony od Logitech objęty jest 2-letnią gwarancją, z możliwością jej przedłużenia.

Zalety

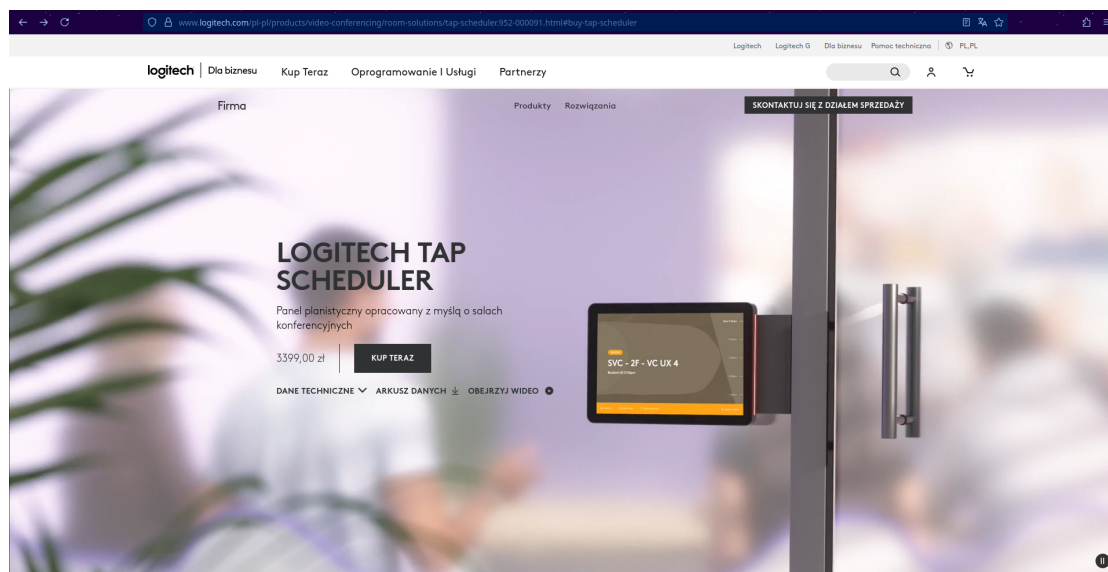
Logitech Tap Scheduler wyposażony jest w czytelny ekran dotykowy o przekątnej 10,1 cala. Jest kompatybilny z popularnymi systemami rezerwacji sal, takimi jak Microsoft, Zoom czy Robin, co czyni go rozwiązaniem uniwersalnym. Dzięki kolorowym kontrolkom [LED](#) użytkownicy mogą z łatwością sprawdzić status dostępności sali nawet z większej odległości. Urządzenie oferuje elastyczne opcje montażu, może być instalowane na szkle, futrynie lub ścianie, a dodatkowe prowadnice w obudowie ułatwiają estetyczne poprowadzenie okablowania. Kolejną zaletą jest integracja z platformą Logitech Sync, która umożliwia centralne zarządzanie urządzeniami i ich konfiguracją. Standardowa 2-letnia gwarancja z możliwością przedłużenia zwiększa bezpieczeństwo inwestycji.

Wady

Do wad urządzenia należy brak natywnej funkcjonalności blokowania sal oraz brak integracji z systemami kontroli dostępu. Logitech Tap Scheduler nie daje również możliwości przeprogramowania w ramach dostępnych systemów rezerwacyjnych, co ogranicza jego elastyczność. Cena jednostkowa wynosząca 3399,00 PLN nie obejmuje kosztów dodatkowych licencji ani subskrypcji Logitech Sync, które są wymagane do pełnego wykorzystania możliwości systemu. Dodatkowo, bardziej zaawansowane funkcje analizy i monitorowania w Logitech Sync są dostępne wyłącznie w droższych planach subskrypcyjnych, co może zwiększać całkowity koszt.

3.4.3 Vidikom

[System Informacji o Salach \(SIS\)](#) firmy Vidikom jest rozwiązaniem dedykowanym dla dużych organizacji, takich jak uczelnie, hotele, centra konferencyjne czy urzędy. Vidikom specjalizuje się w kompleksowym wyposażeniu sal konferencyjnych w



Rysunek 3.3: Logitech Tap Scheduler [2].

sprzęt audiowizualny, oferując również jego instalację i konfigurację. **SIS** jest autorskim produktem firmy, co umożliwia jego szeroką adaptację zarówno pod względem funkcjonalnym, jak i graficznym.

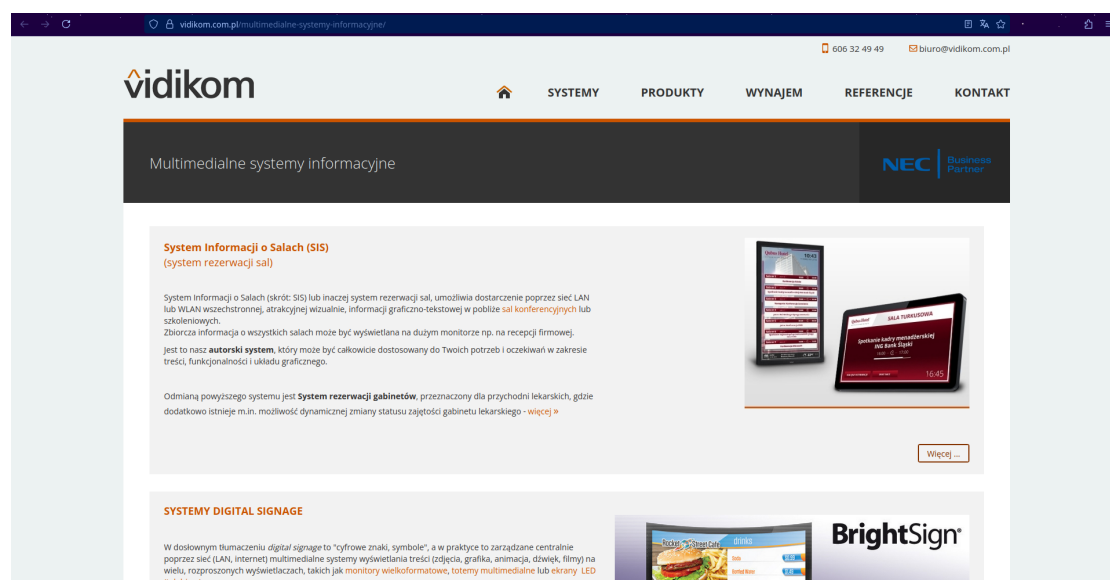
System oparty jest na tabletach z systemem **Android** o rozmiarze ekranu zaczynającym się od 10,1 cala, wyposażonych w wyświetlacze **IPS** o jasności 500 nit, paski **LED** sygnalizujące stan zajętości sali, zasilanie **PoE** oraz mocowanie **VESA**. Panel sterowania dostępny z poziomu sieci **LAN** umożliwia zarządzanie rezerwacjami, konfigurację sal oraz kontrolę dostępu. **SIS** wspiera integrację z popularnymi kalendarzami takimi jak Google Calendar, Outlook Exchange, Lotus Domino oraz może być połączony z zewnętrznymi niezabezpieczonymi bazami danych.

Zalety

System **SIS** charakteryzuje się otwartą strukturą, co pozwala na szeroką personalizację zarówno funkcji, jak i wyglądu interfejsu użytkownika. Urządzenia wykorzystywane w systemie są wyposażone w jasne ekrany **IPS** oraz paski **LED**, które umożliwiają szybką identyfikację stanu zajętości sali. Zastosowanie zasilania **PoE** upraszcza instalację i ogranicza liczbę przewodów. Panel sterowania dostępny przez sieć **LAN** jest intuicyjny i umożliwia kompleksowe zarządzanie salami, użytkownikami oraz rezerwacjami. System oferuje również możliwość integracji z popularnymi kalendarzami oraz z zewnętrznymi bazami danych, co usprawnia proces rezerwacji i synchronizacji informacji.

Wady

System **SIS** nie posiada natywnego wsparcia dla szyfrowanego dostępu do zewnętrznych baz danych, co może stanowić ograniczenie w kontekście bezpieczeństwa informacji. Główna funkcjonalność systemu koncentruje się na zarządzaniu rezerwacjami i prezentacji informacji, natomiast nie oferuje dedykowanego mechanizmu fizycznej kontroli dostępu, takiego jak identyfikacja użytkowników za pomocą **kart magnetycznych**. Zarządzanie użytkownikami oraz salami odbywa się ręcznie poprzez panel **LAN**, co może być czasochłonne w przypadku dużych instytucji. Ponadto, zależność od sprzętu z systemem **Android** może ograniczać elastyczność wdrożenia w środowiskach o innych wymaganiach technicznych lub preferencjach sprzętowych.



Rysunek 3.4: Vidikom System Informacji o Salach [3].

3.5 Propozycja rozwiązania

System **EkeyPJATK** został zaprojektowany jako rozwinięcie koncepcji przedstawionej w projektach **Cyfrowe Klucze** oraz **Cyfrowe Klucze Mobile**. Choć nie stanowi ich bezpośredniej kontynuacji, realizuje kluczowe postulaty rozwojowe zawarte w dokumentacji obu systemów, przekształcając prototyp w rozwiązanie gotowe do wdrożenia. Najważniejszym usprawnieniem jest pełna integracja z systemem **GAKKO**, która w oryginalnych projektach była jedynie propozycją. **Ekey** automatycznie synchronizuje dostęp do sal z harmonogramem zajęć, co eliminuje konieczność ręcznego zarządzania rezerwacjami i znacząco zwiększa spójność danych oraz bezpieczeństwo.

Kluczowym elementem systemu jest wykorzystanie kodów [QR](#) jako alternatywy dla [kart magnetycznych](#). [Ekey](#) generuje dynamiczny kod [QR](#), który użytkownik może zeskanować za pomocą aplikacji [CKM](#). Aplikacja mobilna, po zeskanowaniu kodu [QR](#), przesyła dane identyfikacyjne użytkownika do [Ekey](#), który weryfikuje zgodność z planem zajęć. Na podstawie tej analizy system podejmuje decyzję o przyznaniu dostępu i przekazuje ją do systemu [CK](#), który fizycznie otwiera lub blokuje drzwi. Taki podział odpowiedzialności zapewnia elastyczność, bezpieczeństwo i możliwość dalszej rozbudowy systemu.

W porównaniu do rozwiązań konkurencyjnych, takich jak systemy oferowane przez firmy Tritech, Logitech czy Vidikom, [EkeyPJATK](#) wyróżnia się zakresem automatyzacji oraz bezpośrednią integracją z uczelnianym planem zajęć. Tritech, mimo profesjonalnego podejścia do projektowania i elastyczności w dostosowywaniu rozwiązań, opiera się na sprzęcie zewnętrznych producentów i zamkniętych systemach wymagających dodatkowych licencji, co generuje koszty i ogranicza elastyczność. Logitech Tap Scheduler zapewnia wygodny interfejs rezerwacji i integrację z popularnymi kalendarzami, lecz nie oferuje natywnej kontroli dostępu ani możliwości blokowania sal. Z kolei system Vidikom [SIS](#) koncentruje się na zarządzaniu rezerwacjami i prezentacji informacji, ale nie posiada mechanizmu fizycznej kontroli dostępu, np. poprzez [karty magnetyczne](#). W tym kontekście [Ekey](#) łączy zalety konkurencyjnych rozwiązań, eliminując jednocześnie ich ograniczenia.

Dodatkowym atutem systemu jest odświeżenie paneli zarządzania dla dziekanatu i administracji. Panele te zostały zaprojektowane, a potem dopracowane we współpracy z pracownikami uczelni, co zapewnia ich intuicyjność i dopasowanie do codziennych potrzeb użytkowników. Dzięki nim możliwe jest nie tylko monitorowanie i kontrola dostępu, ale także generowanie raportów na podstawie danych i logów systemowych. Funkcjonalność ta stanowi istotną wartość dodaną, której brakuje w wielu komercyjnych rozwiązaniach.

Istotnym elementem propozycji jest również niski koszt wdrożenia i utrzymania systemu. Podczas gdy rozwiązania komercyjne, takie jak te od Logitecha, wiążą się z wysokimi kosztami zakupu urządzeń, licencji i subskrypcji, [Ekey](#) został opracowany przez zespół studentów, co pozwoliło znacząco ograniczyć koszty prototypowania i wdrożenia. Dzięki temu system jest bardziej dostępny dla instytucji edukacyjnych o ograniczonym budżecie, a jednocześnie oferuje funkcjonalności odpowiadające na realne potrzeby uczelni.

Rozdział 4

Konteksty projektu

4.1 Kontekst systemowy

System [EkeyPJATK](#) został wdrożony jako element większego [Systemu Cyfrowego Dostępu](#), będący infrastrukturą kontroli dostępu na uczelni, wspierającą zarządzanie dostępem do sal dydaktycznych w sposób zgodny z bieżącym harmonogramem zajęć. Głównym zadaniem [Ekey](#) jest automatyzacja decyzji o wejściu do sali, przy jednoczesnym zachowaniu kompatybilności z istniejącym systemem identyfikacji użytkowników opartym o [karty magnetyczne](#). Pełna integracja z systemem [GAK-KO](#) zapewnia synchronizację dostępu do sal z planem zajęć.

Rozwiązanie działa w oparciu o dedykowane [urządzenia brzegowe](#), które komunikują się z [centralną warstwą decyzyjną](#). Urządzenia te wyposażone są w ekran dotykowy i zasilane w standardzie [PoE](#), a ich rolą jest obsługa interakcji użytkownika oraz przekazywanie danych do [Systemu Cyfrowego Dostępu](#). System [Ekey](#) uwzględnia kontekst czasowy i przypisanie do sal, a także integruje się z zewnętrznymi źródłami danych, takimi jak plan zajęć. Weryfikacja tożsamości użytkownika realizowana jest przez system [Cyfrowe Klucze](#), natomiast [EkeyPJATK](#) odpowiada za ocenę uprawnień w danym momencie i miejscu.

Oprócz [kart magnetycznych](#), system obsługuje również dynamiczne kody [QR](#) generowane przez [Ekey](#). Użytkownik może zeskanować kod za pomocą aplikacji [Cyfrowe Klucze Mobile](#), która przesyła dane identyfikacyjne spowrotem. Na tej podstawie [Ekey](#) podejmuje decyzję o przyznaniu dostępu i przekazuje ją do [Cyfrowe Klucze](#), które fizycznie otwierają lub blokują drzwi.

Dla administracji uczelnianej przygotowano odświeżone panele zarządzania, opracowane we współpracy z dziekanatem i administracją. Interfejsy te umożliwiają

monitorowanie, kontrolę dostępu i generowanie raportów na podstawie logów systemowych.

4.1.1 Zakres i granice systemu EkeyPJATK

Cel systemu

Celem systemu EkeyPJATK jest umożliwienie podejmowania automatycznych decyzji o dostępie do sal dydaktycznych na podstawie danych pochodzących z harmonogramu zajęć. Ekey działa w pełnej integracji z istniejącym mechanizmem identyfikacji użytkowników.

W zakresie

System EkeyPJATK obejmuje urządzenia brzegowe wyposażone w ekran dotykowy, które są zasilane w standardzie PoE. Urządzenia te obsługują interakcję z użytkownikiem i przekazują dane do centralnej warstwy decyzyjnej. W ramach systemu SCD działa mechanizm regularnego pobierania i przetwarzania danych z planu zajęć, który pozwala na bieżąco aktualizować informacje o dostępności sal. Warstwa decyzyjna systemu uwzględnia czas, miejsce oraz rolę użytkownika, aby określić jego uprawnienia. System obsługuje zarówno karty magnetyczne, jak i dynamiczne kody QR generowane przez Ekey. Decyzja o przyznaniu dostępu jest przekazywana do systemu CK, który realizuje finalną autoryzację nośnika i fizyczne otwarcie drzwi. W zakresie systemu Ekey znajdują się również interfejsy webowe przeznaczone dla dziekanatu i administracji, które umożliwiają monitorowanie działania systemu SCD, kontrolę dostępu oraz generowanie raportów na podstawie logów.

Poza zakresem

Poza zakresem systemu EkeyPJATK pozostaje właściwa weryfikacja tożsamości użytkownika, która jest realizowana przez system CK. System nie obejmuje także interfejsu webowego przeznaczonego dla ochrony oraz rozwoju i utrzymania aplikacji mobilnej CKM, która stanowi komponent zewnętrzny. W ramach Ekey przewidziano jedynie generowanie i obsługę kodów QR, natomiast sama aplikacja CKM nie jest częścią systemu.

Założenia i ograniczenia

Założono, że uczelnia dysponuje stabilnym źródłem danych o planie zajęć, które może być regularnie pobierane i przetwarzane przez system Ekey. System wymaga

infrastruktury sieciowej zapewniającej zasilanie w standardzie PoE oraz komunikację IP pomiędzy urządzeniami brzegowymi a centralną warstwą decyzyjną. Decyzje o dostępie są podejmowane wyłącznie na podstawie danych z harmonogramu zajęć i przekazywane do systemu Cyfrowe Klucze jako sygnał wykonawczy. Ciągłość działania systemu nie może zależeć od ręcznej rekonfiguracji reguł, ponieważ proces decyzyjny musi być w pełni zautomatyzowany.

4.1.2 Aktorzy i systemy zewnętrzne

Aktorzy

- **Aktorzy osobowi:**

- **Dziekanat i administracja** - odpowiadają za zarządzanie dostępem, nadzór nad systemem oraz generowanie raportów.
- **Goście** - uzyskują dostęp do sal na podstawie tymczasowych uprawnień nadanych przez administrację lub ochronę.
- **IT/BSS** - odpowiadają za utrzymanie infrastruktury sieciowej oraz bezpieczeństwo systemu.
- **Personel techniczny i sprzątający** - posiadają dostęp do sal w ramach wykonywanych obowiązków.
- **Prowadzący** - posiadają dostęp do sal zgodnie z przypisanym harmonogramem zajęć.
- **Studenci** - korzystają z sal otwieranych zgodnie z planem zajęć.
- **Ochrona** - zapewnia wsparcie operacyjne, w tym wydawanie kart tymczasowych i obsługę wyjątkowych sytuacji.

- **Aktorzy systemowi:**

- **System Cyfrowe Klucze** - warstwa sprzętowa wykonująca fizyczne otwarcie drzwi.
- **System GAKKO** - system źródłowy dostarczający harmonogram zajęć.

Systemy i komponenty

Dostawca planu zajęć - źródło danych harmonogramowych wykorzystywanych przez system.

EkeyPJATK - warstwa decyzyjna odpowiedzialna za ocenę uprawnień i synchronizację z planem zajęć.

Cyfrowe Klucze - system realizujący weryfikację tożsamości użytkowników.

Czytnik EkeyPJATK - urządzenie brzegowe obsługujące interakcję z użytkownikiem i sterujące dostępem.

Zamek/elektrozwo - fizyczny mechanizm kontroli wejścia do sal dydaktycznych.

4.1.3 Scenariusze interakcji użytkowników z systemem

Aby w pełni zrozumieć sposób działania systemu oraz zależności pomiędzy jego komponentami, konieczne jest przedstawienie kluczowych scenariuszy operacyjnych. Pokazują one, jak w praktyce przebiega przepływ informacji pomiędzy użytkownikami, urządzeniami brzegowymi, systemem [EkeyPJATK](#) oraz istniejącą infrastrukturą [Cyfrowe Klucze](#). W tym miejscu kontekstu systemowego skupiamy się wyłącznie na logice procesu i wymianie danych - bez wchodzenia jeszcze w szczegóły implementacyjne czy architekturę wewnętrzną. Opisane scenariusze mają za zadanie zobrazować, jak system reaguje na realne działania użytkowników oraz jakie decyzje podejmuje na podstawie harmonogramu i polityk dostępu.

Scenariusz 1: Wejście przy użyciu [karty magnetycznej](#)

1. System [EkeyPJATK](#) synchronizuje dane z planu zajęć.
2. Użytkownik przykłada kartę [kartę magnetyczną](#) do czytnika.
3. Czytnik przesyła dane identyfikacyjne do [EkeyPJATK](#).
4. System ocenia uprawnienia na podstawie harmonogramu i roli użytkownika.
5. Decyzja o dostępie przekazywana jest do [Cyfrowe Klucze](#).
6. Drzwi zostają fizycznie otwarte, a zdarzenie zapisane w logach.

Scenariusz 2: Wejście przy użyciu aplikacji mobilnej i kodu [QR](#)

1. System [EkeyPJATK](#) generuje dynamiczny kod [QR](#) powiązany z salą i czasem zajęć.
2. Użytkownik skanuje kod [QR](#) za pomocą aplikacji [Cyfrowe Klucze Mobile](#).
3. Aplikacja przesyła dane identyfikacyjne użytkownika do [EkeyPJATK](#).
4. System weryfikuje zgodność danych z planem zajęć i ocenia uprawnienia.

5. Decyzja o dostępie przekazywana jest do [Cyfrowe Klucze](#).
6. Drzwi zostają fizycznie otwarte, a zdarzenie zapisane w logach.

Scenariusz 3: Dostęp gościa z uprawnieniami tymczasowymi

1. Administracja lub ochrona nadaje gościowi tymczasowe uprawnienia w systemie.
2. Gość otrzymuje kartę tymczasową.
3. Próba wejścia jest rejestrowana przez czytnik.
4. [EkeyPJATK](#) ocenia uprawnienia na podstawie przydzielonych reguł.
5. Decyzja o dostępie przekazywana jest do [Cyfrowe Klucze](#).
6. Drzwi zostają fizycznie otwarte, a zdarzenie zapisane w logach.

4.1.4 Różnice względem prototypu Cyfrowe Klucze

W porównaniu z prototypem [Cyfrowe Klucze](#), system [EkeyPJATK](#) wprowadza kilka istotnych zmian i usprawnień. Po pierwsze, decyzje o dostępie są w pełni zautomatyzowane i wynikają bezpośrednio z aktualnego planu zajęć. Oznacza to, że system nie opiera się już na ręcznie utrzymywanych listach uprawnień, lecz na danych harmonogramowych synchronizowanych z systemem [GAKKO](#).

Po drugie, architektura sprzętowa została rozszerzona o nowe dedykowane [urządzenia brzegowe](#), które są zasilane w standardzie [PoE](#). Urządzenia te posiadają własne [PCB](#) oraz ekran dotykowy, co umożliwia ich seryjne wdrożenie w budynkach uczelni i zapewnia intuicyjną obsługę dla użytkowników.

Po trzecie, system udostępnia rozbudowane narzędzia administracyjne przeznaczone dla dziekanatu i administracji. Interfejsy te wspierają obsługę wyjątkowych sytuacji, codzienną eksploatację oraz umożliwiają generowanie raportów na podstawie logów systemowych. Obsługa systemu nie wymaga specjalistycznych szkoleń technicznych, dzięki czemu jest bardziej przystępna dla pracowników uczelni.

4.1.5 Wymogi BHP i bezpieczeństwa ewakuacji

Projekt od początku zakłada zgodność z wymaganiami dla urządzeń kontroli dostępu stosowanych na drogach ewakuacyjnych. Docelowy tryb pracy zostanie dobrany na etapie doboru sprzętu, tak aby w scenariuszach awaryjnych zachować możliwość

natychmiastowego opuszczenia obszaru. Po stronie wnętrza przewiduje się element awaryjnego wyjścia umożliwiający natychmiastowe otwarcie drzwi niezależnie od decyzji systemu informatycznego, dostępności sieci czy opóźnień oprogramowania. Logika ewakuacyjna ma priorytet nad logiką kontroli dostępu.

4.2 Udziałowcy

Niniejszy podrozdział identyfikuje i opisuje kluczowych interesariuszy, czyli osoby, grupy lub organizacje mające wpływ na system [EkeyPJATK](#), w odróżnieniu od aktorów, którzy skupiają się na bezpośrednich interakcjach z systemem. Interesariusze ci stanowią główne źródło wymagań, a także będą docelowymi użytkownikami i podmiotami odpowiedzialnymi za utrzymanie systemu.

Wśród kluczowych udziałowców wyróżniono:

Tabela 4.1: Dziekanat

KARTA UDZIAŁOWCA	
Identyfikator:	UOB01
Nazwa::	Dziekanat
Opis::	Jednostka odpowiedzialna za prowadzenie harmonogramu zajęć, wprowadzaniem tymczasowych zmian do planu takich jak zastępstwa i odrabianie zajęć.
Typ udziałowca::	ożywiony, bezpośredni
Punkt widzenia::	perspektywa administracyjna i dydaktyczna
Ograniczenia::	Nie odpowiada za kwestie techniczne ani bezpieczeństwo infrastruktury, ważne jest dla nich aby zajęcia odbywały się zgodnie z planem.
Wymagania::	WO1 , WO3 , WF-GAKKO-2 , NFO2 , IO4

Tabela 4.2: Administracja Uczelni

KARTA UDZIAŁOWCA	
Identyfikator:	UOB02
Nazwa::	Administracja Uczelni
Opis::	Odpowiada za procesy organizacyjne i kadrowe, w tym rozliczanie pracowników dydaktycznych. Korzysta z raportów dotyczących odbytych zajęć, aby realizować obowiązki rozliczeniowe. Wspiera organizację wydarzeń.
Typ udziałowca::	ożywiony, pośredni
Punkt widzenia::	perspektywa organizacyjna i administracyjna
Ograniczenia::	Nie odpowiada za kwestie techniczne ani bezpieczeństwo infrastruktury, a jego rola ogranicza się do obsługi procesów organizacyjnych i kadrowych.
Wymagania::	WO4, NFO2

Tabela 4.3: Promotor

KARTA UDZIAŁOWCA	
Identyfikator:	UOB03
Nazwa::	Promotor
Opis::	Osoba odpowiedzialna za nadzór merytoryczny nad projektem, wspierająca zespół Ekey w zakresie poprawności koncepcyjnej, zgodności z wymaganiami dydaktycznymi oraz jakości dokumentacji. Nadzoruje integrację z podsystemami Systemu Cyfrowego Dostępu .
Typ udziałowca::	ożywiony, pośredni
Punkt widzenia::	perspektywa dydaktyczna i nadzorcza
Ograniczenia::	Nie odpowiada za kwestie techniczne, bezpieczeństwo ani integrację zewnętrzną, ponieważ jego rola ogranicza się do nadzoru merytorycznego i dydaktycznego.
Wymagania::	WO2

Tabela 4.4: Recenzent

KARTA UDZIAŁOWCA	
Identyfikator:	UOB04
Nazwa::	Recenzent
Opis::	Osoba oceniająca projekt EkeyPJATK pod względem merytorycznym, formalnym i zgodności z wymaganiami dydaktycznymi. Analizuje dokumentację, architekturę systemu oraz poprawność realizacji założeń projektowych. Weryfikuje jakość wykonania i spójność rozwiązań z celami pracy.
Typ udziałowca::	ożywiony, bezpośredni
Punkt widzenia::	perspektywa dydaktyczna i oceniająca
Ograniczenia::	Nie definiuje wymagań technicznych ani funkcjonalnych systemu, nie ingeruje w implementację. Jego rola ogranicza się do oceny zgodności projektu z założeniami i standardami akademickimi.
Wymagania::	

Tabela 4.5: Zespół Ekey

KARTA UDZIAŁOWCA	
Identyfikator:	UOB05
Nazwa::	Zespół Ekey
Opis::	Grupa projektowa odpowiedzialna za projektowanie, rozwój i utrzymanie systemu EkeyPJATK , w tym logikę decyzyjną, walidację danych, obsługę błędów oraz integrację interfejsów.
Typ udziałowca::	ożywiony, bezpośredni
Punkt widzenia::	perspektywa organizacyjna i dydaktyczna
Ograniczenia::	Nie definiuje wymagań instytucjonalnych ani finansowych, skupia się na architekturze i implementacji.
Wymagania::	WO2 , WO3 , WF-GAKKO-1 , WF-GAKKO-2 , WF-GAKKO-3 , WF-ACCESS-1 , WF-ACCESS-2 , WF-ACCESS-3 , WF-ACCESS-5 , WF-MAG-1 , WF-MAG-2 , WF-MAG-4 , WF-MAG-5 , NFO3 , NFO4 , IO1 , IO2 , IO3 , IO4 , IO5

Tabela 4.6: BSS

KARTA UDZIAŁOWCA	
Identyfikator:	UOB06
Nazwa::	BSS
Opis::	Jednostka odpowiedzialna za bezpieczeństwo systemu, zgodność z politykami uczelni oraz utrzymanie infrastruktury technicznej i sieciowej. Wspiera integrację projektowanego podsystemu z istniejącym prototypem systemu kontroli dostępu oraz nadzoruje standardy zasilania i komunikacji, w tym wdrożenie PoE
Typ udziałowca::	ożywiony, bezpośredni
Punkt widzenia::	perspektywa bezpieczeństwa i infrastruktury
Ograniczenia::	Nie ingeruje w logikę działania systemu, ponieważ jego rola ogranicza się do definiowania wymagań bezpieczeństwa i zgodności infrastrukturalnej.
Wymagania::	WO4 , WO5 , WF-GAKKO-3 , WF-ACCESS-2 , WF-ACCESS-3 , NFO1 , NFO3 , NFO6

Tabela 4.7: Zespół Cyfrowe Klucze Mobile

KARTA UDZIAŁOWCA	
Identyfikator:	UOB07
Nazwa::	Zespół Cyfrowe Klucze Mobile
Opis::	Zespół odpowiedzialny za rozwój aplikacji mobilnej umożliwiającej otwieranie sal poprzez skanowanie kodów QR generowanych i wyświetlanych na urządzeniach zamontowanych przy salach oraz za integrację z systemem EkeyPJATK .
Typ udziałowca::	ożywiony, pośredni
Punkt widzenia::	perspektywa mobilna i integracyjna
Ograniczenia::	Nie odpowiada za harmonogram zajęć, logikę przydzielania uprawnień ani polityki bezpieczeństwa uczelni.
Wymagania::	WF-ACCESS-1 , IO5

Tabela 4.8: Zespół Cyfrowe Klucze

KARTA UDZIAŁOWCA	
Identyfikator:	UOB08
Nazwa::	Zespół Cyfrowe Klucze
Opis::	Zespół odpowiedzialny za rozwój systemu CK, integrację warstwy sprzętowej oraz ostateczną weryfikację kart magnetycznych prowadzącą do fizycznego odblokowania drzwi.
Typ udziałowca::	ożywiony, pośredni
Punkt widzenia::	perspektywa integracyjna i sprzętowa
Ograniczenia::	Nie odpowiada za harmonogram zajęć ani za przetwarzanie i walidację danych dostarczanych przez GAKKO.
Wymagania::	WF-ACCESS-1, WF-ACCESS-2, WF-ACCESS-4, WF-MAG-1, WF-MAG-2, WF-MAG-3, NFO3, NFO5, IO1, IO2

Tabela 4.9: Personel techniczny

KARTA UDZIAŁOWCA	
Identyfikator:	UOB09
Nazwa::	Personel techniczny
Opis::	Zespół odpowiedzialny za utrzymanie sal w dobrym stanie technicznym, obejmujące naprawy wyposażenia, kontrolę infrastruktury oraz reagowanie na zgłoszenia dotyczące stanu pomieszczeń.
Typ udziałowca::	ożywiony, pośredni
Punkt widzenia::	perspektywa utrzymaniowa i infrastrukturalna
Ograniczenia::	Nie odpowiada za logikę systemu ani procesy przydzielania uprawnień, ponieważ jego rola ogranicza się do obsługi fizycznej infrastruktury sal.
Wymagania::	WO5, NFO1

Tabela 4.10: GAKKO

KARTA UDZIAŁOWCA	
Identyfikator:	UOB10
Nazwa::	GAKKO
Opis::	Zewnętrzny dostawca danych harmonogramowych odpowiedzialny za udostępnianie i utrzymanie API GAKKO oraz zapewnienie wsparcia integracyjnego dla systemów uczelnianych.
Typ udziałowca::	ożywiony, pośredni
Punkt widzenia::	perspektywa techniczna i integracyjna
Ograniczenia::	Nie odpowiada za bezpieczeństwo uczelniane ani za przetwarzanie danych po stronie uczelni, ponieważ jego rola ogranicza się do utrzymania poprawności i dostępności API .
Wymagania::	WO1 , WO3 , WF-GAKKO-1 , NFO4 , IO3

4.3 Kontekst biznesowy

4.3.1 Wstęp

Projekt [EkeyPJATK](#) został opracowany dla konkretnych potrzeb [PJATK](#) i działa w jej technicznym oraz organizacyjnym otoczeniu. Zależy od dostępu do planu zajęć, infrastruktury sieciowej oraz lokalnych procedur bezpieczeństwa. Mimo że komercjalizacja rozwiązania byłaby ryzykowna ze względu na potrzebę adaptacji do każdego nowego kampusu, projekt ma wymierną wartość: porządkuje dostęp do sal, ogranicza koszty obsługi fizycznych kluczy i poprawia rozliczalność zajęć.

4.3.2 Zastosowanie Business Model Canvas

Używamy [BMC](#) do precyzyjnego uchwycenia wewnętrznego modelu wdrożenia: kto jest decydem, jaką wartość dostarczamy, jakie zasoby są krytyczne oraz jakie są koszty. Taki ogląd pozwala zaplanować przekazanie rozwiązania do eksploatacji, oszacować całkowity koszt posiadania i określić mierzalne efekty, takie jak skrócenie czasu otwarcia sal.

4.3.3 Propozycja wartości i segmenty klientów

Bezpośrednim klientem jest [PJATK](#). System eliminuje manualne wydawanie kluczy, wiąże decyzję o dostępie z planem zajęć, a logi udostępnia do raportowania. Wartość dla dziekanatu to klarowność i skrócenie czasu wyjaśnień, dla ochrony mniej interwencji, dla prowadzących wejście bez formalności, dla IT przewidywalna integracja i nadzór.

4.3.4 Kanały i relacje

Po wdrożeniu kanałem utrzymania są ustalone procedury serwisowe i kontakt do IT uczelni. Komunikacja z użytkownikami odbywa się przez istniejące kanały uczelni. Wdrożenie ma charakter współtworzenia z zespołem dziekanatu, ochrony i IT, zaś po uruchomieniu zapewnione jest wsparcie operacyjne i samoobsługowe panele administracyjne.

4.3.5 Strumienie przychodów / korzyści

W ramach wdrożenia wewnętrznego główne „przychody” to korzyści organizacyjne: krótszy czas otwarcia sal, mniej interwencji ochrony, mniejsza liczba wyjaśnień po stronie dziekanatu i lepszy wgląd w faktyczne ramy czasowe zajęć. W modelu zewnętrznym możliwe są dwa klasyczne strumienie: jednorazowa sprzedaż urządzeń

końcowych oraz licencjonowanie oprogramowania. Dodatkowe źródła, jak ekspozycja treści na ekranach, traktujemy jako opcjonalne i zależne od polityk instytucji.

4.3.6 Kluczowe zasoby

Po stronie technicznej kluczowe są: wiedza o systemach wbudowanych i elektronice, kompetencje web/DevOps oraz dostęp do infrastruktury uczelni. Po stronie organizacyjnej istotna jest ścieżka decyzyjna i zaangażowanie interesariuszy. Zasobem materialnym są prototypy i serie urządzeń oraz środowiska serwerowe.

4.3.7 Kluczowe działania

Najważniejsze działania to wytwarzanie i testy urządzeń, integracja z dostawcą planu zajęć, konfiguracja środowisk na uczelni, przygotowanie paneli i szkolenie użytkowników. Po starcie systemu dochodzi monitoring, serwis i regularne przeglądy bezpieczeństwa, w tym testy awaryjnego otwarcia i zgodności z politykami.

4.3.8 Kluczowe partnerstwa

Podstawowym partnerem jest [PJATK](#) jako właściciel procesów i infrastruktury. Dodatkowymi partnerami mogą być dostawcy [PCB](#), firmy montażowe oraz dostawca danych o harmonogramie. W razie skalowania poza uczelnię potrzebni będą integratorzy odpowiedzialni za adaptacje do lokalnych systemów i norm.

4.3.9 Struktura kosztów

W kosztach dominuje hardware, montaż u klienta oraz integracja i testy. Po stronie oprogramowania koszty to rozwój, utrzymanie i aktualizacje. W przypadku postawienia serwera dla klienta mogą wystąpić jednorazowe nakłady na sprzęt serwerowy; w modelu uczelnianym typowe są koszty operacyjne związane z energią i serwisem, przy czym oszczędności czasu pracy administracji i ochrony kompensują część wydatków w całkowitym koszcie posiadania.

4.4 Kontekst społeczny

4.4.1 Interesariusze i role

System [EkeyPJATK](#) jest rozwiązaniem, które wpływa na codzienne funkcjonowanie całej społeczności akademickiej. Nie działa w izolacji, jego wdrożenie modyfikuje sposób poruszania się po budynku, dostęp do sal oraz praktyki administracyjne. Dlatego ważne jest, aby rozumieć, jak różne grupy użytkowników wchodzi

z nim w interakcje i jakie będą tego konsekwencje. Wykładowcy korzystać będą z systemu przede wszystkim w celu płynnego dostępu do przypisanych im sal i uniknięcia ręcznego pobierania kluczy. Studenci, mimo że nie otwierają zamków, pośrednio odczuwają skutki działania systemu przez poprawę organizacji zajęć, możliwość weryfikacji informacji oraz ograniczenie chaosu związanego z pomyłkami sal. Pracownicy dziekanatu i administracji zyskują narzędzia do monitorowania obłożenia sal oraz generowania raportów wspierających planowanie dydaktyczne, natomiast ochrona korzysta z funkcji zdalnego otwierania i podglądu bieżących zdarzeń. Dział IT/BSS odpowiada za utrzymanie, bezpieczeństwo oraz wsparcie techniczne. Grupami uzupełniającymi są pracownicy techniczni i sprzątający, a także goście, dla których system tworzy ograniczony czasowo model dostępu.

4.4.2 Zmiany i spodziewane skutki

Wdrożenie [EkeyPJATK](#) wprowadza widoczne zmiany w życiu uczelni. Najbardziej odczuwalna jest automatyzacja, dostęp do pomieszczeń nie wymaga już ręcznego wydawania kluczy. Dla wykładowców oznacza to mniej przestojów, a dla studentów mniejszą liczbę opóźnień. Pozytywne skutki obejmują poprawę płynności operacyjnej, większą przewidywalność harmonogramu oraz możliwość bieżącego monitorowania wykorzystania przestrzeni dydaktycznej. Dziekanat zyskuje dane, które mogą wspierać procesy administracyjne, a ochrona może skoncentrować się na działaniach związanych z bezpieczeństwem zamiast na manualnym otwieraniu drzwi. Równocześnie pojawiają się możliwe skutki negatywne. Automatyzacja dostępu wymaga przyzwyczajenia się do nowego modelu działania oraz konsekwentnego utrzymywania danych w systemach źródłowych. Nagłe zmiany lub sytuacje wyjątkowe mogą generować chwilową frustrację użytkowników jeśli nie zostaną odpowiednio obsłużone. Infrastruktura techniczna wymaga stałego nadzoru, a pracownicy obsługujący system odpowiednich szkoleń. Sama obecność logów wejść może wzbudzić obawy dotyczące prywatności, co wymaga jasnych zasad i transparentnej komunikacji.

4.4.3 Transparentność i komunikacja

Skuteczne wdrożenie wymaga przejrzystej komunikacji z całą społecznością uczelni. Użytkownicy powinni wiedzieć, jakie dane są gromadzone, w jakim celu i jak długo będą przechowywane. Stworzenie oficjalnej polityki prywatności oraz krótkich materiałów informacyjnych ułatwi rozwianie obaw związanych z logowaniem aktywności wejść. Otwartość w kwestii działania systemu pozwala budować zaufanie i zapobiegać nieporozumieniom. Komunikacja jest istotna także w sytuacjach

wyjątkowych takich jak nagłe zmiany planu, incydenty techniczne czy konieczność ręcznych ingerencji w działanie systemu. Jasny opis procedur i kanałów kontaktu zmniejsza ryzyko chaosu w sytuacjach krytycznych i zwiększa poczucie bezpieczeństwa użytkowników.

4.4.4 Akceptacja społeczna i partycypacja użytkowników

Zaangażowanie użytkowników w proces projektowania i wdrażania miało kluczowe znaczenie dla akceptacji systemu. Konsultacje z administracją, dziekanatem oraz ochroną pozwoliły dostosować funkcjonalności do realnych potrzeb, a nie jedynie do założeń projektowych. Takie podejście znacząco zmniejsza ryzyko oporu wobec zmian. Istotnym elementem akceptacji są także tryby awaryjne, możliwość ręcznego otwarcia, obsługa wyjątków, działania procedur bezpieczeństwa oraz zapewnienie, że krytyczne funkcje budynku nie zależą od jednego systemu.

4.4.5 Miernik wpływu społecznego

Wpływ systemu oceniany jest przede wszystkim poprzez spadek incydentów związanych z fizycznymi kluczami, zmniejszenie liczby interwencji ochrony oraz poprawę punktualności zajęć. Uzupełnia go zwiększony komfort użytkowników, szybkość reakcji w razie problemów oraz przejrzystość procesów administracyjnych. Kluczowa jest także redukcja czasu poświęcanego przez pracowników na manualne operacje związane z dostępem do sal.

4.4.6 Ryzyka społeczne i środki zaradcze

Potencjalne ryzyka obejmują obawy o prywatność, niechęć wobec nowych procedur, błędy wynikające z nieaktualnych danych oraz możliwość nadużyć administracyjnych. Ryzyka te minimalizowane są poprzez projektowanie z poszanowaniem prywatności (ograniczenie danych do niezbędnego minimum, jasna retencja, silna kontrola dostępu do logów), wprowadzenie niezawodnych trybów awaryjnych oraz rozdzielenie ról administracyjnych z pełnym audytem działań. Długofalowe działania obejmują edukację użytkowników, regularne aktualizacje dokumentacji oraz utrzymywanie czytelnych kanałów zgłaszania problemów. Dzięki temu system pozostaje nie tylko sprawny technicznie, lecz także akceptowalny społecznie.

4.5 Kontekst etyczny

System [EkeyPJATK](#) wpływa na realne zachowania użytkowników i sposób funkcjonowania uczelni, dlatego jego projektowanie wymaga świadomego uwzględnienia

zasad etycznych. W tym rozdziale opisano najważniejsze wartości, ryzyka i dobre praktyki, które powinny towarzyszyć wdrożeniu systemu zależnego od danych identyfikacyjnych i logów wejść do pomieszczeń.

4.5.1 Założenia i wartości

Podstawą całego projektu jest poszanowanie prywatności oraz proporcjonalne wykorzystanie danych, system ma rozwiązywać realny problem organizacyjny i bezpieczeństwa, jednocześnie nie ingerując bardziej niż to konieczne. [EkeyPJATK](#) ma być narzędziem transparentnym i sprawiedliwym wobec wszystkich użytkowników kampusu. Jego celem nie jest monitorowanie zachowań, lecz zapewnienie płynnego działania zajęć i odciążenie ochrony oraz administracji. System ma działać tak, by użytkownicy czuli się wspierani, a nie nadzorowani.

4.5.2 Prywatność i minimalizacja danych

[EkeyPJATK](#) przetwarza wrażliwe operacyjnie informacje, identyfikatory użytkownika, salę, znacznik czasu i wynik decyzji o dostępie. W projekcie przyjęto zasadę minimalizacji danych: system nie zbiera niczego poza absolutnym minimum potrzebnym do decyzji oraz późniejszej rozliczalności zajęć. Przechowywane logi obejmują jedynie: identyfikator karty lub roli, nazwę sali, czas oraz informację o przyznaniu lub odmowie dostępu. Dane te służą głównie do weryfikacji zgodności z planem zajęć i ewentualnej diagnostyki problemów. Retencja powinna być z natury ograniczonaa, zalecany okres to od 6 do 12 miesięcy, po którym dane podlegałyby anonimizacji lub usunięciu. Dostęp do logów mają wyłącznie osoby pełniące funkcje służbowe (ochrona, administracja, IT/[BSS](#)), przy czym każda operacja powinna być rejestrowana i audytowalna. Użytkownicy powinni mieć możliwość uzyskania informacji o tym, jakie dane ich dotyczą i w jakim zakresie są przetwarzane.

4.5.3 Proporcjonalność i celowość

Ważne jest, aby automatyzacja była proporcjonalna do celu, system ma otwierać drzwi na podstawie planu zajęć i ról użytkowników, a nie tworzyć nadmiernej warstwy nadzoru. Dlatego zakres monitorowania ogranicza się wyłącznie do momentu podejścia do czytnika i próby wejścia do sali. Logi nie obejmują poruszania się po korytarzach czy aktywności niezwiązanej z harmonogramem. Decyzje o dostępie wynikają wprost z danych w planie, a nie z uznaniowych decyzji administracyjnych. Wyjątki takie jak zastępstwa czy nagłe zmiany sal mają działać ściśle w określonym zakresie czasowym i być jasno rejestrowane. Dzięki temu system pozostaje narzędziem organizacyjnym, a nie mechanizmem kontroli osobistej.

4.5.4 Sprawiedliwość i niedyskryminacja

[EkeyPJATK](#) musi traktować wszystkich użytkowników w jednakowy sposób. Reguły dostępu opierają się wyłącznie na rolach w systemie i harmonogramie zajęć, nie mogą prowadzić do faworyzowania lub wykluczania któregokolwiek z użytkowników. Kluczowe jest, aby proces nadawania i modyfikowania uprawnień był spójny i podlegał przejrzystemu nadzorowi administracyjnemu. Użytkownik powinien mieć jednoznaczny ścieżkę zgłoszenia problemu, na przykład sytuacji, w której system niesłusznie odmówił dostępu lub dane w planie były nieaktualne. Istotne są także okresowe przeglądy zasad działania systemu, wykonywane np. raz na semestr przez administrację i IT/[BSS](#), aby upewnić się, że automatyzacja nie prowadzi do niezamierzonych nierówności.

4.5.5 Transparentność i rozliczalność

Użytkownicy muszą rozumieć, jak działa system, jakie dane są zbierane i dlaczego. W praktyce oznacza to jasne komunikaty przy drzwiach, zrozumiały opis działania systemu oraz dostępność podstawowych informacji o zasadach przetwarzania danych. Dla administracji i działu technicznego niezbędne są czytelne rejestry zmian np. kiedy i kto modyfikował zasady dostępu, dodawał uprawnienia tymczasowe lub dokonywał operacji diagnostycznych. Dzięki temu możliwe jest wykrywanie błędów i ustalanie odpowiedzialności w przypadku incydentów. Okresowe audyty techniczne i organizacyjne, np. na koniec semestru, pomagają w utrzymaniu wysokiej jakości i spójności działania.

4.5.6 Bezpieczeństwo informacji

Bezpieczeństwo danych jest kluczowe dla etycznego działania systemu. Obejmuje to szyfrowanie komunikacji i danych spoczynkowych, regularne kopie zapasowe oraz precyzyjne nadawanie uprawnień administracyjnych zgodnie z zasadą najmniejszych możliwych uprawnień. Ważnym elementem są przeglądy konfiguracji, testy bezpieczeństwa oraz jasno określony plan reagowania na incydenty. System obsługuje dane operacyjne, które choć nie są danymi wrażliwymi w sensie prawnym, mogą mieć znaczenie organizacyjne i nie powinny trafić w niepowołane ręce.

4.5.7 Dostępność i inkluzywność

Projekt musi uwzględniać potrzeby użytkowników z niepełnosprawnościami i gości. Czytniki powinny być rozmieszczone w sposób ergonomiczny i intuicyjny, a komunikaty na ekranie zrozumiałe i czytelne. W razie problemów technicznych system

ma zapewniać alternatywę: możliwość ręcznego otwarcia drzwi przez ochronę lub przełączenie w tryb awaryjny, który nie blokuje ruchu w budynku.

4.5.8 Zarządzanie ryzykiem i nadzór etyczny

Aby ograniczyć ryzyko nadużyć zakresu danych, zarządzanie systemem powinno podlegać jasnym zasadom. Zaleca się okresową ocenę wpływu systemu na prywatność i organizację pracy, a także dokumentowanie wszelkich zmian celu przetwarzania danych i zakresu zbieranych informacji. Każde rozszerzenie np. dodanie nowych typów logów powinno być uzgadniane z interesariuszami reprezentującymi różne grupy użytkowników.

4.5.9 Metryki etyczne

Ocena etycznego działania systemu powinna opierać się na realnych i mierzalnych danych. Do najważniejszych wskaźników należą:

- liczba i charakter incydentów dostępowych oraz szybkość ich rozwiązania,
- zgodność realnego okresu retencji z deklarowaną polityką danych,
- wyniki audytów technicznych i organizacyjnych dotyczących bezpieczeństwa i spójności działania.

4.5.10 Podsumowanie

Etyczne działanie systemu [EkeyPJATK](#) opiera się na równowadze między bezpieczeństwem i efektywnością organizacyjną a prawem użytkowników do prywatności i transparentności. Kluczowe jest świadome projektowanie zasad przetwarzania danych, kontrolowany zakres automatyzacji, wprowadzenie mechanizmów nadzorczych oraz czytelna komunikacja z użytkownikami. Dzięki temu system może skutecznie wspierać funkcjonowanie uczelni, nie naruszając zaufania społeczności akademickiej.

Rozdział 5

Planowanie projektu

5.1 Metodyka pracy

Realizacja systemu [EkeyPJATK](#) przebiegała w warunkach, które trudno byłoby w pełni opisać przy użyciu klasycznych metodyk takich jak tradycyjny [Waterfall](#) czy formalny [Scrum](#). Projekt powstawał przy jednoczesnej współpracy i ciągłej komunikacji z wieloma niezależnymi podmiotami: dziekanatem, zespołem odpowiedzialnym za wcześniejszy projekt [Cyfrowe Klucze](#), osobą rozwijającą aplikację [Cyfrowe Klucze Mobile](#), administracją uczelni, firmą obsługującą system [GAK-KO](#) oraz promotorem. Każde spotkanie (niezależnie od tego, czy odbywało się online czy stacjonarnie) wносиło nowe informacje, ograniczenia, wymagania lub niespodziewane problemy, które bezpośrednio wpływały na kształt kolejnych etapów pracy.

Z tego powodu naturalnym punktem odniesienia stały się założenia charakterystyczne dla *programowania ekstremalnego* (Ang. [Extreme Programming](#))[XP].

Programowanie Ekstremalne jest jednym z wielu procesów zwinnych.

Programowanie Ekstremalne jest skuteczne, dlatego że kładzie nacisk na satysfakcję klienta [...] proces ten dostarcza oprogramowanie, którego potrzebujesz, dokładnie wtedy, gdy jest to potrzebne.

Programowanie Ekstremalne podkreśla znaczenie pracy zespołowej. Menedżerowie, klienci i programiści są równorzędnymi partnerami w jednym współpracującym ze sobą zespole.

Programowanie Ekstremalne usprawnia projekt informatyczny na pięć szczególnych sposobów:

komunikacja, prostota, informacja zwrotna, szacunek i odwaga.

[XP, Własne tłumaczenie]

XP zakłada intensywną komunikację, szybką reakcję na zmiany, krótkie cykle dostarczania i gotowość do ciągłego refaktoryzowania kodu. W praktyce realizacji projektu wszystkie te elementy pojawiły się naturalnie: praca odbywała się iteracyjnie, często z tygodnia na tydzień, a kierunek rozwoju potrafił zmienić się w ciągu jednego spotkania - na przykład po otrzymaniu nowych wymagań od dziekanatu, zmianach po stronie promotora czy konieczności skorygowania integracji z zespołem [Cyfrowe Klucze](#).

Projekt był rozwijany równolegle przez podzespoły, które płynnie przełączały się między zadaniami: od pisania i testowania kodu backendu, przez prace nad hardware i prototypami czytnika, po przygotowanie dokumentacji technicznej i pisanie pracy inżynierskiej. Taki sposób działania wymagał nie tylko elastyczności, ale również szybkiego dzielenia się wiedzą i koordynacji - szczególnie w momentach, gdy jeden zespół kończył pracę nad fragmentem, który inny musiał przejąć, rozszerzyć lub dostosować do nowej sytuacji.

Cechą charakterystyczną całego procesu było to, że projekt rozwijał się w rytmie wyznaczanym przez kolejne spotkania z interesariuszami. Po każdej rozmowie ustalenia były natychmiast przekładane na konkretne zadania, a plan pracy na nowo porządkowany zgodnie z aktualnymi potrzebami. Programowanie ekstremalne zakłada, że zmiana wymagań jest zjawiskiem naturalnym, nie wyjątkiem - i dokładnie tak wyglądała realizacja [EkeyPJATK](#). Zdarzało się, że kluczowe elementy należało przeprojektować, znacząco zmienić lub uprościć, jednak dzięki krótkim cyklom wdrożeniowym projekt nieustannie posuwał się do przodu.

W efekcie metodyka pracy przyjęła formę hybrydową: z jednej strony opartą na praktykach znanych z programowania ekstremalnego (ciągła integracja, refaktoryzacja, codzienna komunikacja, iteracyjny rozwój), z drugiej - dostosowaną do specyfiki uczelni, wielości interesariuszy oraz równoległego rozwoju hardware'u i oprogramowania. Ten elastyczny model pozwolił na sprawne reagowanie na zmiany, skuteczne rozwiązywanie problemów oraz konsekwentne doprowadzenie projektu do kolejnych działających etapów, mimo zmiennego i często nieprzewidywalnego otoczenia organizacyjnego.

5.2 Infrastruktura

Efektywna realizacja projektu wymagała doboru odpowiedniego zestawu narzędzi wspierających komunikację, zarządzanie wersjami kodu, projektowanie oraz wymianę dokumentacji. Infrastruktura projektu została zorganizowana w sposób zapewniający płynny przepływ informacji w zespole oraz bezpieczeństwo wytwarzanych dóbr cyfrowych.

5.2.1 Kontrola Wersji i Organizacja Repozytoriów

Podstawowym narzędziem do zarządzania kodem źródłowym projektu był system **Git**, obsługiwany za pośrednictwem platformy **GitHub**. W celu zachowania porządku i separacji odpowiedzialności, zamiast jednego monolitycznego repozytorium, zastosowano strukturę organizacji (GitHub Organization) z podziałem na dedykowane repozytoria dla poszczególnych modułów systemu.

- **Modularyzacja systemu:** Architektura repozytoriów ściśle odzwierciedla podział technologiczny i logiczny projektu. Wyodrębniono niezależne repozytoria dla warstwy sprzętowej (`door-panel` w języku C), interfejsu użytkownika (`digikey-view` w technologii Svelte) oraz logiki backendowej i bazodanowej (`digikey-DB`, `digikey-raport`).
- **Symulacja środowiska ([Mocking](#)):** Ze względu na zależność od zewnętrznych systemów uczelnianych, kluczowym elementem infrastruktury stały się repozytoria `mock-gakko` oraz `mock-CyfroweKlucze`. Pozwoliły one na emulację działania zewnętrznych API, umożliwiając zespołowi nieprzerwaną pracę deweloperską i testy integracyjne nawet bez dostępu do produkcyjnych wersji systemów GAKKO czy Cyfrowe Klucze.
- **Dokumentacja jako kod (Docs-as-code):** Praca inżynierska oraz dokumentacja projektowa były rozwijane w systemie $\text{\LaTeX}$ w dedykowanym repozytorium `digikey-thesis`. Pozwoliło to na zastosowanie tych samych praktyk kontroli wersji (commitowanie zmian, historia edycji), które stosowano przy tworzeniu oprogramowania.

5.2.2 Komunikacja Zespołowa i Interesariusze

Ze względu na konieczność szybkiej wymiany informacji, komunikację podzielono na dwa główne kanały:

1. **Komunikacja wewnętrzna (Discord):**

Na potrzeby bieżącej współpracy utworzono dedykowany serwer na platformie Discord. Wykorzystano go do:

- codziennej komunikacji tekstowej z podziałem na kanały,
- spotkań głosowych i sesji cotygodniowych spotkań omawiających postępy pracy, napotkane problemy i rozdział prac na następny tydzień,
- szybkich statusów, zastępujących długie spotkania formalne.

2. **Komunikacja zewnętrzna (Gmail):**

Do kontaktu z interesariuszami spoza zespołu deweloperskiego (Promotor, Dziekanat, Administracja Uczelni) wykorzystano pocztę elektroniczną. Kanał ten służył do:

- formalnych ustaleń dotyczących zakresu projektu,
- przesyłania dokumentacji interfejsów,
- umawiania spotkań stacjonarnych.

5.2.3 Zarządzanie Wiedzą i Projektowanie

Do tworzenia artefaktów projektowych wykorzystano pakiet narzędzi w chmurze, co umożliwiło pracę równoległą wielu osób w czasie rzeczywistym.

- **Google Workspace (Drive, Docs, Sheets):** Służył jako centralne repozytorium dokumentacji nietechnicznej. Przechowywano tam notatki ze spotkań, harmonogramy, dokumentację tworzoną o tym projekcie ale na potrzeby innych przedmiotów.
- **Draw.io:** Narzędzie wykorzystane do modelowania architektury systemu. Stworzono w nim diagramy UML oraz schematy bazy danych, które stały się podstawą implementacji.
- **Figma:** Wykorzystana w fazie projektowania interfejsu użytkownika (UI). Stworzono w niej makiety aplikacji webowej, co pozwoliło na weryfikację użyteczności (UX) jeszcze przed rozpoczęciem implementacji.
- **ChatGPT:** Wspierał zespół w procesie redakcji dokumentacji oraz w analizie problemów technicznych. Narzędzie wykorzystywano do formatowania i korekty tekstu, wyjaśniania niejasności związanych z technologiami, a także do identyfikacji drobnych błędów w kodzie oraz generowania sugestii dotyczących rozwiązań projektowych.

5.2.4 Ewolucja Narzędzi Zarządzania

W początkowej fazie projektu do zarządzania zadaniami wdrożono system **Jira** oraz tablice **Miro**. Jednakże w toku prac zespół podjął decyzję o uproszczeniu środowiska pracy.

- Wysoka dynamika zmian i iteracyjny charakter pracy sprawiły, że narzut czasowy na utrzymanie aktualności zadań w systemie Jira okazał się nieproporcjonalny do korzyści.
- Zdecydowano się przenieść zarządzanie bieżącymi zadaniami bezpośrednio na dedykowane kanały platformy Discord oraz listy kontrolne w Google Docs, co zwiększyło elastyczność i szybkość reakcji zespołu na zmieniające się wymagania.

5.3 Podział ról

Tabela 5.1: Podział ról i odpowiedzialności w zespole projektowym

Rola w zespole	Główne obowiązki i realizowane zadania	Obszar / Moduł
Bartosz Bohatyrewicz (Hardware)	Research dotyczący hardware, rozwój hardware, elementy powiązane z hardware w dokumentacji, wsparcie rozwoju back-endu	Hardware (door-panel)
Łukasz Korycki (Dokumentacja, Testy, Backend)	Dokumentacja, wsparcie rozwoju back-endu oraz tworzenie testów jednostkowych i implementacyjnych.	Dokumentacja (digikey-thesis, digikey-SW0); Testy (digikey-DB, digikey-QR, digikey-raport)
Kamil Maliński (Dokumentacja, Testy, Raporty)	Dokumentacja, moduł raportowania, testy i analiza cyberbezpieczeństwa, przeprowadzenie testów penetracyjnych.	Dokumentacja (digikey-thesis, digikey-raport), Testy
Jan Szydłowski (Hardware)	Research dotyczący hardware, rozwój hardware, elementy powiązane z hardware w dokumentacji, wsparcie rozwoju back-endu, projekt obudowy	Hardware (door-panel), Projektowanie 3D
Igor Wojciechowski (MVP)	Koordinacja prac, integracja podsystemów, rozwój back-endu oraz front-endu, zarządzanie repozytoriami	Full stack

5.4 Ryzyka i ograniczenia

Zespoły [Cyfrowe Klucze](#) oraz [Cyfrowe Klucze Mobile](#) przeprowadziły niezależne analizy ryzyk i ograniczeń związanych z realizacją swoich projektów. Ze względu na przyjęte założenia projektowe, zidentyfikowane ryzyka w ramach [CK](#) mają bezpośrednie przełożenie na projekt [EkeyPJATK](#). Natomiast ryzyka i ograniczenia wskazane przez zespół [CKM](#) oddziałują na projekt [Ekey](#) w sposób pośredni.

Oba zespoły zastosowały odmienne podejścia metodologiczne do analizy ryzyk.

Zespół [Cyfrowe Klucze](#) skoncentrował się na identyfikacji ryzyk i ich kategoryzacji, wyróżniając cztery główne grupy:

- Ataki hakerskie
- Błędy ludzkie
- Problemy techniczne
- Problemy wdrożeniowe

Określono ryzyka projektu, ich typy, skutki oraz sposoby przeciwdziałania (Rys. 4). Analiza ryzyk oraz strategię mierzenia się z nimi pozwalają na wprowadzenie funkcjonalności, oraz lepsze przygotowanie projektu. [CK, rozdział 4.4]

Ataki hakerskie

Techniczne - wydajność i niezawodność

Systemy działające w sieci narażone są na ataki, występuje ryzyko wycieku poufnych informacji, skutkujące uzyskaniem dostępu do ważnych pomieszczeń i zasobów.

Skutki - zaburzenie działania systemu, fałszywe informacje o odbywających się zajęciach, nieautoryzowany dostęp do sal

Strategia przeciwdziałania - Wzmocnienie: regularne audyty i testy bezpieczeństwa



Błędy ludzkie

Zewnętrzne - odbiorca

Ludzkie błędy, ryzyko utraty identyfikatorów dostępu lub niedokładne przypisanie uprawnień, mogą wpłynąć na skuteczność systemu.

Skutki - dydaktyk nie może wejść do sali, w której ma zajęcia

Strategia przeciwdziałania - Łagodzenie: karty gościa, dodatkowe karty, możliwość zdalnego otwierania sal



Problemy techniczne

Techniczne - złożoność i powiązanie

Awarie sprzętu lub oprogramowania mogą wpłynąć na efektywność systemu.

Skutki - dydaktyk nie może wejść do sali, w której ma zajęcia

Strategia przeciwdziałania - Akceptacja: awarie będą występowały, wprowadzenie system monitorującego i logującego stan systemu.



Problemy wdrożeniowe

Zarządzanie projektem - planowanie

Brak pokrycia siecią wifi, niewłaściwy montaż systemu, urządzeń

Skutki - przedłużający się czas wdrożenia systemu, rosnące koszty

Strategia przeciwdziałania - Łagodzenie: różne opcje konfiguracji urządzenia, przeszkolenie pracowników



Rys. 4 Rejestr ryzyk produktu

Rysunek 5.1: Kategorie ryzyk w [Cyfrowe Klucze \(CK\)](#) [CK, rozdział 4.4].

Z kolei zespół [Cyfrowe Klucze Mobile](#) przyjął podejście oparte na wskazaniu najbardziej prawdopodobnych ryzyk i ograniczeń, które mogą wystąpić w trakcie realizacji projektu.

Nazwa	Typ	Opis	Skutki	Klasyfikacja
Atak na System	Ryzyko	Osoby trzecie zyskują dostęp do systemu z uprawnieniami administratora.	Wyciek danych, dostęp do budynku dla osób nieautoryzowanych	Wysokie
Brak Internetu	Ograniczenie	W budynku z systemem kontroli dostępu zabraknie Internetu, co powoduje brak możliwości komunikacji między aplikacją a serwerem.	Brak dostępu do sali.	Średnie
Identyfikacja imienna	Ograniczenie	Aplikacje są przypisane do maila uczelnianego, co powoduje brak możliwości udostępniania tego systemu gościom.	Brak dostępu do rezerwacji dla osób zewnętrznych, nie uwzględnionych w systemie.	Niskie
Odrzucenie dostępu aplikacji do aparatu	Ograniczenie	Aplikacja bez odpowiednich uprawnień nie będzie w stanie uruchomić aparatu urządzenia.	Brak możliwości skczytania kodu QR, co uniemożliwiło otwieranie sali.	Niskie
Awaria Systemu	Ryzyko	System straci informacje w bazie danych lub straci kontakt z serwerem.	Uniemożliwienie logowania przez aplikację, wywołanie błędów przy próbie autoryzacji.	Średnie

Tabela 2 - Ryzyka i ograniczenia systemu

Rysunek 5.2: Ryzyka i ograniczenia w [Cyfrowe Klucze Mobile \(CKM\)](#) [CKM, rozdział 6.2].

Oba podejścia dostarczają wartościowych perspektyw, jednak w kontekście projektu [EkeyPJATK](#) konieczne jest ujęcie ryzyk w sposób przekrojowy, obejmujący cały [System Cyfrowego Dostępu](#). Z tego względu w niniejszej analizie przyjęto podejście syntetyczne, łączące elementy obu wcześniejszych metod, lecz ukierunkowane na identyfikację ryzyk i ograniczeń:

- najbardziej prawdopodobnych,
- najbardziej wrażliwych z punktu widzenia bezpieczeństwa, dostępności i poprawności działania,
- mających realny wpływ na funkcjonowanie całego systemu, a nie jedynie jego pojedynczych komponentów.

Aby umożliwić jednoznaczne powiązanie każdego ryzyka z odpowiedzialnym obszarem technicznym, wprowadzono dodatkową kolumnę „zależność”, wskazującą, którego podsystemu dane ryzyko lub ograniczenie dotyczy w największym stopniu. Pozwala to na precyzyjne określenie odpowiedzialności, ułatwia planowanie działań zapobiegawczych oraz umożliwia ocenę wpływu potencjalnych problemów na pozostałe elementy architektury.

Tak zdefiniowane podejście zapewnia spójność analizy na poziomie całego **SCD** oraz umożliwia efektywne zarządzanie ryzykiem w projekcie **Ekey**.

Tabela 5.2: Tabela ryzyk i ograniczeń dla **Systemu Cyfrowego Dostępu** z uwzględnieniem zależności.

Nazwa	Opis	Skutki	Dotkliwość	Zależność
Przejęcie karty magnetycznej	Możliwość sklonowania lub zgubienia karty przez użytkownika.	Nieautoryzowany dostęp do sal, konieczność wydania nowych kart i usunięcia starych kart z systemu.	Umiarkowana	CK
Nadużycia związane z QR	Potencjalna próba podmiany lub rozpowszechnienia kodu QR .	Brak realnej możliwości obejścia systemu - kod nie niesie uprawnień, a dostęp wymaga autoryzacji użytkownika i weryfikacji rezerwacji.	Niskie	CKM
Ataki na Cyfrowe Klucze Mobile	Przejęcie sesji aplikacji lub konta użytkownika, w tym konta z rozszerzonymi uprawnieniami.	Możliwość wykonywania akcji w imieniu użytkownika; w przypadku kont uprzywilejowanych - nieautoryzowane otwieranie wielu sal.	Znacząca	CKM
Integracja z GAKKO	Ryzyko błędów synchronizacji lub opóźnień w aktualizacji danych harmonogramowych.	Nieprawidłowe przydzielanie uprawnień, błędne decyzje dostępu, zakłócenia organizacyjne.	Umiarkowana	Ekey

Ciąg dalszy na następnej stronie...

Tabela 5.2: Tabela ryzyk i ograniczeń dla SCD z uwzględnieniem zależności. - kontynuacja

Nazwa	Opis	Skutki	Dotkliwość	Zależność
Awaria infrastruktury	Niedostępność czytników, serwera bazy danych lub sieci.	Brak możliwości weryfikacji dostępu, przerwy w zajęciach.	Znacząca	Ekey
Ryzyka organizacyjne	Niewłaściwe zarządzanie uprawnieniami, brak szybkiej reakcji na incydenty.	Nieautoryzowany dostęp, naruszenia polityki bezpieczeństwa.	Umiarkowana	Wspólne
Wyciek prywatnych danych	Nieautoryzowany dostęp do danych użytkowników.	Konsekwencje prawne, utrata reputacji uczelni.	Znacząca	Wspólne
Backup danych	Ryzyko wycieku danych lub braku możliwości odtworzenia systemu.	Utrata danych rezerwacyjnych, kartach i logach; zakłócenia działania całego systemu.	Umiarkowana	CK
Klucze szyfrujące	Utrata lub kompromitacja kluczy szyfrujących.	Pełna kompromitacja systemu.	Znacząca	Wspólne
Atak na system	Osoby trzecie uzyskują dostęp do systemu z uprawnieniami administratora.	Wyciek danych, nieautoryzowany dostęp do sal.	Wysokie	Ekey
Brak Internetu	Niedostępność sieci powoduje brak komunikacji między aplikacją mobilną a serwerem.	Brak możliwości weryfikacji rezerwacji i otwierania sal.	Średnie	CKM

Ciąg dalszy na następnej stronie...

Tabela 5.2: Tabela ryzyk i ograniczeń dla **SCD** z uwzględnieniem zależności. - kontynuacja

Nazwa	Opis	Skutki	Dotkliwość	Zależność
Awaria systemu	Utrata danych w bazie lub utrata kontaktu z serwerem.	Uniemożliwienie logowania i autoryzacji, błędy w procesie rezerwacji.	Średnie	Ekey
Odrzucenie dostępu aplikacji do aparatu	Brak uprawnień do aparatu uniemożliwia skanowanie kodów QR .	Nieemożność otwierania sal przez aplikację mobilną.	Niskie	CKM
Identyfikacja imienna	Aplikacja przypisana do maila uczelnianego, brak obsługi gości.	Brak możliwości rezerwacji i dostępu dla osób zewnętrznych.	Niskie	CKM
Błędna konfiguracja systemu	Nieprawidłowe ustawienia uprawnień, ról lub parametrów integracji.	Nieautoryzowany dostęp lub blokada dostępu, błędne decyzje systemu.	Znacząca	Ekey

5.4.1 Podsumowanie ryzyk z perspektywy systemu Ekey-PJATK

Analiza ryzyk wskazuje, że system **Ekey** pełni kluczową rolę w całym **ekosystemie kontroli dostępu**, a jego poprawne działanie jest niezbędne dla bezpieczeństwa, dostępności i spójności procesów. **Ekey** stanowi centralny punkt decyzyjny, odpowiedzialny za weryfikację uprawnień, synchronizację danych harmonogramowych oraz generowanie zdarzeń i raportów. Z tego względu większość ryzyk o najwyższej wadze dotyczy bezpośrednio właśnie tego podsystemu.

Najbardziej istotne zagrożenia obejmują:

- awarie infrastruktury i systemu, które uniemożliwiają autoryzację dostępu i zakłócają ciągłość działania,
- atak na system prowadzący do przejęcia uprawnień administracyjnych,

- błędną konfigurację wpływającą na poprawność decyzji dostępowych,
- problemy z integracją z [GAKKO](#), które mogą skutkować nieprawidłowym przydzielaniem uprawnień,
- ryzyka związane z backupem danych, kluczowych dla odtwarzania stanu systemu i zachowania integralności informacji.

Choć część ryzyk dotyczy aplikacji mobilnej [CKM](#) lub systemu [CK](#), to właśnie [Ekey](#) jest najbardziej wrażliwy na błędy danych, awarie serwerowe i ataki na warstwę logiczną. W konsekwencji to on ma największy wpływ na funkcjonowanie [SCD](#) - zarówno w zakresie bezpieczeństwa, jak i dostępności oraz poprawności działania.

5.5 Proces planowania

- **Marzec / Kwiecień 2025 - Początkowe koncepcje**

Projekt przechodził przez wiele etapów planowania, które potrafiły całkowicie zmienić kierunek jego rozwoju. Początkowo zakładano, że będziemy bezpośrednio rozbudowywać istniejący system [Cyfrowe Klucze](#), jednak zostało to porzucone, przez brak zgody jednego z oryginalnych twórców systemu.

Następnie rozważano stworzenie całkowicie nowego systemu od podstaw, jednak ze względu na ograniczony czas realizacji projektu oraz istniejącą infrastrukturę systemu [Cyfrowe Klucze](#), zdecydowano się na podejście hybrydowe - wykorzystanie już stworzonych komponentów tam, gdzie to możliwe, oraz stworzenie nowych modułów i funkcjonalności, które będą integrować się z istniejącym systemem.

- **Maj / Czerwiec 2025 - Współpraca z zespołem Cyfrowe Klucze Mobile**

W wyniku nawiązania współpracy z zespołem odpowiedzialnym za rozwój aplikacji [Cyfrowe Klucze Mobile](#), planowane zostało wprowadzenie modułu umożliwiającego korzystanie z systemu [Cyfrowe Klucze](#) za pomocą kodów QR, poprzez aplikację mobilną. Rozpoczęto planowanie budowy nowego modułu zamka.

- **Lipiec / Sierpień 2025 - Konsultacje i planowanie integracji z Gakko**

Planowane były regularne spotkania z promotorem oraz konsultantem należącym do zespołu [Cyfrowe Klucze](#) oraz przygotowanie aplikacji pod integrację z systemem uczelnianym [GAKKO](#). Rozplanowano również pracę nad oprogramowaniem fizycznego modułu zamka, zaczynając od [UI](#).

- **Wrzesień 2025 - Przerwa w pracach**

Planowano kontynuację prac w momencie otrzymania niezbędnych danych do integracji z systemem [GAKKO](#).

- **Październik 2025 - Integracja z Gakko**

Po otrzymaniu od firmy obsługującej system [GAKKO](#) niezbędnych danych do integracji. Planowane było rozpoczęcie implementacji pobierania i mapowania danych w systemie [EkeyPJATK](#) oraz przygotowanie środowiska testowego. Zaplanowano zmiany elementów modułu zamka ze względu na brak kompatybilności między niektórymi z nich.

- **Listopad 2025 - Rozwój submodułów**

Celem było rozwinięcie poszczególnych submodułów systemu, oraz stworzenie sztucznej infrastruktury symulującej istniejące elementy systemu [Cyfrowe Klucze](#). Przygotowano też submoduły oprogramowania modułu zamka oraz rozplanowano wdrożenie [PoE](#).

- **Grudzień 2025 - Uściślenie integracji między wszystkimi elementami i systemami**

Zfinalizowanie połączeń między elementami systemu oraz istniejąca infrastrukturą. Przygotowanie prototypowego czytnika do montażu.

- **Styczeń 2026 - Prace wykończeniowe**

Dostarczenie systemu gotowego do wdrożenia lub przygotowanego na finalne integracje z systemem [Cyfrowe Klucze](#). Ogranicza się to do zakresu możliwości jakie ma zespół [Digikey](#), bez dostępu do wprowadzania bezpośrednich zmian w systemie [Cyfrowe Klucze](#).

Rozdział 6

Analiza wymagań

6.1 Wymagania ogólne i dziedzinowe

W tej sekcji zdefiniowano wymagania ogólne i dziedzinowe, które stanowią fundament dla całego projektu. Są to zasady, które określają kluczowe założenia, ograniczenia technologiczne oraz strategiczne cele systemu [EkeyPJATK](#). Wymagania te, w przeciwieństwie do wymagań funkcjonalnych, nie opisują konkretnych działań, lecz narzucają ogólne ramy, w jakich system musi funkcjonować, aby spełnić oczekiwania interesariuszy i wpisać się w środowisko uczelni.

Tabela 6.1: Integracja z planem zajęć

KARTA WYMAGANIA			
Identyfikator:	WO1	Priorytet:	Wysoki
Nazwa:	Synchronizacja z planem zajęć uczelni		
Opis:	System EkeyPJATK musi opierać swoje decyzje o dostępie na danych pochodzących z planu zajęć dostarczanych przez system GAKKO . Logika działania systemu musi zakładać, że poprawnie pobrany harmonogram stanowi jedyne wiarygodne źródło informacji o tym, kto, kiedy i do której sali ma uprawniony dostęp.		
Udziałowiec:	GAKKO , Dziekanat		
Wymagania powiązane:	WF-GAKKO-1 , WF-ACCESS-1		

Tabela 6.2: Modularność systemu

KARTA WYMAGANIA			
Identyfikator:	WO2	Priorytet:	Wysoki
Nazwa:	Architektura mikroserwisowa		
Opis:	System musi być zaprojektowany tak aby możliwa była niezależna aktualizacja i rozwój jego części: urządzeń brzegowych, backendu Ekey-PJATK , panelu administracyjnego oraz integracji z zewnętrznymi systemami (GAKKO , Cyfrowe Klucze , Cyfrowe Klucze Mobile).		
Udziałowiec:	Promotor , Zespół Ekey		
Wymagania powiązane:	NFO3 , IO1 , IO2 , IO3		

Tabela 6.3: Obsługa dynamicznych zmian planu

KARTA WYMAGANIA			
Identyfikator:	WO3	Priorytet:	Wysoki
Nazwa:	Synchronizacja zmian w harmonogramie		
Opis:	System musi być zdolny do szybkiego reagowania na zmiany w planie zajęć, takie jak zastępstwa, przeniesienia sal czy odwołania zajęć. Aktualizacje muszą się odbywać często, automatycznie, domyślnie co pół godziny.		
Udziałowiec:	Dziekanat , GAKKO , Zespół Ekey		
Wymagania powiązane:	WF-GAKKO-1 , IO3		

Tabela 6.4: Zgodność z politykami bezpieczeństwa uczelni

KARTA WYMAGANIA			
Identyfikator:	WO4	Priorytet:	Średni
Nazwa:	Wymagania instytucjonalne dotyczące bezpieczeństwa		
Opis:	System musi działać zgodnie z politykami bezpieczeństwa uczelni, w szczególności dotyczącymi ochrony danych, kontroli dostępu, retencji logów, administrowania uprawnieniami i zgodności z wymaganiami BSS . Dotyczy to zarówno zasad projektowych, jak i eksploatacyjnych systemu.		
Udziałowiec:	BSS , Administracja Uczelni		
Wymagania powiązane:	NFO1 , NFO2 , NFO3		

Tabela 6.5: Tryb awaryjny i degradacja działania

KARTA WYMAGANIA			
Identyfikator:	WO5	Priorytet:	Średni
Nazwa:	Obsługa awarii i tryb degradacji		
Opis:	System musi zapewniać możliwość ograniczonego działania w przypadku awarii sieci, braku dostępu do API GAKKO , przerw w komunikacji z systemem Cyfrowe Klucze lub przerwy w zasilaniu urządzeń brzegowych. Tryb awaryjny musi umożliwiać minimalne, lecz bezpieczne działanie zgodne z zasadami BHP i wymogami uczelni.		
Udziałowiec:	BSS , Personel techniczny		
Wymagania powiązane:	WF-GAKKO-3 , IO1 , IO2		

6.2 Wymagania funkcjonalne

Wymagania funkcjonalne opisują dokładnie, co system ma robić - jakie konkretne działania musi wykonywać, aby był użyteczny. Od synchronizacji z planem zajęć po podjęcie decyzji o otwarciu drzwi, te wymagania definiują kluczową logikę i zachowanie systemu [EkeyPJATK](#). Dla większej przejrzystości podzieliliśmy je na logiczne grupy.

Tabela 6.6: Cykliczna synchronizacja danych z GAKKO

KARTA WYMAGANIA			
Identyfikator:	WF-GAKKO-1	Priorytet:	Wysoki
Nazwa:	Pobieranie, zapisywanie i aktualizacja danych z planu zajęć		
Opis:	System EkeyPJATK musi cyklicznie pobierać aktualne dane o planie zajęć z systemu GAKKO , z wykorzystaniem udostępnionego przez zewnętrznego dostawcę endpointu API . Synchronizacja powinna odbywać się automatycznie, w ustalonych odstępach czasu, aby odzwierciedlać bieżące zmiany planu, w tym zastępstwa oraz przeniesienia zajęć.		
Udziałowiec:	GAKKO , Zespół Ekey		
Wymagania powiązane:			

Tabela 6.7: Aktualizacja rekordów planu zajęć

KARTA WYMAGANIA			
Identyfikator:	WF-GAKKO-2	Priorytet:	Wysoki
Nazwa:	Przechowywanie i aktualizacja danych planu zajęć		
Opis:	System musi zapisywać wszystkie nowe dane pobrane z GAKKO we własnej bazie danych. W przypadku braku zmian w planie system musi zaktualizować tylko informację o czasie ostatniej synchronizacji oraz lokalizacji ostatnio pobranych danych.		
Udziałowiec:	Dziekanat , Zespół Ekey		
Wymagania powiązane:	WF-GAKKO-1		

Tabela 6.8: Obsługa awarii komunikacji z GAKKO

KARTA WYMAGANIA			
Identyfikator:	WF-GAKKO-3	Priorytet:	Średni
Nazwa:	Mechanizmy tolerancji błędów przy synchronizacji		
Opis:	Jeśli pobranie danych z GAKKO nie powiedzie się (błąd sieci, błąd API, brak dostępności usługi), system powinien: • odnotować incydent w logach, • działać na podstawie ostatnio poprawnie pobranych danych, Błąd synchronizacji nie może uniemożliwić funkcjonowania logiki dostępowej.		
Udziałowiec:	Zespół Ekey , BSS		
Wymagania powiązane:	WF-GAKKO-1		

Tabela 6.9: Podstawowa decyzja o dostępie

KARTA WYMAGANIA			
Identyfikator:	WF-ACCESS-1	Priorytet:	Wysoki
Nazwa:	Ocena uprawnień dostępowych na podstawie planu zajęć		
Opis:	System EkeyPJATK musi zwrócić informację o tym, czy użytkownik ma prawo wejścia do danej sali w określonym czasie, na podstawie danych z planu zajęć pobranych z GAKKO. Decyzja powinna uwzględniać: • powiązanie użytkownika z zajęciami (prowadzący, gość), • czas względem rozpoczęcia i zakończenia zajęć, • przypisaną salę zgodnie z harmonogramem.		
Udziałowiec:	Zespół Ekey , Zespół Cyfrowe Klucze , Zespół Cyfrowe Klucze Mobile		
Wymagania powiązane:	WF-GAKKO-1 , WF-ACCESS-5		

Tabela 6.10: Obsługa okna czasowego weryfikacji

KARTA WYMAGANIA			
Identyfikator:	WF-ACCESS-2	Priorytet:	Wysoki
Nazwa:	Weryfikacja w dopuszczalnym oknie czasowym		
Opis:	Silnik decyzyjny systemu musi uwzględniać dopuszczalne okno czasowe wejścia, na ten moment przyjmujemy domyślnie 15 minut przed i godzinę po zakończeniu zajęć lub do czasu rozpoczęcia kolejnych zajęć. Próby wejścia poza tym zakresem powinny skutkować odmową i zostać zarejestrowane w logach.		
Udziałowiec:	Zespół Ekey, BSS, Zespół Cyfrowe Klucze		
Wymagania powiązane:	WF-ACCESS-1		

Tabela 6.11: Powiązanie decyzji z salą

KARTA WYMAGANIA			
Identyfikator:	WF-ACCESS-3	Priorytet:	Wysoki
Nazwa:	Weryfikacja zgodności sali		
Opis:	System musi odrzucić próby wejścia do sali, które nie są zgodne z przypisaniem w planie zajęć.		
Udziałowiec:	BSS, Zespół Ekey		
Wymagania powiązane:	WF-ACCESS-1		

Tabela 6.12: Przekazywanie decyzji do systemu CK

KARTA WYMAGANIA			
Identyfikator:	WF-ACCESS-4	Priorytet:	Wysoki
Nazwa:	Komunikacja decyzji do Cyfrowe Klucze		
Opis:	Po podjęciu decyzji system EkeyPJATK musi przekazać wynik (dopuszczony, odrzucony, nie obsługiwany) do systemu Cyfrowe Klucze , odpowiedzialnego za ostateczną decyzję o fizycznym odblokowaniu drzwi. Komunikacja musi przebiegać przez WebSocket zgodnie z ustalonym protokołem.		
Udziałowiec:	Zespół Cyfrowe Klucze		
Wymagania powiązane:	WF-ACCESS-1		

Tabela 6.13: Rejestrowanie decyzji o dostępie

KARTA WYMAGANIA			
Identyfikator:	WF-ACCESS-5	Priorytet:	Wysoki
Nazwa:	Logowanie procesu decyzyjnego		
Opis:	System musi rejestrować wszystkie decyzje o dostępie. Log powinien zawierać minimalny zakres danych: identyfikator użytkownika, czas próby wejścia, identyfikator sali, oraz wynik decyzji (w razie nieobsługiwanego przypadku zapisywane jest minimum dostępnych danych).		
Udziałowiec:	Zespół Ekey		
Wymagania powiązane:	WF-ACCESS-1, WF-ACCESS-2		

Tabela 6.14: Odczyt identyfikatora karty magnetycznej

KARTA WYMAGANIA			
Identyfikator:	WF-MAG-1	Priorytet:	Wysoki
Nazwa:	Podstawowy odczyt karty magnetycznej		
Opis:	Urządzenie hardware stworzone przez zespół musi umożliwiać poprawny odczyt identyfikatora karty magnetycznej użytkownika oraz przekazanie go do systemu Cyfrowe Klucze za pomocą ustalonego protokołu, przez WebSocket. Odczyt powinien następować natychmiast po przyłożeniu karty do czytnika, z minimalnym opóźnieniem, umożliwiając płynne wejście użytkownika.		
Udziałowiec:	Zespół Ekey, Zespół Cyfrowe Klucze		
Wymagania powiązane:			

Tabela 6.15: Przekazywanie identyfikatora do backendu

KARTA WYMAGANIA			
Identyfikator:	WF-MAG-2	Priorytet:	Wysoki
Nazwa:	Przekazanie danych do systemu EkeyPJATK		
Opis:	System EkeyPJATK otrzymuje żądanie oceny dostępu użytkownika do wybranej sali na podstawie planu zajęć. Zwraca odpowiednią odpowiedź (przyznano, odrzucono, nieobsłużono).		
Udziałowiec:	Zespół Ekey, Zespół Cyfrowe Klucze		
Wymagania powiązane:	WF-MAG-1, WF-ACCESS-1, IO2		

Tabela 6.16: Obsługa kart tymczasowych

KARTA WYMAGANIA			
Identyfikator:	WF-MAG-3	Priorytet:	Średni
Nazwa:	Karty tymczasowe		
Opis:	SCD musi obsługiwać karty tymczasowe, które nie są powiązane z harmonogramem zajęć. W przypadku takich kart decyzja o dostępie powinna być podejmowana na podstawie reguł zdefiniowanych w systemie CK.		
Udziałowiec:	Zespół Cyfrowe Klucze		
Wymagania powiązane:	WF-ACCESS-5		

Tabela 6.17: Obsługa błędów odczytu karty magnetycznej

KARTA WYMAGANIA			
Identyfikator:	WF-MAG-4	Priorytet:	Średni
Nazwa:	Reakcja na nieudany odczyt karty		
Opis:	Czytnik musi wykrywać błędy odczytu karty i zgłaszać je użytkownikowi poprzez komunikat na ekranie oraz w logach systemowych EkeyPJATK. Błędny odczyt nie może być traktowany jako odmowa dostępu, musi zostać wyraźnie odróżniony od negatywnej decyzji systemu.		
Udziałowiec:	Zespół Ekey		
Wymagania powiązane:	WF-MAG-1		

Tabela 6.18: Rejestrowanie prób wejścia kartą magnetyczną

KARTA WYMAGANIA			
Identyfikator:	WF-MAG-5	Priorytet:	Wysoki
Nazwa:	Logowanie prób wejścia		
Opis:	System musi rejestrować każdą próbę wejścia kartą magnetyczną, niezależnie od wyniku. Log powinien zawierać: identyfikator karty, czas odczytu, identyfikator czytnika oraz wynik weryfikacji (błąd, odmowa, zgoda).		
Udziałowiec:	Zespół Ekey		
Wymagania powiązane:	WF-MAG-2, WF-ACCESS-5		

6.3 Wymagania zewnętrzne

[EkeyPJATK](#) musi przestrzegać reguł narzuconych z zewnątrz - od przepisów prawa, takich jak RODO i normy BHP, po techniczne specyfikacje innych systemów, z którymi się integruje. W tej sekcji zebraliśmy wszystkie te zewnętrzne ograniczenia i wymogi, które musieliśmy uwzględnić, aby nasze rozwiązanie było nie tylko funkcjonalne, ale także legalne, bezpieczne i kompatybilne z otoczeniem.

Tabela 6.19: Zgodność z przepisami bezpieczeństwa (BHP/ppož.)

KARTA WYMAGANIA			
Identyfikator:	NFO1	Priorytet:	Wysoki
Nazwa:	Zgodność z normami dla wyjść ewakuacyjnych		
Opis:	System EkeyPJATK oraz wykorzystywane urządzenia kontroli dostępu muszą spełniać obowiązujące normy bezpieczeństwa dotyczące elektrycznie sterowanych wyjść na drogach ewakuacyjnych. Urządzenia blokujące drzwi muszą działać w trybie <i>fail-safe</i> , a każde przejście musi być wyposażone w element awaryjnego otwarcia umożliwiający natychmiastowe odblokowanie niezależnie od działania systemu.		
Kryteria akceptacji:	• Zanik zasilania powoduje automatyczne odblokowanie drzwi. • Przycisk ewakuacyjny wymusza natychmiastowe otwarcie bez udziału backendu. • Otworzenie drzwi awaryjnie nie zależy od dostępności sieci ani serwera.		
Udziałowiec:	BSS , Personel techniczny		
Wymagania powiązane:			

Tabela 6.20: Wymagania dotyczące ochrony danych osobowych

KARTA WYMAGANIA			
Identyfikator:	NFO2	Priorytet:	Wysoki
Nazwa:	Zgodność z RODO		
Opis:	System musi przetwarzać dane osobowe w sposób zgodny z obowiązującymi przepisami RODO. Zakres danych musi być ograniczony do niezbędnego minimum (ID użytkownika, rola, sala, czas, wynik decyzji). Logi muszą posiadać jasno zdefiniowaną politykę retencji i przetwarzania.		
Kryteria akceptacji:	• Zaimplementowana polityka retencji danych (6-12 miesięcy, potem anonimizacja). • Przegląd danych wykazuje brak zbędnych informacji osobowych. • Dokument opisujący podstawę prawną przetwarzania dostępny dla użytkowników.		
Udziałowiec:	Dziekanat , Administracja Uczelni		
Wymagania powiązane:			

Tabela 6.21: Wymagania dotyczące infrastruktury sieciowej

KARTA WYMAGANIA			
Identyfikator:	NFO3	Priorytet:	Wysoki
Nazwa:	Wymagania sieciowe i zasilanie PoE		
Opis:	Urządzenia EkeyPJATK muszą działać w sieci uczelnianej z wykorzystaniem zasilania PoE . Komunikacja między urządzeniami, backendem a systemem Cyfrowe Klucze musi być realizowana poprzez bezpieczne protokoły (HTTPS, WebSocket TLS).		
Kryteria akceptacji:	• Każdy czytnik otrzymuje zasilanie PoE zgodnie ze standardem. • Cała komunikacja szyfrowana. • Urządzenie działa poprawnie w środowisku VLAN uczelni.		
Udziałowiec:	BSS , Zespół Cyfrowe Klucze , Zespół Ekey		
Wymagania powiązane:			

Tabela 6.22: Zewnętrzne wymagania integracyjne GAKKO

KARTA WYMAGANIA			
Identyfikator:	NFO4	Priorytet:	Wysoki
Nazwa:	Zgodność z API GAKKO		
Opis:	Integracja z GAKKO musi odbywać się zgodnie ze specyfikacją dostarczoną przez firmę zewnętrzną. System musi obsługiwać określony format API , limity zapytań, wymagania dotyczące autoryzacji oraz reagować poprawnie na błędy po stronie GAKKO .		
Kryteria akceptacji:	• Backend poprawnie obsługuje aktualny format odpowiedzi GAKKO. • Błędy są logowane i obsługiwane zgodnie z polityką retry. • Zmiana formatu danych po stronie GAKKO wymaga modyfikacji mapera tak aby zachować spójność bazy danych.		
Udziałowiec:	Zespół Ekey , GAKKO		
Wymagania powiązane:	WF-GAKKO-1		

Tabela 6.23: Wymagania zewnętrzne dotyczące integracji z Cyfrowe Klucze

KARTA WYMAGANIA			
Identyfikator:	NFO5	Priorytet:	Wysoki
Nazwa:	Kompatybilność z protokołem Cyfrowe Klucze		
Opis:	EkeyPJATK musi komunikować się z systemem Cyfrowe Klucze za pomocą oficjalnego WebSocket/API dostarczanego przez zespół Cyfrowe Klucze . Format komunikatów, sekwencje żądań oraz obsługa błędów muszą pozostać zgodne ze specyfikacją.		
Kryteria akceptacji:	• Backend wysyła decyzję o dostępie w formacie ustalonym z zespołem Cyfrowe Klucze. • W przypadku braku połączenia system przechodzi w tryb cache.		
Udziałowiec:	Zespół Cyfrowe Klucze , Zespół Ekey		
Wymagania powiązane:	WF-ACCESS-1 , WF-ACCESS-4		

Tabela 6.24: Wymagania bezpieczeństwa dla systemu EkeyPJATK

KARTA WYMAGANIA			
Identyfikator:	NFO6	Priorytet:	Wysoki
Nazwa:	Bezpieczeństwo aplikacji i danych		
Opis:	System musi zapewniać bezpieczeństwo danych, zgodnie z zasadą najmniejszych uprawnień. Kontrola dostępu oparta na rolach i harmonogramie, szyfrowanie danych, logowanie zdarzeń oraz cykliczne przeglądy bezpieczeństwa.		
Kryteria akceptacji:	• Wszyscy użytkownicy mają dostęp tylko do funkcji wynikających z roli. • Szyfrowanie wrażliwych informacji. • Logi bezpieczeństwa dostępne tylko dla administratorów.		
Udziałowiec:	BSS		
Wymagania powiązane:			

6.4 Interfejs z otoczeniem

Ta sekcja opisuje, jak wyglądają interfejsy, czyli punkty łączące nasz system z światem zewnętrznym. Definiujemy tu, jakie dane są przesyłane, w jakim formacie i jakie warunki muszą być spełnione, aby komunikacja przebiegała sprawnie. To kluczowe, by wszystkie elementy sprawnie współpracowały.

Tabela 6.25: Interfejs: moduł sprzętowy → system EkeyPJATK

KARTA WYMAGANIA			
Identyfikator:	IO1	Priorytet:	Wysoki
Nazwa:	Komunikacja danych odczytu karty magnetycznej		
Opis:	Moduł Sprzętowy musi komunikować się z systemem Cyfrowe Klucze poprzez WebSocket. Interfejs powinien umożliwiać przesłanie: identyfikatora karty, identyfikatora modułu sprzętowego, znacznika czasu oraz dodatkowych metadanych. Komunikacja powinna być szyfrowana.		
Kryteria akceptacji:	• System poprawnie odbiera i przetwarza dane z modułu sprzętowego. • Komunikacja jest szyfrowana i spełnia wymagania bezpieczeństwa. • Błędne payloady są wykrywane i raportowane.		
Dane wejściowe:	ID karty magnetycznej; ID modułu sprzętowego; timestamp próby wejścia.		
Warunki początkowe:	Czytnik posiada połączenie z siecią PoE ; interfejs backendu jest osiągalny.		
Warunki końcowe:	Dane są przekazane do systemu Cyfrowe Klucze , który następnie przesyła je do EkeyPJATK .		
Sytuacje wyjątkowe:	Brak odpowiedzi serwera; payload niepoprawny; brak łączności; skutkuje ponowieniem wysyłki przez moduł sprzętowego.		
Szczegóły implementacji:	WebSocket ; JSON jako format danych.		
Udziałowiec:	Zespół Cyfrowe Klucze , Zespół Ekey		
Wymagania powiązane:	WF-MAG-1 , WF-MAG-2		

Tabela 6.26: Interfejs: EkeyPJATK → Cyfrowe Klucze

KARTA WYMAGANIA			
Identyfikator:	IO2	Priorytet:	Wysoki
Nazwa:	Przekazywanie decyzji o dostępie do systemu CK		
Opis:	System EkeyPJATK musi przesyłać decyzję o dostępie do systemu Cyfrowe Klucze za pomocą dedykowanego interfejsu. Komunikat musi zawierać minimalny zestaw danych: identyfikator modułu sprzętowego , wynik decyzji (pozwól / odmów).		
Kryteria akceptacji:	• CK otrzymują i realizują decyzję w czasie rzeczywistym. • Brak błędów formatowania lub utraty danych. • W przypadku braku połączenia system przechodzi w tryb degradacji.		
Dane wejściowe:	ID użytkownika; ID modułu sprzętowego; decyzja logiczna o dostępie.		
Warunki początkowe:	Sesja WebSocket CK aktywna; użytkownik przeszedł weryfikację.		
Warunki końcowe:	CK wykonuje odblokowanie lub blokadę drzwi.		
Sytuacje wyjątkowe:	Zerwanie sesji; niepoprawny format; timeout.		
Szczegóły implementacji:	Dwukierunkowy WebSocket ; komunikaty JSON zgodne ze specyfikacją CK .		
Udziałowiec:	Zespół Cyfrowe Klucze , Zespół Ekey		
Wymagania powiązane:	WF-ACCESS-4 , NFO5		

Tabela 6.27: Interfejs: GAKKO → EkeyPJATK

KARTA WYMAGANIA			
Identyfikator:	IO3	Priorytet:	Wysoki
Nazwa:	Integracja z API GAKKO		
Opis:	EkeyPJATK musi korzystać z API udostępnionego przez firmę GAKKO. Interfejs musi umożliwiać pobieranie aktualnego planu zajęć, zgodnie z narzuconym formatem odpowiedzi API, autoryzacją i limitami zapytań narzuconymi przez dostawcę.		
Kryteria akceptacji:	• System Ekey poprawnie pobiera dane i aktualizuje bazę. • Wszelkie błędy odpowiednio rejestrowane.		
Dane wejściowe:	Zapytania HTTP GET; parametry autoryzacyjne.		
Warunki początkowe:	Dostępność API GAKKO.		
Warunki końcowe:	Zaktualizowane rekordy zajęć w bazie.		
Sytuacje wyjątkowe:	Brak odpowiedzi; błąd formatu; brak autoryzacji.		
Szczegóły implementacji:	REST API; JSON; harmonogram CRON.		
Udziałowiec:	GAKKO, Zespół Ekey		
Wymagania powiązane:	WF-GAKKO-1, WF-GAKKO-2, NFO4		

Tabela 6.28: Interfejs: Panel dziekanatu → Backend

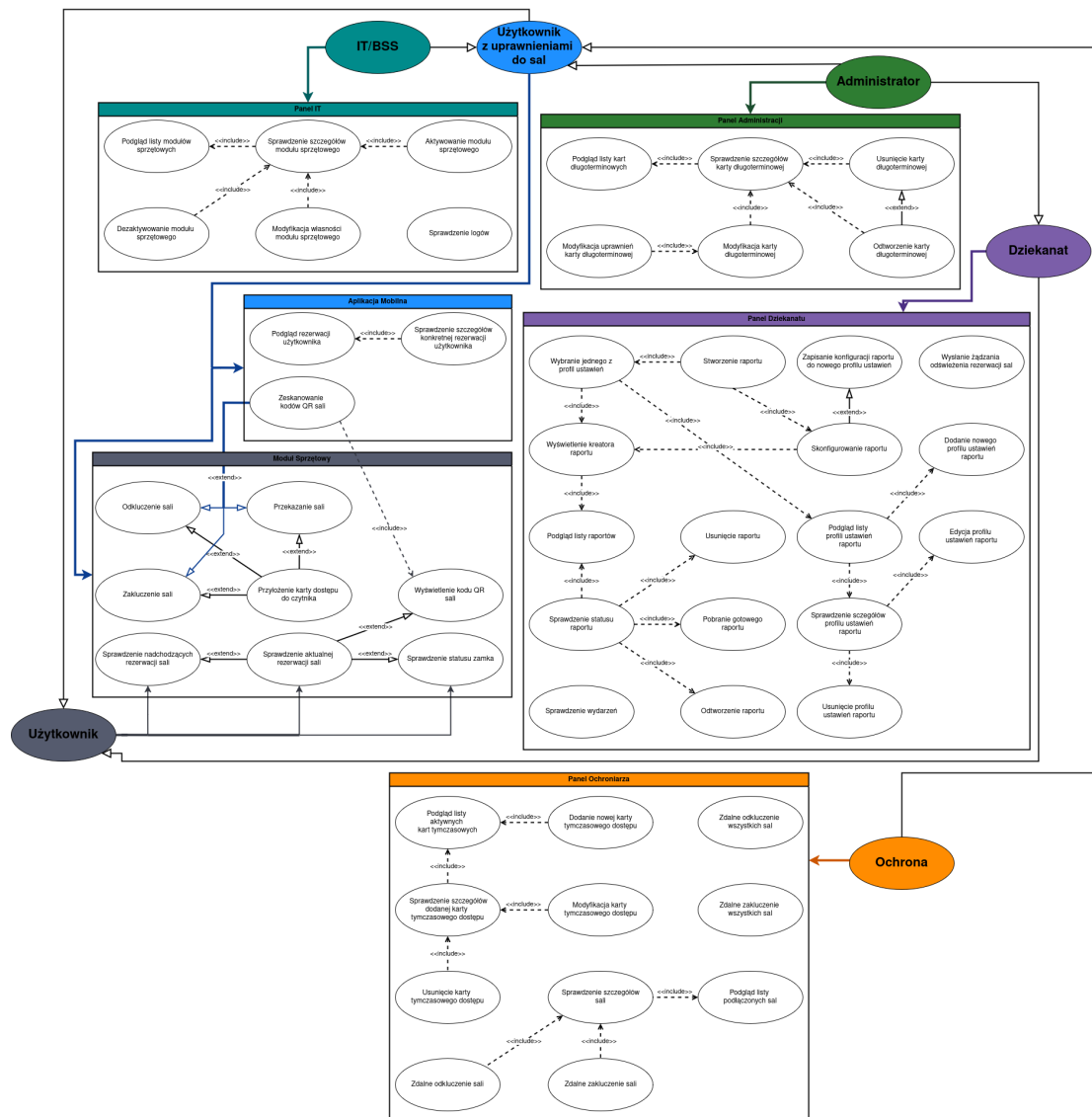
KARTA WYMAGANIA			
Identyfikator:	IO4	Priorytet:	Średni
Nazwa:	Panel dziekanatu		
Opis:	System musi udostępniać interfejs webowy umożliwiający pracownikom dziekanatu: tworzenie raportów odbytych zajęć, przeglądanie wydarzeń związanych z zajęciami. Panel musi być dostępny przez przeglądarkę bez konieczności instalacji dodatkowego oprogramowania.		
Kryteria akceptacji:	• Panel poprawnie prezentuje dane w czasie zbliżonym do rzeczywistego. • Logowanie.		
Dane wejściowe:	Żądania HTTP GET/POST; identyfikacja użytkownika.		
Warunki początkowe:	Użytkownik zalogowany; połączenie z backendem aktywne.		
Warunki końcowe:	Zmiany zapisane w systemie; logi działań uaktualnione.		
Sytuacje wyjątkowe:	Brak uprawnień; błędny format danych; komunikat o błędzie.		
Szczegóły implementacji:	Aplikacja webowa; REST API ; HTTPS; Architektura Heksagonalna		
Udziałowiec:	Dziekanat , Zespół Ekey		
Wymagania powiązane:	WO1 , NFO6		

Tabela 6.29: Interfejs: urządzenie → generowanie kodów QR

KARTA WYMAGANIA			
Identyfikator:	IO5	Priorytet:	Średni
Nazwa:	Wyświetlanie dynamicznego kodu QR na module sprzętowym		
Opis:	Urządzenie musi umożliwiać pobranie dynamicznego kodu QR generowanego przez backend EkeyPJATK oraz wyświetlenie go użytkownikowi. Kod QR musi być powiązany z konkretną salą, czasem oraz aktualnymi zajęciami.		
Kryteria akceptacji:	• Kod QR jest aktualny i ważny tylko w określonym czasie. • Błędne pobranie kodu wyświetla komunikat o niedostępności. • Aplikacja CKM poprawnie rozpoznaje zakodowane dane.		
Dane wejściowe:	ID sali; timestamp; token sesji.		
Warunki początkowe:	Czytnik połączony z backendem.		
Warunki końcowe:	Kod wyświetlony na ekranie i gotowy do skanowania.		
Sytuacje wyjątkowe:	Brak łączności; timeout; błędny token; odmowa wyświetlenia.		
Szczegóły implementacji:	REST API; QR		
Udziałowiec:	Zespół Cyfrowe Klucze Mobile, Zespół Ekey		
Wymagania powiązane:	WF-ACCESS-1, WF-ACCESS-2		

6.5 Przypadki użycia

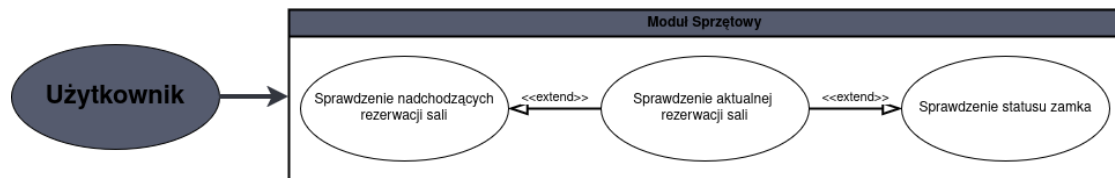
System Cyfrowego Dostępu PJATK został zaprojektowany w sposób umożliwiający wyraźne rozdzielenie ról oraz zakresów odpowiedzialności poszczególnych użytkowników. Diagram **przypadków użycia** przedstawia główne grupy aktorów, ich uprawnienia oraz interakcje z systemem, podkreślając różnice w dostępnych funkcjach i poziomach dostępu.



Rysunek 6.1: Diagram przypadków użycia.

6.5.1 Podstawowi użytkownicy

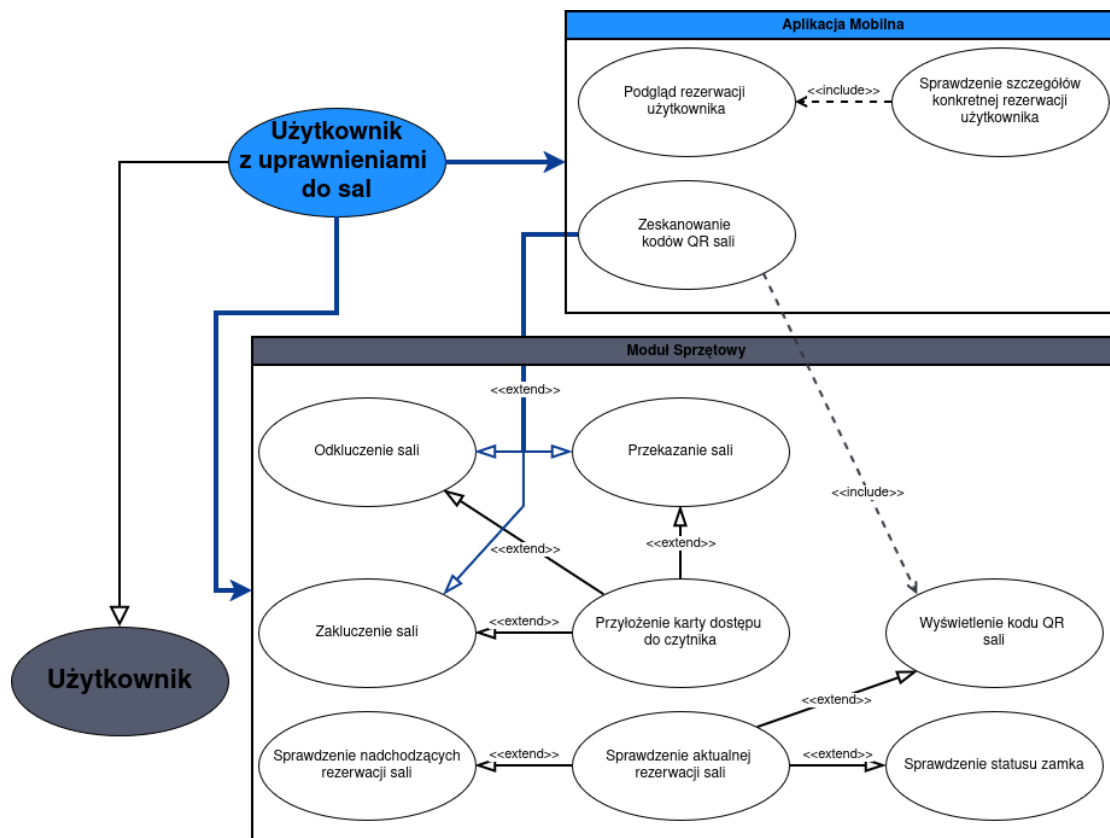
Podstawowi użytkownicy - w szczególności studenci oraz goście bez rezerwacji - posiadają ograniczony zakres interakcji z systemem. Mogą jedynie przeglądać bieżące i nadchodzące rezerwacje oraz sprawdzać status zamka sali obsługiwanej przez [Moduł Sprzętowy](#). Ich rola ma charakter informacyjny i nie obejmuje możliwości wpływania na działanie systemu.



Rysunek 6.2: Diagram przypadków użycia dla podstawowych użytkowników.

6.5.2 Użytkownicy z uprawnieniami do sal

Znacznie szerszy zakres funkcjonalności posiadają użytkownicy z uprawnieniami dostępu do sal - m.in. wykładowcy, goście posiadający rezerwacje oraz personel techniczny i sprzątający. Dysponują oni możliwością otwierania sal za pomocą [kart magnetycznych](#) (zarówno krótkoterminowych, jak i długoterminowych), przejmowania odpowiedzialności za salę po poprzednim użytkowniku oraz korzystania z aplikacji mobilnej w celu przeglądania szczegółów rezerwacji. Integracja z kodami [QR](#) zapewnia dodatkową elastyczność w dostępie. W większości przypadków konieczne jest posiadanie aktywnej rezerwacji, jednak niektóre role - takie jak personel sprzątający czy techniczny - mają dostęp ogólny, niezależny od harmonogramu zajęć.



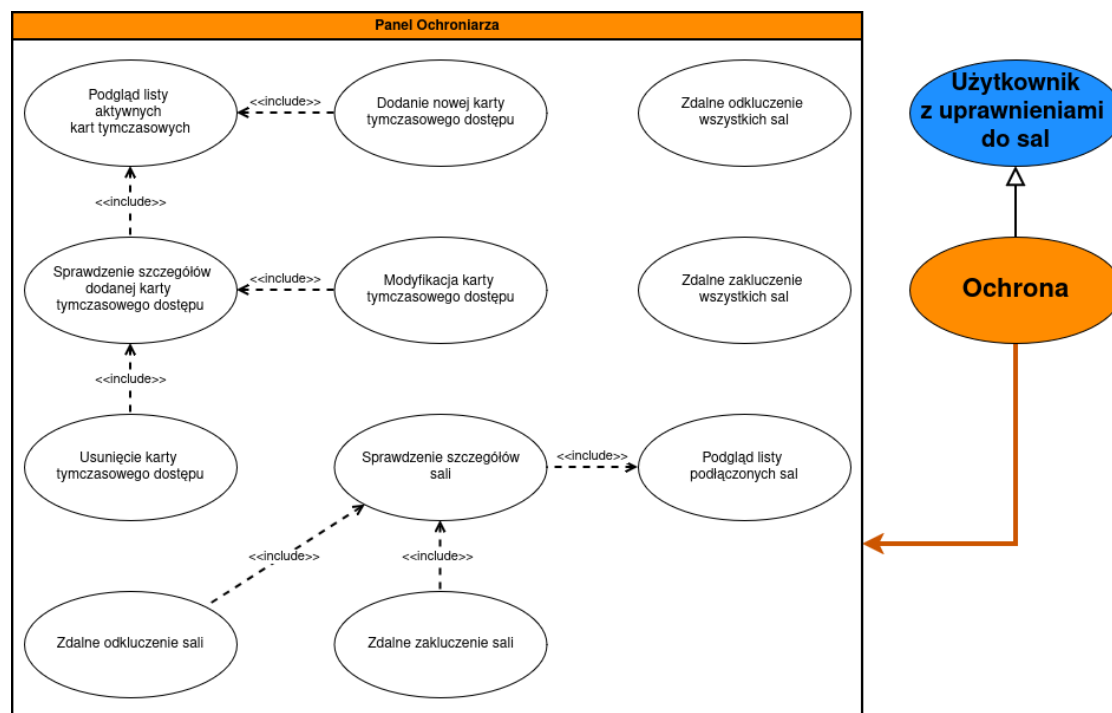
Rysunek 6.3: Diagram przypadków użycia dla użytkowników z uprawnieniami do sal.

6.5.3 Dziekanat

Dziekanat korzysta z dedykowanego panelu administracyjnego, który umożliwia przegląd ostatnich zdarzeń w systemie, ręczne odświeżanie rezerwacji oraz generowanie raportów z odbytych zajęć. Rozbudowane narzędzia raportowania, w tym możliwość konfiguracji i zapisu profili raportów, zapewniają szybki dostęp do potrzebnych zestawień i ułatwiają bieżące zarządzanie procesem dydaktycznym.

6.5.4 Ochrona

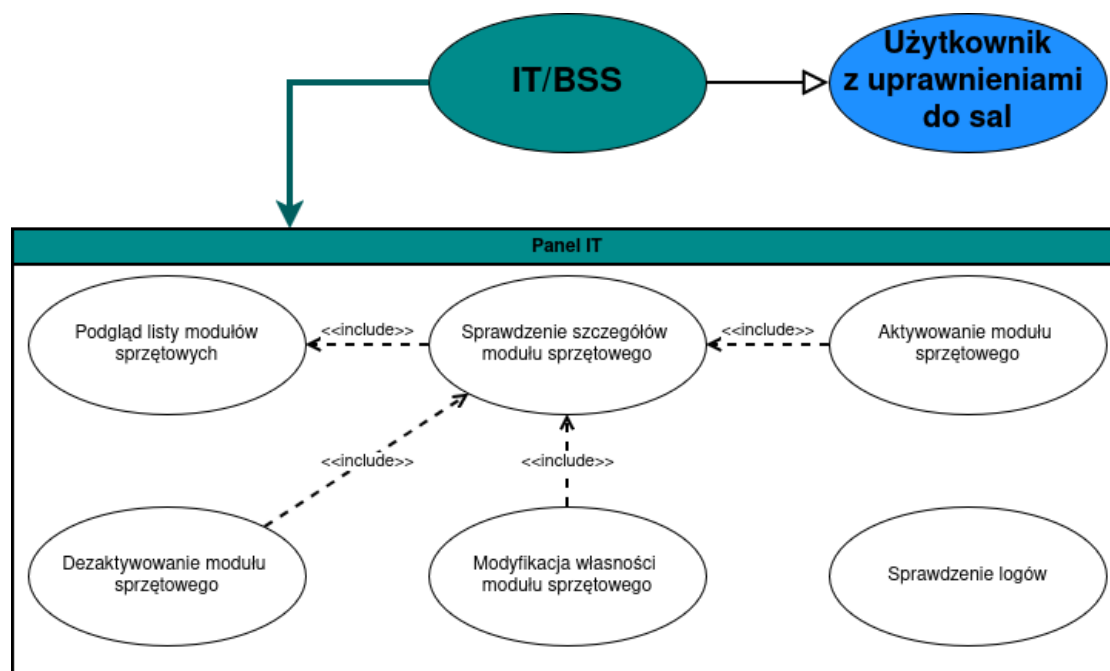
Ochrona dysponuje specjalnym panelem operacyjnym obejmującym funkcje zdalnego zarządzania salami, obsługi **kart magnetycznych** tymczasowego dostępu oraz awaryjnego blokowania i odblokowywania wszystkich pomieszczeń. Rola ta łączy odpowiedzialność operacyjną z możliwością natychmiastowego reagowania w sytuacjach kryzysowych, jednocześnie dziedzicząc uprawnienia przysługujące użytkownikom z rezerwacjami.



Rysunek 6.5: Diagram przypadków użycia dla ochrony.

6.5.5 IT/BSS

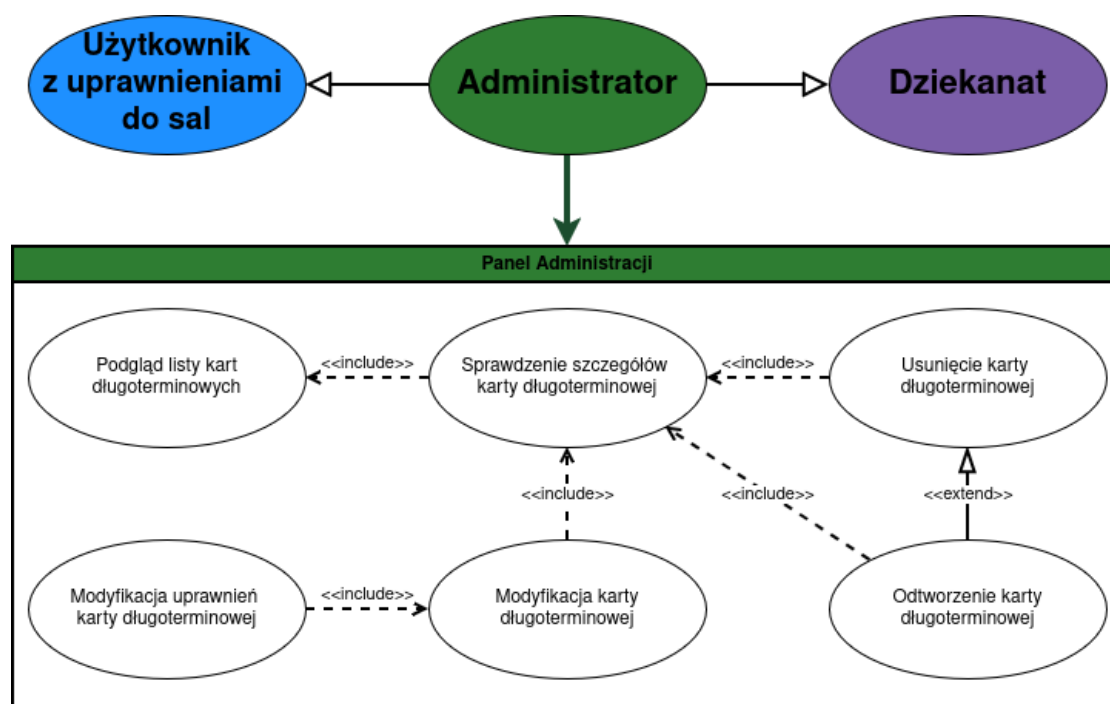
Zespół IT/BSS odpowiada za utrzymanie techniczne i nadzór nad poprawnym działaniem systemu. Panel tej roli zawiera narzędzia służące do aktywacji i konfiguracji modułów sprzętowych, podglądu listy aktywnych urządzeń oraz analizy logów systemowych. Dzięki temu zespół IT/BSS zapewnia stabilność działania i umożliwia szybkie reagowanie na ewentualne problemy techniczne.



Rysunek 6.6: Diagram przypadków użycia dla IT/BSS.

6.5.6 Administracja

Administratorzy dziedziczą uprawnienia użytkowników z dostępem do sal oraz dziekanatu, a ponadto posiadają możliwość zarządzania długoterminowymi **kartami magnetycznymi**. Ich panel umożliwia dodawanie, usuwanie i modyfikację kart, co czyni ich rolę kluczową dla zapewnienia integralności i bezpieczeństwa systemu.



Rysunek 6.7: Diagram przypadków użycia dla administratorów.

Tak zdefiniowane role i przypadki użycia tworzą spójny obraz funkcjonowania [Systemu Cyfrowego Dostępu](#). Diagram 6.1 stanowi wizualne podsumowanie tej struktury, ukazując relacje pomiędzy użytkownikami a systemem oraz hierarchię dostępnych uprawnień.

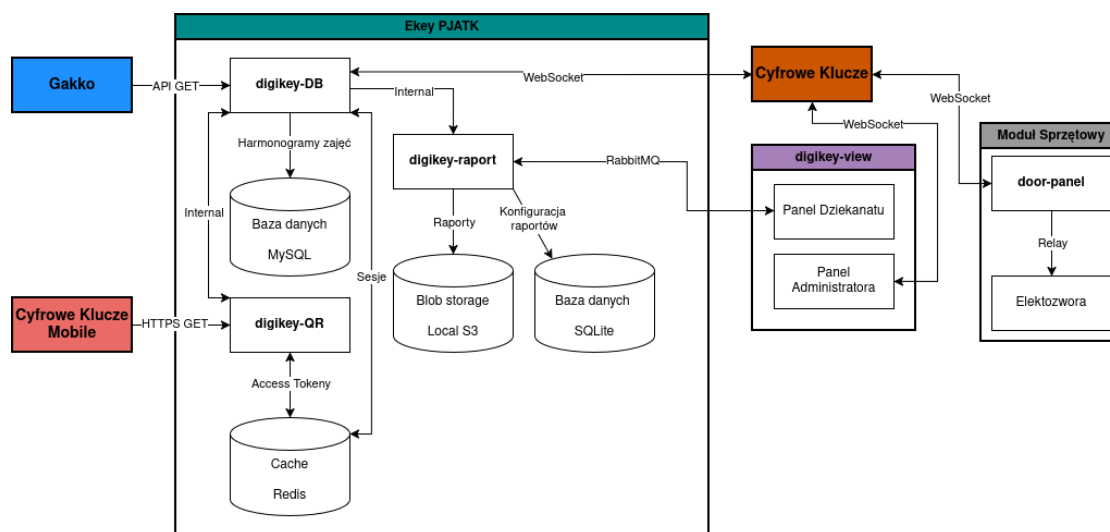
Rozdział 7

Projektowanie systemu

7.1 Architektura Systemu

Zgodnie z założeniami programowania ekstremalnego, szczegółowe wymagania systemu [EkeyPJATK](#) były doprecyzowywane iteracyjnie, podczas kolejnych spotkań projektowych. Jednocześnie od początku ustalono kluczowy kierunek w projektowaniu architektury systemu: system miał zostać zbudowany w oparciu o modularną architekturę. Pozwoliło to projektować funkcjonalności równolegle z dyskusjami dotyczącymi sposobu działania poszczególnych elementów. W celu lepszego zobrazowania przepływów i zależności przygotowano [diagram architektury systemowej](#).

Ze względu na ścisłą współpracę z [Cyfrowe Klucze](#), [Cyfrowe Klucze Mobile](#) oraz [GAKKO](#), system [Ekey](#) został zaprojektowany tak, aby zapewnić maksymalną kompatybilność z istniejącymi elementami [SCD](#).

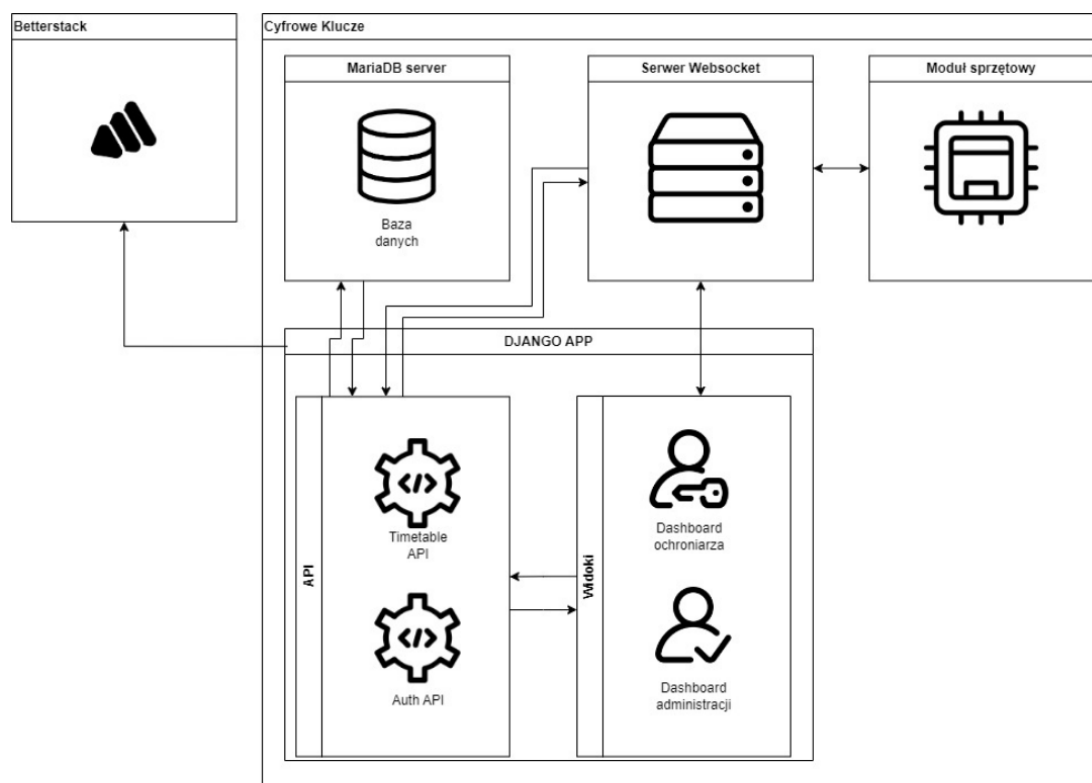


Rysunek 7.1: Architektura [Systemu Cyfrowego Dostępu](#) z wyróżnionymi komponentami [EkeyPJATK](#) i ich powiązaniami.

7.1.1 Integracja z infrastrukturą Cyfrowych Kluczy

System [Cyfrowe Klucze](#) wykorzystuje komunikację opartą na [WebSocket](#) do wymiany danych pomiędzy swoimi komponentami. Umożliwiło to bezpośrednie połączenie się do istniejącego kanału [WebSocket](#) w celu integracji zaktualizowanych [modułów sprzętowych](#) oraz panelu administracyjnego z infrastrukturą [CK](#).

Oprogramowanie zostało zaprojektowane do wdrożenia w oparciu o model system-on-premises (SoP) (Rys. 39) z dodatkowym wsparciem zewnętrznego systemu monitorowania zdarzeń oraz przechowywania i analizy logów (Betterstack). Dzięki takiemu podejściu, zapewniamy działanie wszystkich kluczowych funkcji systemu, niezależnie od aktualnego stanu połączenia z Internetem. [[CK](#), rozdział 8.1].

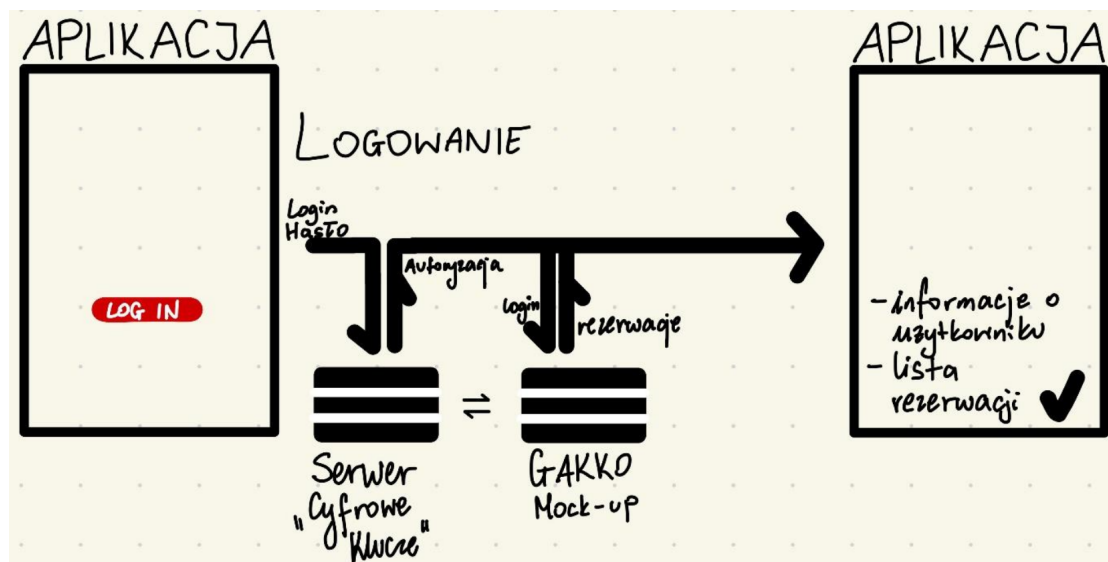


Rys. 39 Diagram systemu

Rysunek 7.2: Architektura [Cyfrowe Klucze](#) przed integracją w [Systemie Cyfrowego Dostępu PJATK](#) [[CK](#), rozdział 8.1.1].

7.1.2 Integracja z Cyfrowymi Kluczami Mobile

Ze względu na trudności integracyjne po stronie [Ekey](#), w dokumentacji [CKM](#) przyjęto założenie bezpośredniej integracji z [CK](#) i [GAKKO](#), najlepiej widzianym na [diagramie ogólnej komunikacji między modułowej podczas logowania](#).

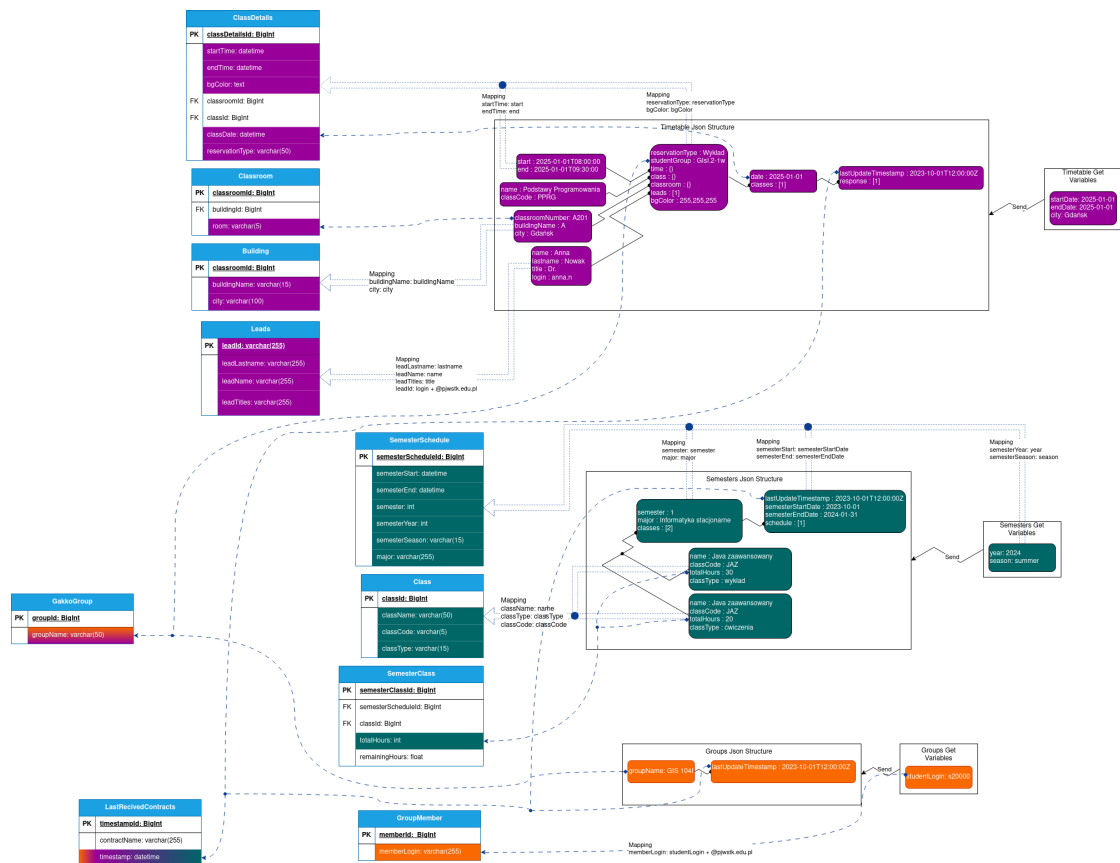


Rysunek 7.3: Interakcja systemów podczas logowania w [Cyfrowe Klucze Mobile](#) [CKM, rozdział 8.2.1].

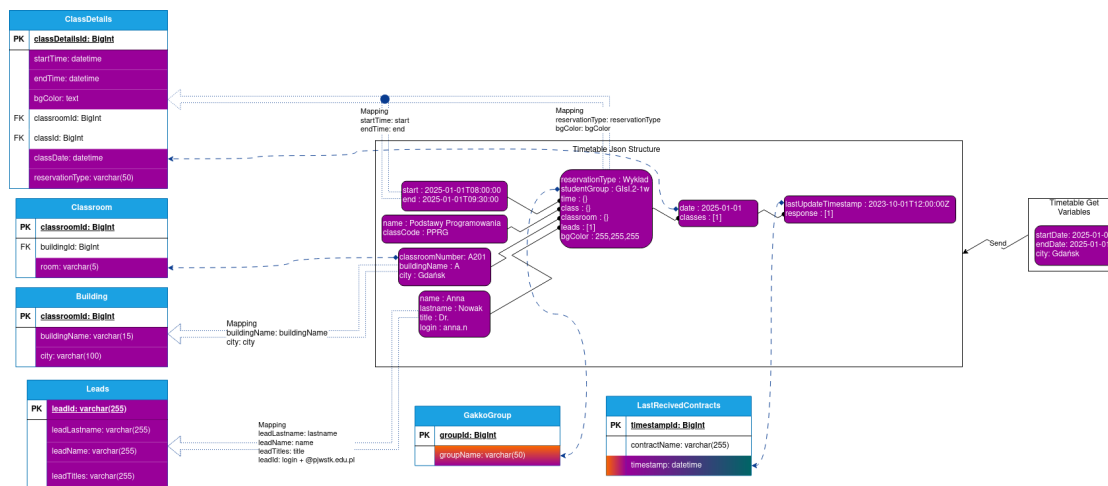
Wprowadzonym rozwiązaniem, które umożliwiło pełną integrację CKM w ramach SCD, było zastosowanie Ekey jako warstwy pośredniczącej pomiędzy CK, GAKKO i CKM. Pozwoliło to uprościć komunikację i ujednolicić przepływ danych między systemami.

7.1.3 Połączenie z Gakko

Integracja z GAKKO stanowi kluczowy element projektu EkeyPJATK. Wstępnie zaplanowano trzy kontrakty oraz Webhook, który umożliwia natychmiastową aktualizację harmonogramu rezerwacji po stronie Ekey w momencie wprowadzenia zmian w GAKKO.



Kontrakt rezerwacji



Rysunek 7.5: Diagram kontraktu rezerwacji od GAKKO z mapowaniem do bazy danych.

```

1 {
2     "lastUpdateTimestamp": "2023-10-01T12:00:00Z",
3     "response": [
4         {
5             "date": "2025-01-01",
6             "classes": [
7                 {
8                     "reservationType": "Wykład",
9                     "studentGroup": "GisI.2-1w",
10                    "time": {
11                        "start": "2025-01-01T08:00:00Z",
12                        "end": "2025-01-01T09:30:00Z"
13                    },
14                    "class": {
15                        "name": "Podstawy Programowania",
16                        "semester": 1,
17                        "classCode": "PPRG"
18                    },
19                    "classroom": {
20                        "classroomNumber": "A201",
21                        "buildingName": "A",
22                        "city": "Gdańsk"
23                    },
24                    "leads": [
25                        {
26                            "name": "Anna",
27                            "lastname": "Nowak",
28                            "title": "Dr.",
29                            "login": "anna.n"

```

```

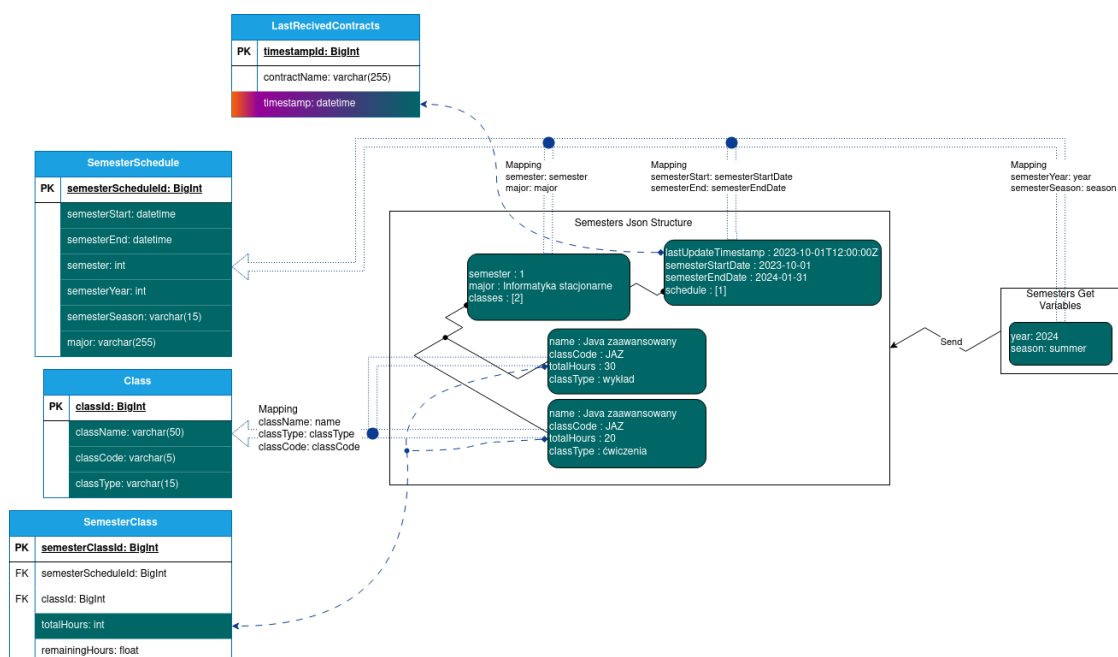
30     }
31     ],
32     "bgColor": "255,255,255"
33 }
34 ]
35 }
36 ]
37 }
38

```

Listing 7.1: Przykład konfiguracji kontraktu rezerwacji

Kontrakt semestralny

Kontrakt ten rozszerza moduł raportowania o dane dotyczące liczby godzin przypisanych do danego przedmiotu. Pozwala to precyzyjniej raportować liczbę przeprowadzonych godzin zajęć przez wykładowców oraz wykrywać przypadki rozbieżności pomiędzy harmonogramem a rzeczywistymi danymi.



Rysunek 7.6: Diagram kontraktu semestralnego od GAKKO z mapowaniem do bazy danych.

```

1 {
2     "lastUpdateTimestamp": "2023-10-01T12:00:00Z",
3     "response":{
4         "semesterStartDate": "2023-10-01",
5         "semesterEndDate": "2024-01-31",
6         "schedule":[
7             {

```

```

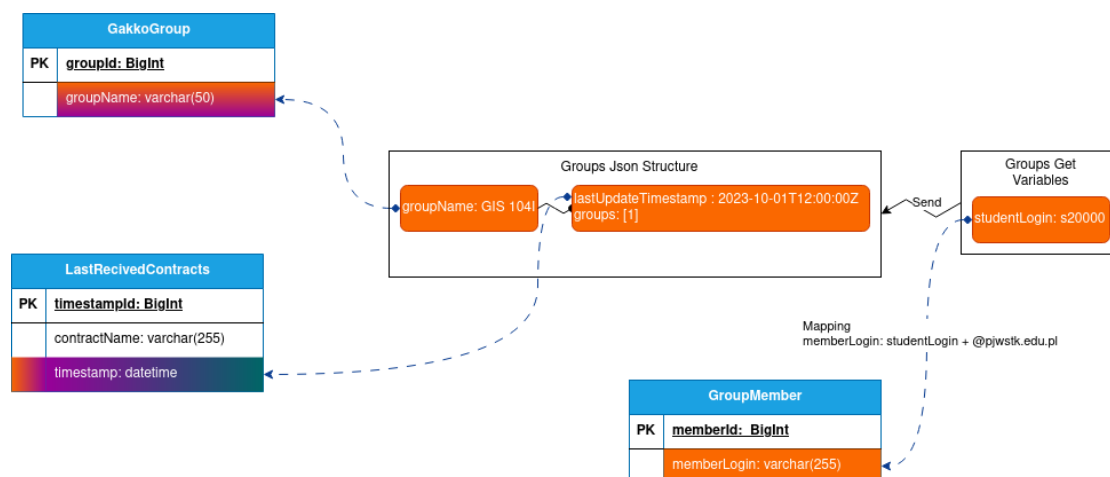
8         "semester": 1,
9         "major": "Informatyka stacjonarne",
10        "classes": [
11            {
12                "name": "Java zaawansowany",
13                "classCode": "JAZ",
14                "totalHours": 30,
15                "classType": "wykład",
16                "spec": null
17            },
18            {
19                "name": "Java zaawansowany",
20                "classCode": "JAZ",
21                "totalHours": 20,
22                "classType": "ćwiczenia",
23                "spec": null
24            }
25        ]
26    }
27 ]
28 }
29 }
30

```

Listing 7.2: Przykład konfiguracji kontraktu semestralnego

Kontrakt grup uczelnianych

Kontrakt ten umożliwia jednoznaczne określenie grup, do których należy użytkownik. Otwiera to możliwość rozszerzenia funkcjonalności [Cyfrowe Klucze Mobile](#) o dostęp dla studentów, w tym przegląd planu zajęć oraz potencjalne otwieranie sal, w których mają zaplanowane zajęcia.



Rysunek 7.7: Diagram kontraktu grup uczelnianych od GAKKO z mapowaniem do bazy danych.

```

1 {
2   "lastUpdateTimestamp": "2023-10-01T12:00:00Z",
3   "groups": [
4     {
5       "groupName": "GIS 1041"
6     }
7   ]
8 }
9

```

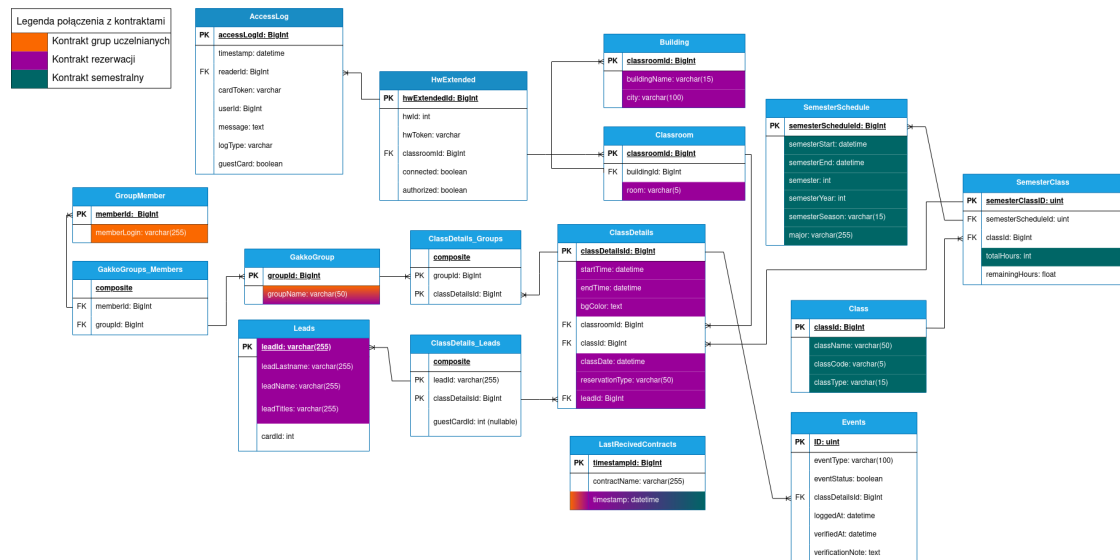
Listing 7.3: Przykład konfiguracji kontraktu grup uczelnianych

7.1.4 Bazy danych EkeyPJATK

Zgodnie z zasadami bezpiecznego przechowywania danych oraz minimalizacji ich duplikacji, w systemie Ekey utworzono trzy bazy danych.

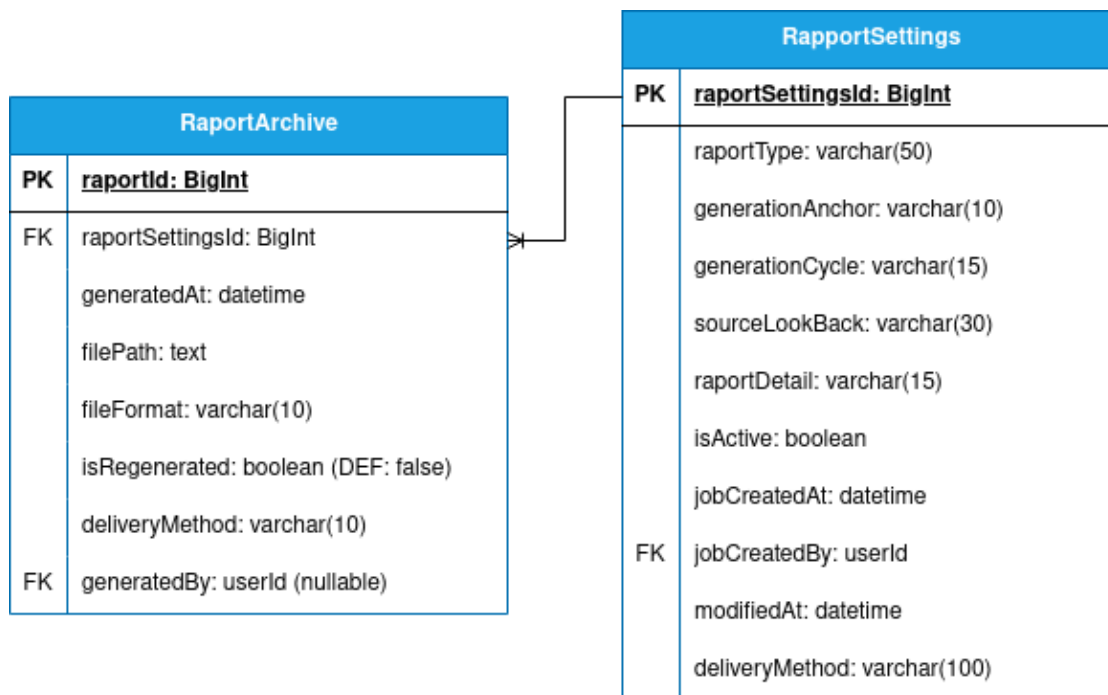
Najmniejszą z nich jest baza **Redis**, przechowująca dane tymczasowe wykorzystywane podczas autoryzacji użytkowników aplikacji Cyfrowe Klucze Mobile.

Główna baza danych, odpowiedzialna za przechowywanie harmonogramu oraz wykazu przedmiotów w semestrze, została zrealizowana w technologii **MySQL**. Wybór ten był podyktowany zgodnością z rozwiązaniami stosowanymi w Cyfrowe Klucze.



Rysunek 7.8: Diagram głównej bazy danych **EkeyPJATK** ze zaznaczonymi połączeniami z kontraktami.

Ze względu na rozbudowaną funkcjonalność panelu dziekanatu oraz generatora raportów odbytych zajęć, utworzono dodatkową bazę danych w technologii **SQLite**, dedykowaną obsłudze modułu raportowania.



Rysunek 7.9: Diagram bazy danych raportów [EkeyPJATK](#).

7.2 Technologie i ewolucja stosu technologicznego

Projekt [EkeyPJATK](#) od początku zakładał iteracyjny rozwój oraz wysoki poziom modularności. Dobór technologii nie był jednorazową decyzją, lecz procesem, który ewoluował wraz ze zmianami wymagań, integracji z systemami zewnętrznymi oraz doświadczeniami zespołu projektowego.

7.2.1 Modularność i struktura repozytoriów

Modularność systemu była uwzględniona już na najwyższym poziomie organizacji kodu. Wszystkie moduły powstawały równolegle i od początku były umieszczane we wspólnym repozytorium typu *monorepo*. W celu zachowania izolacji komponentów oraz umożliwienia ich niezależnego wersjonowania zastosowano mechanizm **git submodule**. Rozwiązanie to pozwoliło:

- rozwijać moduły niezależnie od siebie,
- testować je w odseparowanych środowiskach,
- zachować spójność wersji w całym systemie,
- uprościć proces integracji i wdrażania.

7.2.2 Pierwotny stos technologiczny

We wczesnej fazie projektu przyjęto architekturę , wynikającą z planowanego wykorzystania frameworka **Ruby on Rails**. Wstępny podział technologiczny obejmował:

- **Ruby on Rails** - jako główny framework dla kluczowych modułów systemu (panel administracyjny, logika harmonogramów, integracje),
- **Go (Golang)** - dla modułów pomocniczych, takich jak [mocki](#) systemów [Cyfrowe Klucze](#) i [GAKKO](#),
- **PostgreSQL** - jako główna baza danych, naturalnie współpracująca z **Rails**.

Takie podejście umożliwiało szybkie prototypowanie, jednak w miarę rozwoju projektu zaczęło ujawniać swoje ograniczenia.

7.2.3 Problemy z pierwotnym podejściem

Wraz z rosnącą złożonością systemu pojawiły się trudności, które wymusiły zmianę stosu technologicznego:

- skomplikowana konfiguracja środowiska **Ruby on Rails**,
- wysoka dynamika zmian wymagań, utrudniająca utrzymanie spójności kodu,
- duplikacja logiki pomiędzy modułami **Rails** i **Go**,
- konieczność zachowania kompatybilności z [Cyfrowe Klucze](#), które wykorzystywały **MySQL**,
- ograniczenia architektury **MVC** w kontekście integracji z wieloma systemami zewnętrznymi.

7.2.4 Migracja na Go i unifikacja technologii

Po analizie kosztów utrzymania i złożoności integracji podjęto decyzję o:

- całkowitym przejściu na język **Go** jako jedyną technologię backendową,
- rezygnacji z **Ruby on Rails**,
- ujednoliceniu architektury modułów w oparciu o wzorce projektowe dopasowane do mikroservisów.

Zmiana ta uprościła środowisko developerskie, zwiększyła wydajność oraz ułatwiła integrację z systemami [Cyfrowe Klucze](#) i [GAKKO](#). Dodatkowo wykorzystanie natywnej współbieżności **Go** okazało się szczególnie korzystne w kontekście obsługi wielu równoległych integracji i zadań harmonogramowych.

7.2.5 Zmiana architektury: od MVC do architektury heksagonalnej

We wczesnej fazie projektu moduły systemu były projektowane zgodnie z architekturą [MVC](#), co wynikało z pierwotnego planu wykorzystania frameworka **Ruby on Rails**. Podejście to sprawdzało się podczas szybkiego prototypowania, jednak wraz z rozwojem systemu zaczęło ujawniać swoje ograniczenia, szczególnie w kontekście integracji z wieloma systemami zewnętrznymi oraz konieczności utrzymania spójnej logiki domenowej.

Po migracji całego systemu na język **Go** podjęto decyzję o zastosowaniu architektury [heksagonalnej \(Ports and Adapters\)](#), jednak wyłącznie w **głównym module backendu**. Moduł ten pełni rolę centralnego komponentu systemu [Ekey-PJATK](#), odpowiedzialnego za:

- komunikację z pozostałymi modułami systemu,
- integrację z [GAKKO](#),
- integrację z [Cyfrowe Klucze](#) oraz [Cyfrowe Klucze Mobile](#),
- przetwarzanie logiki domenowej związanej z harmonogramem i dostępem do sal.

Zastosowanie architektury heksagonalnej w tym module pozwoliło na:

- pełną izolację logiki domenowej od warstw komunikacji (HTTP, [WebSocket](#), kontrakty zewnętrzne),
- łatwe tworzenie adapterów dla różnych systemów zewnętrznych,
- testowanie logiki domenowej w całkowitej izolacji od infrastruktury,
- uproszczenie integracji z wieloma źródłami danych i protokołami,
- zwiększenie elastyczności w kontekście przyszłych zmian technologicznych.

Pozostałe moduły [EkeyPJATK](#) zachowały prostszą strukturę, dopasowaną do ich roli pomocniczej (np. moduły sprzętowe, [mocki](#), narzędzia integracyjne). Dzięki temu architektura [Heksagonalna](#) została zastosowana tam, gdzie przynosiła największą wartość - w centralnym komponencie odpowiedzialnym za logikę domenową i komunikację między systemami.

7.2.6 Ewolucja technologii baz danych

Pierwotnie wybrana baza danych **PostgreSQL** została zastąpiona przez **MySQL**, aby zachować pełną kompatybilność z infrastrukturą [Cyfrowe Klucze](#). Zmiana ta umożliwiła:

- współdzielenie części modeli danych,
- uproszczenie integracji,
- wykorzystanie istniejących narzędzi i konfiguracji.

Dodatkowo w systemie zastosowano:

- **Redis** - do przechowywania danych tymczasowych i sesyjnych, głównie na potrzeby autoryzacji w [Cyfrowe Klucze Mobile](#),
- **SQLite** - jako lekką bazę danych dla modułu raportowania, umożliwiającą szybkie generowanie zestawień bez obciążania głównej bazy.

7.3 Projekt interfejsu użytkownika (UI)

Proces projektowania interfejsu użytkownika dla systemu EkeyPJATK od początku skupił się na potrzebach pracowników Dziekanatu i jak najpłynniejszej integracji z ich istniejącym przepływem pracy. Postawiliśmy na prostotę, czytelność i minimalizację liczby kroków potrzebnych do interakcji z naszym systemem tak aby nie dodawać dodatkowego obciążenia użytkownikom końcowym. System nie może wymagać długotrwałego szkolenia ani wymuszać zmiany przyzwyczajeń. Funkcje mają być czytelne, łatwo dostępne, a nawigacja intuicyjna.

Prace rozpoczęto od przygotowania makiet niskiej wierności (Lo-Fi) w środowisku Figma. Pozwoliło to na szybką iterację pomysłów bez rozpraszania się aspektami wizualnymi. Poniżej przedstawiono kluczowe widoki koncepcyjne, które stały się fundamentem dla ostatecznego wyglądu aplikacji.

7.3.1 Moduł zarządzania rezerwacjami

Centralnym punktem pracy Dziekanatu z naszej perspektywy jest harmonogram zajęć. Projektując widok rezerwacji (Rysunek 7.10), zrezygnowano z tabelarycznych list na rzecz wizualnej osi czasu. Takie podejście pozwala użytkownikowi natychmiastowo zidentyfikować okienka w planie zajęć oraz uniknąć nakładania się na siebie rezerwacji.

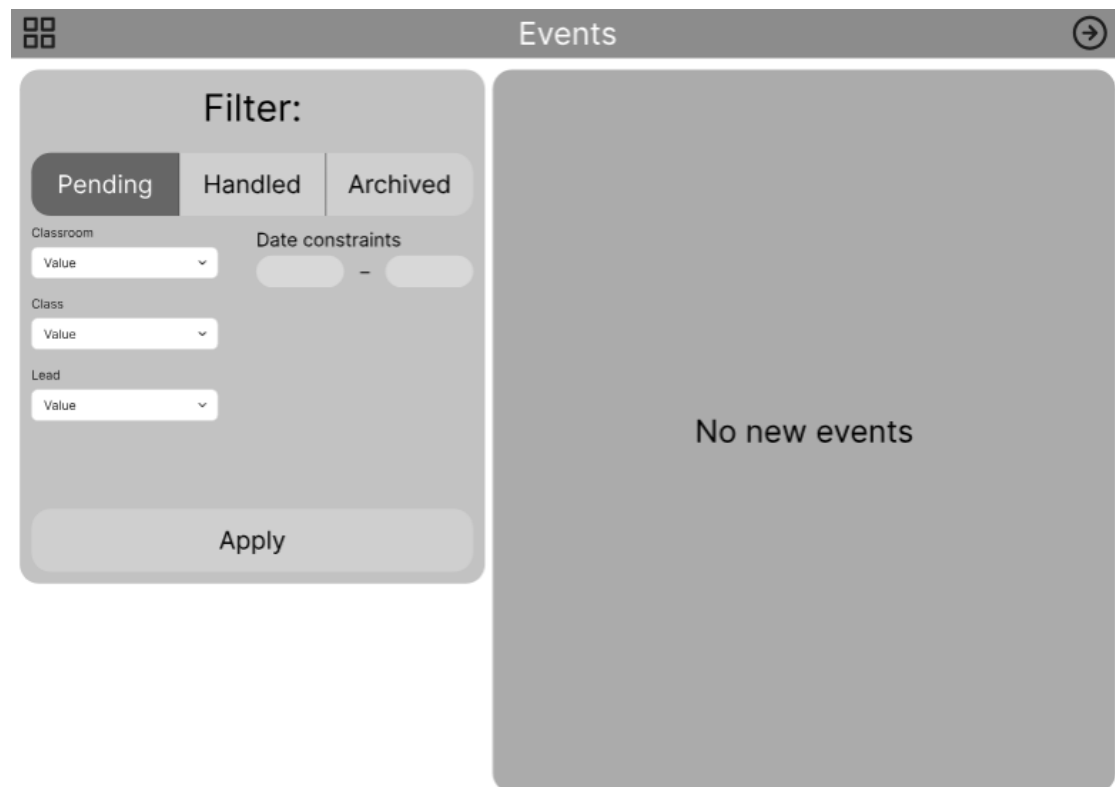
Panel boczny służy do szybkiego wprowadzania nowych rezerwacji. Formularz został uproszczony do niezbędnego minimum, a pola wyboru dropdown sugerują dostępne opcje, co minimalizuje ryzyko wprowadzenia błędnych danych.

Rysunek 7.10: Projekt Lo-Fi widoku zarządzania rezerwacjami.

7.3.2 Obsługa zdarzeń i zgłoszeń

Dla zapewnienia płynności obsługi incydentów (np. próby wejścia poza godzinami), zaprojektowano widok „Events” (Rysunek 7.11). Kluczowym elementem jest tutaj workflow oparty na stanach: *Oczekujące (Pending)*, *Obsłużone (Handled)* oraz *Zarchiwizowane (Archived)*. Taki podział pozwala pracownikowi administracji skupić się wyłącznie na aktualnej sytuacji, jasno pokazując, które sprawy wymagają interwencji. Filtry po lewej stronie umożliwiają szybkie zawężenie listy do konkretnej

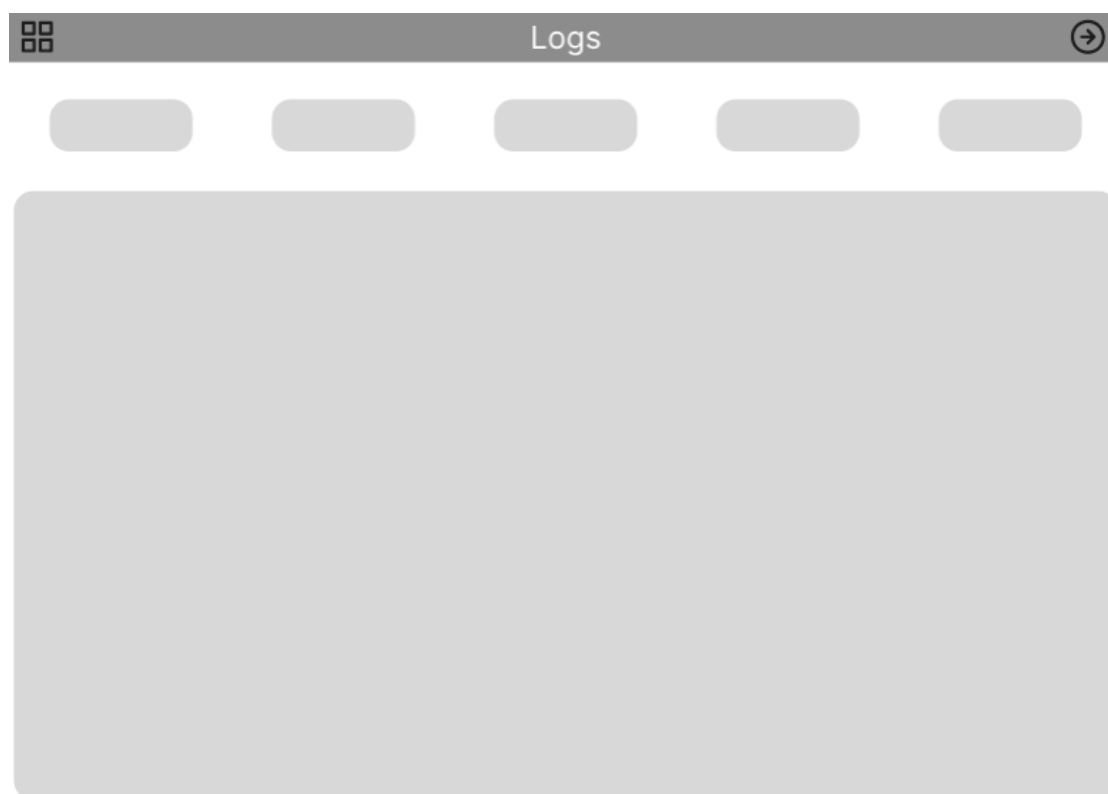
sali lub prowadzącego, co usprawnia pracę w przypadku większej liczby zgłoszeń.



Rysunek 7.11: Koncepcja widoku obsługi zdarzeń z filtrowaniem.

7.3.3 Dziennik zdarzeń (Logi)

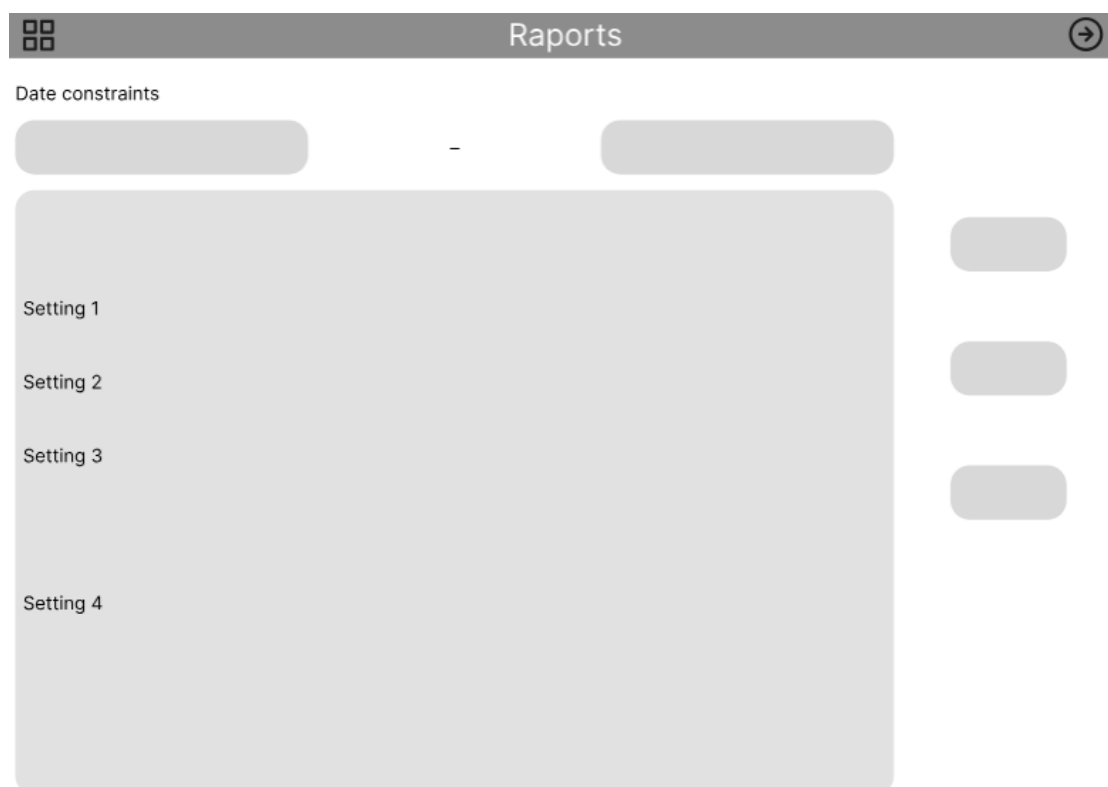
Wgląd w historię wejść jest niezbędny z punktu widzenia bezpieczeństwa jak i generowania raportów, jednak standardowe logi mogą być nieczytelne dla osób nietechnicznych. Projekt widoku logów (Rysunek 7.12) zakłada prezentację danych w formie czytelnej listy z kluczowymi dla użytkownika informacjami.



Rysunek 7.12: Projekt interfejsu przeglądarki logów.

7.3.4 Generator raportów

Funkcja raportowania (Rysunek 7.13) została zaprojektowana z myślą o automatyzacji cyklicznych obowiązków administracyjnych. Oszczędziliśmy użytkownikom tworzenia zapytań SQL, otrzymuje prosty interfejs z wyborem zakresu dat oraz predefiniowanych ustawień. Celem tego widoku jest umożliwienie wygenerowania potrzebnego dokumentu w intuicyjny sposób.



Rysunek 7.13: Koncepcja modułu generowania raportów.

7.3.5 Ewolucja projektu

Przedstawione powyżej makiety Lo-Fi stanowiły punkt wyjścia do dyskusji z interesariuszami. Uwagi zebrane na tym etapie pozwoliły na wyeliminowanie potencjalnych problemów użyteczności jeszcze przed przystąpieniem do pracy nad frontendem. Ostateczna wersja aplikacji (Hi-Fi), prezentowana w dalszej części pracy, jest bezpośrednim rozwinięciem tych koncepcji, wzbogaconym o design graficzny pasujący do stylu uczelni.

7.4 Implementacja interfejsu (Hi-Fi)

Na bazie zweryfikowanych prototypów Lo-Fi przystąpiono do stworzenia docelowego frontendu systemu. Proces przejścia od koncepcji projektowych do działającego interfejsu wymagał przełożenia oszczędnych szkiców z Figmy na spójny, responsywny i funkcjonalny interfejs użytkownika. Kluczowym założeniem na tym etapie było zachowanie prostoty i intuicyjności zaprojektowanej w fazie Lo-Fi, przy jednoczesnym zapewnieniu profesjonalnego wyglądu zgodnego z identyfikacją wizualną [PJATK](#) oraz współczesnymi standardami projektowania interfejsów. Wdrożony frontend, oparty na technologii [SvelteKit](#) oraz frameworku [TailwindCSS](#), wprowadza ustandaryzowany układ aplikacji, który pozostaje niezmienny pomiędzy różny-

mi widokami. Takie podejście minimalizuje czas uczenia się systemu, użytkownik raz zapoznany ze strukturą nawigacyjną potrafi, w teorii, swobodnie poruszać się po całej aplikacji bez konieczności ponownego orientowania się w interfejsie. Wybór [SvelteKit](#) jako frameworka frontendowego podyktowany był przede wszystkim jego reaktywnością i wydajnością. Framework ten umożliwia szybkie renderowanie komponentów oraz efektywne zarządzanie stanem aplikacji, co przekłada się na płynność interakcji z systemem.

7.4.1 Struktura nawigacyjna i układ widoku

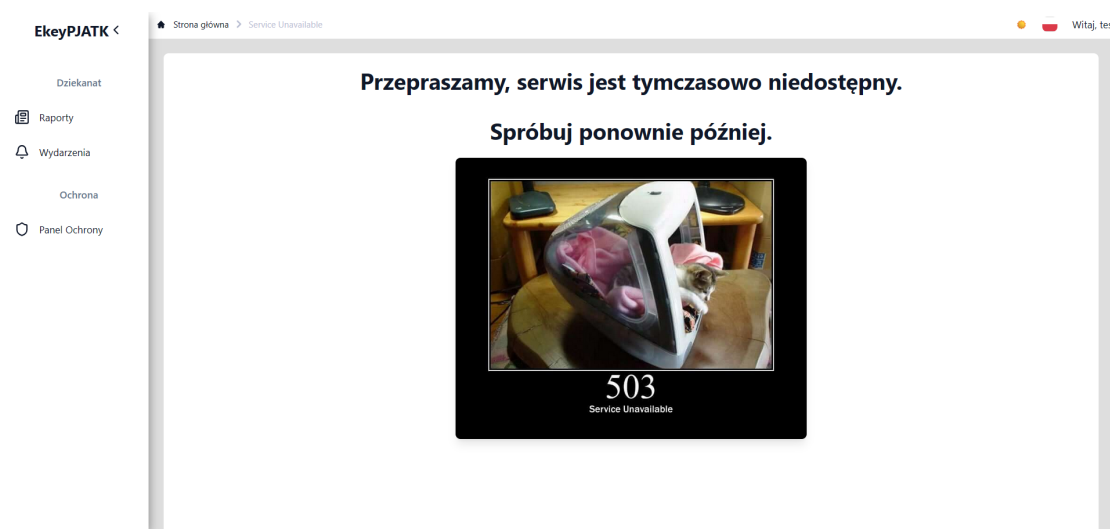
Zrzut ekranu (Rysunek 7.14) prezentuje ostateczny szkielet aplikacji, który składa się z trzech stałych stref, wyraźnie oddzielonych wizualnie i funkcjonalnie:

1. **Pasek boczny (Sidebar):** Umieszczony po lewej stronie ekranu, zawiera proste i czytelne menu nawigacyjne. Grupuje ono funkcjonalności w logiczne sekcje odpowiadające rolom i zadaniom użytkowników: „Dziekanat” agreguje funkcje związane z harmonogramem i synchronizacją danych, „Raporty” zapewnia dostęp do modułu raportowania, natomiast „Ochrona” umożliwia przeglądanie zdarzeń systemowych i zarządzanie incydentami. Taka struktura pozwala pracownikom uczelni na błyskawiczne przełączanie pomiędzy stronami bez konieczności przechodzenia przez wielopoziomowe menu czy formularze pośrednie.
2. **Pasek górny (Header):** Zawiera elementy kontekstowe informujące użytkownika, w jakim miejscu systemu się znajduje. Ścieżka nawigacyjna (breadcrumbs) wyświetla hierarchię lokalizacji, np. *Strona główna > Dziekanat > Raporty*, co ułatwia orientację w strukturze aplikacji i umożliwia szybki powrót do wcześniejszych widoków. W prawej części paska umieszczono elementy personalizacji interfejsu: przełącznik motywu kolorystycznego (jasny/ciemny), wybór języka (polski/angielski) oraz przycisk logowania z informacją o aktualnie zalogowanym użytkowniku. Takie rozwiązanie pozwala na dostosowanie wyglądu aplikacji do indywidualnych preferencji bez ingerencji w jej funkcjonalność.
3. **Obszar roboczy (Main Content):** Centralna część ekranu, w której dynamicznie ładowane są właściwe formularze, tabele i komponenty danych. Obszar ten zajmuje większość dostępnej przestrzeni i dostosowuje się do rozmiaru okna przeglądarki, zachowując responsywność na różnych urządzeniach. To w tej strefie użytkownik wykonuje główne zadania systemu – przegląda logi, generuje raporty, zarządza rezerwacjami czy obsługuje zdarzenia.

Taki trójstrefowy układ aplikacji wynika bezpośrednio z projektowania Lo-Fi. Podczas konsultacji z pracownikami Dziekanatu wyraźnie zaznaczono potrzebę intuicyjności interfejsu. Pasek boczny i nagłówek pozostają widoczne niezależnie od aktualnego widoku, co eliminuje ryzyko zagubienia się w systemie.

7.4.2 Komunikacja z użytkownikiem i obsługa błędów

Istotnym aspektem User Experience jest sposób, w jaki system komunikuje problemy. Zamiast wyświetlania surowych kodów błędów, które mogłyby zdezorientować pracowników nietechnicznych, zaprojektowano przyjazne strony informacyjne. Takie podejście jest zgodne z zasadami projektowania interfejsów użytkownika, które zakładają, że komunikaty systemowe powinny być zrozumiałe dla wszystkich grup użytkowników, niezależnie od ich kompetencji technicznych. Widoczny na Rysunku 7.14 ekran błędu „503 Service Unavailable” jest przykładem tzw. *Friendly Error Page*. Jasny komunikat („Przepraszamy, serwis jest tymczasowo niedostępny”) oraz humorystyczny akcent graficzny mają na celu obniżenie poziomu frustracji użytkownika w sytuacji awaryjnej i poinformowanie go, że problem leży po stronie systemu, a nie wynika z jego błędu. Dodatkowo, komunikat zawiera informację o charakterze tymczasowym problemu, co redukuje niepokój użytkownika i daje mu wskazówkę, że warto spróbować ponownie za chwilę. Podobne podejście zastosowano do innych stanów wyjątkowych, takich jak brak dostępu do zasobu, problemy z synchronizacją danych czy błędy walidacji formularzy. W każdym przypadku priorytetem było przedstawienie informacji w języku zrozumiałym dla użytkownika końcowego, z jasnymi wskazówkami dotyczącymi dalszych działań.



Rysunek 7.14: Przykładowy ekran obsługi błędu 503.

7.4.3 Przejście od prototypu do implementacji

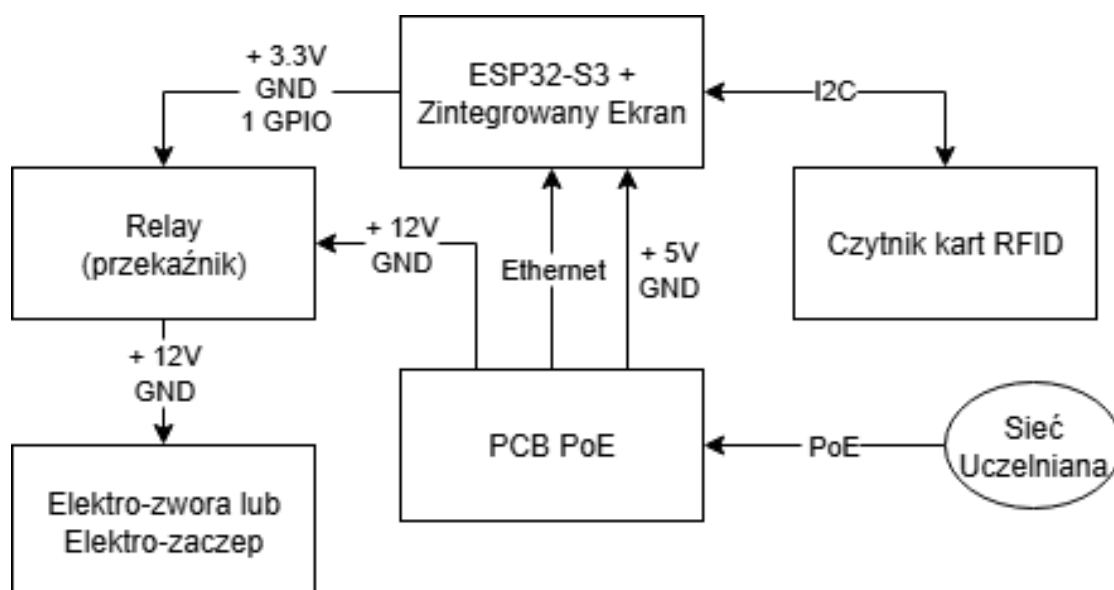
Przedstawiony interfejs Hi-Fi jest bezpośrednim rozwinięciem koncepcji zaprojektowanych w fazie Lo-Fi. Uwagi zebrane podczas konsultacji z interesariuszami, w szczególności z pracownikami Dziekanatu, zostały uwzględnione w ostatecznej wersji aplikacji. Proces przejścia od prostych szkiców do funkcjonalnego interfejsu wymagał również decyzji dotyczących kolorystyki i układu elementów graficznych. Ostateczny design graficzny nawiązuje do identyfikacji wizualnej [PJATK](#), dominuje kolor czerwony charakterystyczny dla uczelni, co zwiększa spójność systemu z innymi narzędziami używanymi w szkole. Jednocześnie zachowano uniwersalność i czytelność interfejsu, tak aby nie ograniczać użyteczności systemu w przypadku jego potencjalnej adaptacji w innych biznesach. Dalsza część pracy, w szczególności rozdział 9, szczegółowo opisuje techniczne aspekty implementacji frontendu, w tym strukturę komponentów, mechanizmy komunikacji z backendem oraz konkretne widoki aplikacji.

7.5 Projekt modułu sprzętowego

Proces projektowania zaczął się od dobrania odpowiednich komponentów. Zwracano uwagę na stabilne działanie panelu przy drzwiach oraz na integrację z istniejącymi rozwiązaniami. Ze względu na niedostępność wszystkich informacji od producentów danych elementów, powstawała potrzeba wielokrotnej wymiany na podobne komponenty spowodowane brakiem kompatybilności z resztą zakupionych części. Wdrożona płyta wykorzystuje mikrokontroler ESP32-S3, którego relatywnie duża pamięć i szybkość wykonywania obliczeń pozwalają na płynne działanie systemu przy większych obciążeniach.

7.5.1 Architektura elektroniki

Wzięliśmy również za zadanie ulepszenie wcześniejszego modułu sprzętowego Cyfrowych Kluczy, głównymi ulepszeniami za jakie postanowiliśmy się się zabrać było wymiana ekranu e-ink na ekran LCD umożliwiający wyświetlanie informacji w czytelniejszy i przyjemniejszy wizualnie sposób dla użytkownika, drugim i najważniejszym ulepszeniem było dodanie zasilania przez PoE, umożliwiającego łatwiejszy montaż i ograniczający liczę przewodów jakie potrzebuje urządzenie. System pobiera zasilanie i sieć przez PoE, ESP32-S3 miał za zadanie komunikację z serwerem oraz czytnikiem RFID, a po autoryzacji użytkownika zmianę stanu przekaźnikiem, który zasila elektrozaczep i umożliwia otwarcie drzwi.



Rysunek 7.15: Diagram połączeń modułu sprzętowego.

7.5.2 Projekt interfejsu użytkownika (UI)

Interfejs z zamysłu opierał się na potrzebach pracowników i studentów uczelni. Minimalizacja interakcji oraz czytelność były najważniejszymi cechami, na które zwracaliśmy uwagę przy jego projektowaniu. Wygląd musiał zostać zaadaptowany do wymagań instytucji, żeby zachować spójność stylów.

Najbliższe zajęcia	LOGO	
	8:30 - 10:00	ZAJ1 - Zajęcia 1 - Prof. Adam Kowalski
	10:15 - 11:45	ZAJ2 - Zajęcia 2 - Prof. Adam Kowalski
Nadchodzące zajęcia	12:00 - 13:30	ZAJ3 - Zajęcia 3 - Prof. Adam Kowalski
	14:00 - 14:30	ZAJ4 - Zajęcia 4 - Prof. Adam Kowalski

Rysunek 7.16: Przykładowy widok przedmiotów na wyświetlaczu

Powyższa makietka Lo-Fi pozwoliła na wprowadzenie dalszych zmian po dyskusjach między członkami zespołu oraz innymi interesariuszami. Nie powstała żadna

makieta Hi-Fi ze względu na potrzebę aktywnego testowania UI na [module sprzętowym](#) aby znaleźć najlepsze rozwiązania wspierające User Experience dla formy interfejsu w postaci ekranu dotykowego.

7.5.3 Backend

Połączenie WebSocket

Moduł wykorzystuje połączenie [WebSocket](#) do komunikacji z systemem [Cyfrowe Klucze](#), przez który wykonywana jest autoryzacja oraz dostarczane są informacje dotyczące planu zajęć.

```
1 {  
2     "hwId": "A203",  
3     "hwToken": "dummy"  
4 }  
5
```

Listing 7.4: Przykładowy kontrakt autoryzujący moduł do komunikacji

Pozostałe elementy Backend

Reszta elementów odpowiada za obsługę czytnika kart oraz

7.5.4 Projekt obudowy

Pierwszymi i najważniejszymi wartościami, od których zaczęło się modelowanie obudowy, były kompaktowość, łatwość montażu oraz estetyka urządzenia. Sam projekt przechodził wiele poprawek i iteracji w związku z dużą rotacją używanych komponentów. Prowadzone były również konsultacje z członkami zespołu, jak i innymi stronami zainteresowanymi, między innymi na temat najodpowiedniejszego sposobu montażu oraz kształtu samego urządzenia.

Rozdział 8

Opis realizacji

8.1 Semestr 6.

8.1.1 Marzec - Czerwiec 2025

Rozpoczęły się prace nad projektem, od najwcześniejszych etapów zakładana była współpraca z zespołem odpowiedzialnym za system [Cyfrowe Klucze](#). Niedługo potem nawiązano kontakt z zespołem odpowiedzialnym za aplikację [Cyfrowe Klucze Mobile](#), co ustawiło kierunek rozwoju projektu na tworzenie modułu udostępniającego rozwiązanie systemu [Cyfrowe Klucze](#) za pomocą kodów [QR](#) w aplikacji [CKM](#).

Podjęte zostały decyzje dotyczące sposobu integracji z aplikacją [CKM](#), sposób zabezpieczenia danych przesyłanych za pomocą kodów [QR](#) oraz metody weryfikacji tożsamości użytkownika. Zaimplementowano funkcjonalności generowania kodu [QR](#) oraz metodę weryfikacji danych za jego pomocą przesyłanych. Wprowadzono również mechanizmy szyfrowania i zabezpieczania danych przesyłanych w kodach [QR](#).

Rozpatrzone zostały komponenty potrzebne aby powstał nowy [Moduł Sprzętowy](#) i zaprojektowana wstępna architektura systemu [EkeyPJATK](#).

Przez utrudnienia w komunikacji między zespołami, moduł powstał ale nie został wdrożony. Postanowiono powrócić do tematu na dalszym etapie prac.

8.2 Wakacje

8.2.1 Lipiec - Sierpień 2025

Przez wakacje odbywały się spotkania zespołu z promotorem oraz konsultantem należącym do zespołu [Cyfrowe Klucze](#). W trakcie spotkań omawiano postępy prac,

napotkane problemy oraz dalsze kroki w realizacji projektu. Wynikiem spotkań było opracowanie diagramów UML systemu oraz przygotowanie ich odpowiedników w kodzie.

Również nawiązano współpracę z administracją uczelni oraz firmą obsługującą system [GAKKO](#), stworzone zostały kontrakty API umożliwiające pobieranie danych o planie zajęć.

Zamówiona i otrzymana została płytką [PCB](#) z wbudowanym mikrokontrolerem i ekranem [LCD](#) potrzebna do rozpoczęcia prac nad modulem zamka. Rozpoczęta została praca nad [UI](#) dla tego modułu. Znaczną część czasu zajęła konfiguracja ekranu.

8.2.2 Wrzesień 2025

Prace spowolniły ze względu na brak działań po stronie [GAKKO](#), które były niezbędne do dalszej realizacji projektu. Prace nad modulem zamka były kontynuowane oraz zostały zamówione inne komponenty potrzebne do realizacji fizycznej części projektu, m.in. [karty magnetyczne](#) i [Moduł Sprzętowy](#).

8.3 Semestr 7.

8.3.1 Październik 2025

Wznowiono intensywnie pracę nad projektem. Pod koniec poprzedniego miesiąca otrzymano od firmy obsługującej system [GAKKO](#) niezbędne dane do integracji. Rozpoczęto implementację pobierania i mapowania danych w systemie [Ekey-PJATK](#).

W połowie miesiąca doszło do przebudowania architektury głównego modułu systemu. Uprościło to dalsze zmiany w logice aplikacji oraz umożliwiło łatwiejsze dodawanie nowych funkcjonalności.

Pod koniec miesiąca system synchronizował się z systemem [GAKKO](#) i potrafił pobierać dane o planie zajęć. Został również wdrożony w formie „dev” na maszynie wirtualnej dostępnej w sieci uczelnianej.

Prace nad [UI](#) dla modułu zamka były kontynuowane i przedstawiona została pierwsza wersja. Zrobiono również elementy kodu służące do czytania kart magne-

tycznych przy pomocy czytników RFID. Zostały napotkane problemy z wiązką z brakiem kompatybilności niektórych elementów między sobą co wywołało potrzebę zakupu innych komponentów i minimalnego przeprojektowania architektury tego systemu.

8.3.2 Listopad 2025

Kontynuowano pracę nad integracją z systemem [Cyfrowe Klucze](#) oraz aplikacją [Cyfrowe Klucze Mobile](#).

Został zoptymalizowany proces pobierania i mapowania danych z systemu [GAK-KO](#), co poprawiło wydajność i stabilność działania systemu [EkeyPJATK](#). Wprowadzono również nowe rozwiązania do wewnętrznej komunikacji między modułami systemu [EkeyPJATK](#).

Moduł QR został dopracowany i przystosowany do zmian w głównym module systemu [EkeyPJATK](#). Wdrożono mechanizmy na maszynie wirtualnej.

Zamówione i otrzymane zostały nowe komponenty dla modułu zamka. Wymiana wywołała potrzebę zmiany środowiska programistycznego aby kontynuować dalszą pracę. Kod został przerobiony według potrzeb związanych z migracją między środowiskami i na nową architekturę. Dokonano konfiguracji nowego ekranu i nowego mikrokontrolera.

Zaprojektowana została płytką [PCB](#) służąca jako moduł [PoE](#) oraz pierwsza wersja obudowy modułu.

8.3.3 Grudzień 2025

Wdrożono symulowane połączenie z websocketem systemu [Cyfrowe Klucze](#). Określony został zakres zmian jakie muszą być wprowadzone po stronie [Cyfrowe Klucze](#) by moduły [EkeyPJATK](#): główny oraz raportowy mogły działać w środowisku produkcyjnym.

Wprowadzono przekazywanie planu zajęć na dany dzień dla wybranego czytnika przy konkretnej sali.

Powstał punkt dostępu dla czytników, by te mogły odpytać główny moduł [EkeyPJATK](#) o zgodność prośby o dostęp z planem zajęć.

Rozpoczęto dokładniejsze prace nad modulem raportowym [EkeyPJATK](#). Bazując na symulowanym systemie [Cyfrowe Klucze](#), który ma dostarczać logi z dostępu do sal, na podstawie których mają być generowane raporty.

8.3.4 Styczeń 2026

Ostatni miesiąc prac nad projektem został poświęcony na domknięcie wszystkich elementów systemu oraz uporządkowanie funkcjonalności, które powstały w poprzednich etapach. Priorytetem było osiągnięcie stabilnej wersji [EkeyPJATK](#), możliwej do zaprezentowania jako spójny i działający prototyp, a także przygotowanej do dalszych integracji z systemem [Cyfrowe Klucze](#), na które zespół nie miał bezpośredniego wpływu.

W tym okresie przeprowadzono przegląd wszystkich modułów, wprowadzając poprawki wynikające z testów oraz wcześniejszych zmian architektonicznych. Ujednolicono sposób komunikacji między modułami, dopracowano obsługę błędów oraz uporządkowano logikę odpowiedzialną za synchronizację danych z [GAKKO](#). Finalne prace obejmowały również dopracowanie modułu raportowego oraz przygotowanie środowiska umożliwiającego jego dalszą integrację z logami dostarczonymi przez system [Cyfrowe Klucze](#).

W styczniu podjęto także decyzję o rezygnacji z części pierwotnych założeń, których realizacja okazała się niemożliwa bez zmian po stronie zewnętrznych systemów. Zamiast tego skupiono się na dostarczeniu stabilnej i kompletnej wersji funkcjonalności, które mogły zostać w pełni zrealizowane w ramach możliwości zespołu [Digikey](#). Pozwoliło to na zakończenie prac w sposób uporządkowany i zgodny z zakresem, który był możliwy do osiągnięcia w warunkach projektu.

Ostatecznie system został przygotowany w formie gotowej do dalszych integracji oraz potencjalnego wdrożenia testowego, spełniając założenia etapu „prac wykończeniowych” przewidzianych w procesie planowania.

Rozdział 9

Szczegóły implementacji

9.1 Informacje techniczne i architektura systemu

System składa się z mikroserwisowego backendu, frontendu oraz modułu zamka.

9.1.1 Główny moduł backendowy (digikey-DB)

- **Technologia:**
 - **Język:** Go (wersja: 1.24.3)
 - **Framework webowy:** Gin (wersja: 1.10.1)
 - **ORM:** Gorm (wersja: 1.30.0)
 - **Architektura:** [Heksagonalna \(Porty i adaptery\)](#)
 - **Komunikacja:** HTTP + [RabbitMQ](#)
- **Dane:**
 - **Baza danych:** MySQL (wersja: 9.3)
- **Rola:**
 - Źródło prawdy dla danych systemu (budynki, sale, przedmioty, prowadzący, grupy, plan zajęć).
 - Działa jako pośrednik umożliwiający komunikację głównie między [Cyfrowe Klucze](#) a [GAKKO](#).
 - Udostępnia [REST API](#) oraz [RabbitMQ RPC](#) dla innych modułów backendowych i frontendu.

9.1.2 Moduł raportów (digikey-reports)

- **Technologia:**
 - **Język:** Go (wersja: 1.24.3)
 - **Framework webowy:** Gin (wersja: 1.10.1)
 - **ORM:** Gorm (z pakietem `gorm.io/datatypes`)
 - **Biblioteki do generowania raportów:**
 - * **gofpdf** - generowanie plików PDF
 - * **ORM:** Gorm (z pakietem `gorm.io/datatypes`)
 - * **encoding/csv** - eksport do CSV
 - **Architektura:** [Model-View-Controller \(MVC\)](#)
 - **Komunikacja:** [RabbitMQ](#) REST HTTP
- **Dane:**
 - **Baza danych:** SQLite (wersja: -TBA-)
 - **Blob storage:** -TBA- (Local S3, MinIO lub inny)
- **Rola:**
 - Generowanie raportów dotyczących przedmiotów, prowadzących i grup na podstawie danych z planu zajęć.
 - Przechowywanie wygenerowanych plików raportów.
 - Przechowywanie metadanych raportów w lokalnej bazie SQLite, by umożliwić łatwą regenerację i dostęp do raportów.

9.1.3 Moduł QR (digikey-QR)

- **Technologia:**
 - **Język:** Go (wersja: 1.24.4)
 - **Dodatkowe biblioteki:**
 - * **go-qrcode** (wersja: 0.0.0-20200617195104-da1b6568686e) - generowanie kodów QR
 - **Architektura:** [Model-View-Controller \(MVC\)](#)
 - **Komunikacja:** [RabbitMQ](#)
- **Dane:**

- **Cache:** [Redis](#) (wersja: 8.0.2)

- **Rola:**

- Generowanie kodów QR do wyświetlania na modułach zamka.
- Tworzenie i zarządzanie tymczasowymi tokenami dla użytkowników.

9.1.4 Frontend

- **Technologia:**

- **Framework:** SvelteKit (wersja: 5.0.0, TypeScript)
- **Bundler:** Vite (wersja: 7.0.4)
- **CSS:** TailwindCSS (wersja: 4.0.0)

- **Rola:**

- Interfejs użytkownika dla Dziekanatu i Administracji do korzystania z funkcji systemu.

9.1.5 Moduł Sprzętowy (door-panel)

- **Technologia:**

- **Architektura:** ESP32
- **SoC:** ESP32-P4
- **Język:** C/C++
- **Framework:** ESP-IDF
- **Komponenty:**
 - * **Waveshare ESP32-P4-NANO** - Płytką z mikrokontrolerem
 - * **Raspberry PI Touch Display 2, 7"** - Ekran dotykowy
 - * **Moduł PN532** - Czytnik kart RFID
 - * **Przełącznik 3v3** - Przełącznik sterowany sygnałem 3,3V
 - * **Regulator Step-Down 12V** - Regulator redukujący napięcie do 12V
 - * **Regulator Step-Down 5V** - Regulator redukujący napięcie do 5V
- **Dodatkowe biblioteki:**
 - * **LVGL** (wersja: 9.2.0) - Obsługa grafiki dla ESP32

- * **FreeRTOS** (wersja: -TBA-) - System operacyjny czasu rzeczywistego, zarządzanie zadaniami
- * **ESP32-P4-DSI-Support-Hub** (wersja: 9d5b14b) - Obsługa złącza MIPI-DSI dla ESP32-P4
- * **cJSON** (wersja: -TBA-) - Obsługa formatu JSON w językach C/C++
- * **pn532-esp-idf** (wersja: 8a9cb43) - Obsługa czytników RFID PN532 z kartami Mifare Classic

- **Rola:**

- Czytanie kart RFID w celu otwierania i zamykania drzwi, czyli kontrola dostępu do sali.
- Wyświetlanie bieżących zajęć oraz całego planu na dany dzień.

9.2 Działanie elementów systemu

9.2.1 Moduł Sprzętowy(door-panel)

9.2.2 Moduł DB (digikey-DB)

Moduł DB stanowi centralny komponent backendowy systemu [EkeyPJATK](#), odpowiedzialny za przetwarzanie danych z harmonogramem zajęć oraz weryfikacją uprawnień dostępu do sal. W przeciwieństwie do pozostałych modułów systemu, które zachowały prostszą strukturę MVC, moduł ten został zaprojektowany w oparciu o architekturę [Heksagonalna](#), co pozwoliło na pełną izolację logiki od warstw komunikacji zewnętrznej i oferuje możliwość łatwej rozbudowy w przyszłości.

Architektura heksagonalna i organizacja kodu

Zastosowanie architektury heksagonalnej wynika z centralnej roli modułu, jest on odpowiedzialny za komunikację z wieloma zewnętrznymi systemami ([GAKKO](#), [Cyfrowe Klucze](#), [Cyfrowe Klucze Mobile](#)) przy użyciu HTTP, [WebSocket](#), [RabbitMQ](#). Struktura katalogów projektu odzwierciedla podział na warstwy charakterystyczny dla tego wzorca:

- **domain/entities** - encje domenowe reprezentujące pojęcia biznesowe (plan zajęć, snapshoty, prowadzący),
- **application/port/in** - porty wejściowe definiujące przypadki użycia systemu,

- `application/port/out` - porty wyjściowe definiujące jakie metody mają być zaimplementowane w adapterze,
- `adapters/db` - adaptory bazodanowe (GORM dla MySQL, Redis dla danych tymczasowych),
- `adapters/external` - adaptory komunikacji z systemami zewnętrznymi ([GAK-KO](#), [Cyfrowe Klucze](#)).

Taki podział umożliwia testowanie logiki w izolacji od infrastruktury zewnętrznej oraz łatwą wymianę adapterów bez modyfikacji jądra aplikacji.

Mechanizm snapshotów i wersjonowania planu zajęć

Kluczowym elementem architektury danych jest mechanizm migawek, który zamiast nadpisywać plan zajęć w bazie, tworzy jego [snapshot](#). Podejście to zapewnia pełną historię zmian planu oraz umożliwia odtworzenie stanu systemu w dowolnym momencie przeszłości.

Encja `ScheduleSnapshot` reprezentuje nagłówek snapshotu, zawierający metadane oraz hash danych:

```

1  type ScheduleSnapshot struct {
2      ID                uint64
3      PulledAt          time.Time
4      SourceHash        [32]byte
5      BaseSnapshotID    *uint64
6      Meta              map[string]any
7  }
8
9  type ClassDetailSnapshot struct {
10     SnapshotID          uint64
11     StartTime           time.Time
12     EndTime             time.Time
13     ClassDate           time.Time
14     BgColor             string
15     ReservationType     string
16
17     ClassroomCode       string
18     ClassroomID         uint
19     ClassCode           *string
20     ClassID             *uint
21
22     GroupIDs            []uint
23     GroupCodes          []string
24
25     LeadIDs             []uint
26     LeadNames           []string
27     Extra               map[string]any
28 }

```

Listing 9.1: Encje domenowe snapshotu (`schedule_snapshot.go`)

Proces synchronizacji danych z systemem GAKKO realizowany jest przez funkcję `SyncSnapshot`, która implementuje algorytm detekcji zmian oparty na hashowaniu. Zamiast porównywać wiele rekordów, system oblicza hash SHA-256 z znormalizowanych danych i porównuje go z haszem poprzedniego snapshotu.

```
1 func (s *TimetableCommandsSvc) SyncSnapshot(ctx context.Context
2 ) (string, apperrors.ApiError) {
3     // Pobranie danych z zewnętrznego źródła (GAKKO)
4     rows, err := s.fetcher.TimeTable(ctx)
5     if err != nil {
6         return "", apperrors.ExternalSourceUnavailableError()
7     }
8     {...}
9     // Mapowanie i normalizacja danych
10    upserts, mapErr := gakkomap.
11    MapTimetableToClassDetailsUpserts(
12        rows, buildingMap)
13
14    hash, hErr := snaps.ComputeFeedHash(upserts)
15    {...}
16
17    prev, _ := s.snap.LatestHeader(ctx)
18    {...}
19
20    if prev == nil || prev.SourceHash != hash {
21        // Dane się zmieniły - zapisz pełny snapshot
22        if _, err := s.snap.SaveBase(ctx, hdr, rowsOut); err !=
23        nil {
24            return "", err
25        }
26    } else {
27        // Dane identyczne - zapisz tylko alias (wskaznik)
28        baseID := prev.ID
29        if prev.BaseSnapshotID != nil {
30            baseID = *prev.BaseSnapshotID
31        }
32        if _, err := s.snap.SaveAlias(ctx, hdr, baseID); err !=
33        nil {
34            return "", err
35        }
36    }
37    return "Success", nil
38 }
```

Listing 9.2: Algorytm synchronizacji snapshotów (`sync_service.go`)

Mechanizm aliasów stanowi istotną optymalizację, jeżeli plan zajęć nie uległ zmianie od ostatniej synchronizacji, system tworzy jedynie mały rekord wskazujący na istniejący [snapshot](#), zamiast duplikować tysiące wierszy w bazie danych. W praktyce, przy synchronizacji co 30 minut, większość snapshotów to aliasy, co znacząco redukuje zużycie pamięci.

Logika weryfikacji dostępu

Fundamentem systemu jest autorski algorytm odpowiedzialny za przyznawanie lub odmowę dostępu do sali. Implementacja uwzględnia nie tylko statyczne dane z planu zajęć, ale również kontekst okna czasowego wokół planowego czasu zajęć, mechanizm przejmowania własności sali oraz stan przechowywany w pamięci podręcznej Redis.

Funkcja `DecideAccess` realizuje wieloetapowy proces weryfikacji:

```
1 func (s *AccessVerifySvc) DecideAccess(ctx context.Context,
2   req in.AccessDecisionRequest) (in.AccessDecisionResult,
3   apperrors.ApiError) {
4   // Walidacja danych wejściowych
5   if req.Timestamp.IsZero() || strings.TrimSpace(req.
6     ReaderHwID) == "" {
7     return in.AccessDenied, apperrors.NewBadRequest("
8     incomplete_request")
9   }
10
11   reader, err := s.readers.FindByHwID(ctx, req.ReaderHwID)
12   classroomID := reader.ClassroomID
13
14   lead, err := s.lead.FindByLeadLogin(ctx, req.UserLogin)
15   if lead == nil {
16     return in.AccessNotHandled, nil
17   }
18
19   // Sprawdzenie własności sali w Redis
20   // jeśli redis wciąż trzyma własność sali dla tego prowadzą
21   // cego, zezwól na dostęp
22   if s.own != nil {
23     if owner, oerr := s.own.GetOwner(ctx, classroomID);
24     oerr == nil {
25       if owner.LeadID == lead.ID {
26         return in.AccessAllowed, nil // Szybka ścieżka
27       }
28     }
29   }
30
31   // Pobranie aktualnego snapshotu planu
32   hdr, err := s.snap.HeaderAtOrBefore(ctx, req.Timestamp)
33   rows, err := s.snap.ListRowsByClassroomFromTime(ctx, baseID
34   ,
35   classroomID, dayStart)
36
37   // Sprawdzenie zgodności z planem w oknie czasowym
38   hasMatch := false
39   for _, row := range rows {
40     if !containsUint(row.LeadIDs, lead.ID) {
41       continue
42     }
43     winFrom := row.StartTime.Add(-accessWindowMinutes *
44     time.Minute)
45     winTo := row.EndTime
```

```

41         if req.Timestamp.After(winFrom) && req.Timestamp.Before
42         (winTo) {
43             hasMatch = true
44             break
45         }
46     }
47     if !hasMatch {
48         return in.AccessDenied, apperrors.AccessDeniedError("
49         no_match")
50     }
51     // Ustawienie własności sali dla prowadzącego
52     s.setOwnershipForDay(ctx, classroomID, lead.ID, rows)
53
54     return in.AccessAllowed, nil
55 }

```

Listing 9.3: Algorytm weryfikacji dostępu (service.go)

Stała `accessWindowMinutes` (domyślnie 15 minut) definiuje bufor czasowy, w którym prowadzący może uzyskać dostęp przed planowanym rozpoczęciem zajęć. Wartość ta została ustalona we współpracy z pracownikami uczelni, aby odzwierciedlać realia prowadzenia zajęć.

Mechanizm przejmowania własności sali

Istotnym usprawnieniem logiki dostępu jest mechanizm `ownership` (przejmowania własności), który pozwala prowadzącemu pozostać w sali po zakończeniu zajęć bez konieczności ponownej autoryzacji. Stan własności przechowywany jest w Redis z automatycznym czasem wygaśnięcia (TTL).

```

1  type ClassroomOwnership struct {
2      ClassroomID uint
3      LeadID      uint
4      ValidUntil  time.Time
5  }
6
7  type OwnershipStore interface {
8      GetOwner(ctx context.Context, classroomID uint) (*
9      ClassroomOwnership, error)
10     SetOwner(ctx context.Context, ownership *ClassroomOwnership
11     ) error
12     ClearOwner(ctx context.Context, classroomID uint) error
13 }

```

Listing 9.4: Interfejs przechowywania własności sali (ownership_store.go)

Funkcja `setOwnershipForDay` oblicza czas ważności własności na podstawie harmonogramu dnia:

```

1 func (s *AccessVerifySvc) setOwnershipForDay(ctx context.
    Context,
2     classroomID uint, leadID uint, rows []d.ClassDetailSnapshot
    ) error {
3
4     // Znajdz ostatnie zajecia tego prowadzacego w tej sali
5     var lastEnd time.Time
6     for _, row := range rows {
7         if containsUint(row.LeadIDs, leadID) && row.EndTime.
After(lastEnd) {
8             lastEnd = row.EndTime
9         }
10    }
11
12    // Znajdz najwczesniejsze zajecia innego prowadzacego po
lastEnd
13    var nextOtherStart *time.Time
14    for _, row := range rows {
15        if row.StartTime.Before(lastEnd) || containsUint(row.
LeadIDs, leadID) {
16            continue
17        }
18        if nextOtherStart == nil || row.StartTime.Before(*
nextOtherStart) {
19            t := row.StartTime
20            nextOtherStart = &t
21        }
22    }
23
24    // Bazowa wlasnosc: 1 godzina po zakonczeniu zajec
validUntil := lastEnd.Add(time.Hour)
25
26
27    // Jesli inny prowadzacy ma zajecia wczesniej, skroc
wlasnosc
28    if nextOtherStart != nil && nextOtherStart.Before(
validUntil) {
29        validUntil = *nextOtherStart
30    }
31
32    return s.own.SetOwner(ctx, &out.ClassroomOwnership{
33        ClassroomID: classroomID,
34        LeadID:      leadID,
35        ValidUntil:  validUntil,
36    })
37 }

```

Listing 9.5: Logika ustalania własności sali (service.go)

Autorski algorytm gwarantuje, że własność sali zostanie automatycznie przekazana następnemu prowadzącemu w momencie rozpoczęcia jego zajęć, nawet jeśli poprzedni prowadzący nie opuścił sali fizycznie.

Integracja z czytnikami przez WebSocket

Moduł DB odpowiada za dostarczanie aktualnego planu zajęć do fizycznych czytników zamontowanych przy salach. System aktywnie wysyła dane do czytników po wykryciu zmian, zamiast czekać na zapytania.

```
1 func (s *CkSchedulePushService) pushSnapshotForReader(ctx
    context.Context,
2     hwID string, date time.Time) apperrors.ApiError {
3
4     //Pobranie planu dla konkretnego czytnika
5     rows, err := s.ttQueries.SnapshotRowsForClassroomOnDate(ctx
6     , hwID, date)
7     if len(rows) == 0 {
8         return nil
9     }
10
11     // Budowanie payloadu JSON
12     payload, _ := ck.BuildTTBody(hwID, rows)
13
14     // Sprawdzenie czy dane sie zmieniły (optymalizacja)
15     hash := scheduleHash(payload)
16     prevHash, _ := s.cache.GetScheduleHash(ctx, hwID)
17
18     if prevHash == hash {
19         log.Printf("Schedule unchanged for reader %s", hwID)
20         return nil // Nie wysylaj jesli nic sie nie zmienilo
21     }
22
23     // Wyslanie przez WebSocket
24     if err := s.ckWS.SendTimetable(payload); err != nil {
25         return apperrors.NewInternal("failed to send schedule")
26     }
27
28     // Zapisanie nowego hasha w cache
29     s.cache.SetScheduleHash(ctx, hwID, hash)
30     return nil
31 }
32
33 func scheduleHash(payload []byte) string {
34     sum := sha256.Sum256(payload)
35     return hex.EncodeToString(sum[:])
36 }
```

Listing 9.6: Serwis pushowania planu do czytników (ck_schedule_push_service.go)

Podobnie jak w przypadku snapshotów, zastosowano mechanizm hashowania do wykrywania zmian, jeżeli plan dla danego czytnika nie uległ modyfikacji, transmisja jest pomijana. Klient WebSocket obsługuje połączenie z infrastrukturą [Cyfrowe Klucze](#), włącznie z uwierzytelnianiem [TLS](#):

```
1 func NewWSClient() (*WSClient, error) {
2     base := FromEnv()
3     wsURL, _ := wsFromHTTP(base)
```

```

4
5     var tlsCfg *tls.Config
6     caPath := os.Getenv("CK_CA_CERT")
7     if caPath != "" {
8         rootCAs, _ := loadRootCAsFromFile(caPath)
9         tlsCfg = &tls.Config{
10             RootCAs:      rootCAs,
11             ServerName: "ck.pjwstk.edu.pl",
12         }
13     }
14
15     dialer := &websocket.Dialer{
16         HandshakeTimeout: 10 * time.Second,
17         TLSClientConfig:  tlsCfg,
18         EnableCompression: true,
19     }
20     return &WSCClient{dialer: dialer, wsURL: wsURL}, nil
21 }
22
23 func (c *WSCClient) SendTimetable(ttJSON []byte) error {
24     msg := "TT:" + strings.TrimSpace(string(ttJSON))
25     return c.SendRaw(msg)
26 }

```

Listing 9.7: Klient WebSocket do komunikacji z czytnikami (ws.go)

Serwer HTTP API dla aplikacji mobilnej

Komunikacja z aplikacją mobilną [Cyfrowe Klucze Mobile](#) odbywa się poprzez synchroniczne wywołania [API REST](#), udostępnionego przez serwer HTTP. Zastosowanie standardowego modelu żądanie-odpowiedź upraszcza integrację z klientami mobilnymi, a także ułatwia debugowanie i monitorowanie ruchu. Do implementacji serwera wykorzystano framework **Gin**, popularny w Go ze względu na swoją wydajność i elastyczność.

```

1
2 // GetByLeadName godoc
3 // @Summary Get schedule by lecturer name (next 7 days)
4 // @Description Returns entries for a given lecturer name for
5 // the next 7 days from given date (format: YYYY-MM-DD). If no
6 // date is provided, current date is used.
7 // @Tags Timetable
8 // @Param name path string true "ex. Doe John"
9 // @Param from query string false "Start date in format YYYY-MM
10 // -DD"
11 // @Produce json
12 // @Success 200 {array} responses.MobileScheduleDTO
13 // @Failure 400 {object} responses.ErrorResponse
14 // @Failure 404 {object} responses.ErrorResponse
15 // @Failure 500 {object} responses.ErrorResponse
16 // @Router /timetables/{name} [get]
17 func (tc *TimetableController) GetByLeadName(c *gin.Context) {
18     name := c.Param("name")

```

```

16     startDate := c.Query("from")
17
18     data, err := tc.qry.ByLecturerName(c.Request.Context(),
19     name, startDate)
20     if err != nil {
21         c.Error(err)
22         return
23     }
24     out, err := translate.MapToMobileScheduleDTOs(data)
25     if err != nil {
26         c.Error(err)
27         return
28     }
29     c.JSON(http.StatusOK, out)

```

Listing 9.8: Kontroler HTTP do obsługi zapytań o plan zajęć zwracający odpowiedź w formacie MobileScheduleDTO (timetable_controller.go)

Wybór architektury opartej na [REST API](#) podyktowany był dążeniem do prostoty integracji. W przeciwieństwie do systemów opartych na kolejkach, które doskonale sprawdzają się w komunikacji backend-backend, standardowe [API REST API](#) jest naturalnym wyborem dla interakcji z aplikacjami takimi jak [Cyfrowe Klucze Mobile](#). Umożliwia to bezproblemową komunikację z wykorzystaniem powszechnie dostępnych bibliotek i narzędzi deweloperskich.

Hybrydowe przechowywanie danych

System wykorzystuje dwie technologie bazodanowe, dobrane odpowiednio do charakterystyki przechowywanych danych:

- **MySQL** - trwałe dane domenowe: snapshoty planu zajęć, informacje o salach, prowadzących i grupach.
- **Redis** - dane tymczasowe i szybkozmiennie: aktualny właściciel sali, cache hashów harmonogramów czytników, dane sesyjne.

Repozytorium snapshotów implementuje interfejs portu wyjściowego, ukrywając szczegóły persystencji przed warstwą aplikacji:

```

1 func (s *SnapshotRepository) SaveBase(ctx context.Context,
2     header *d.ScheduleSnapshot, rows []d.ClassDetailSnapshot) (
3     uint64, error) {
4
5     tx := s.db.WithContext(ctx).Begin()
6
7     // Zapis nagłówka snapshotu
8     h := t.ToModelScheduleSnapshot(header)
9     h.BaseSnapshotID = nil
10    tx.Create(&h)

```

```

10
11 // Zapis wierszy w partiach
12 if len(rows) > 0 {
13     batch := make([]m.ClassDetailSnapshot, 0, len(rows))
14     for _, row := range rows {
15         batch = append(batch, t.ToModelClassDetailSnapshot(
16             h.ID, row))
17     }
18     const chunk = 1000
19     for i := 0; i < len(batch); i += chunk {
20         j := min(i+chunk, len(batch))
21         tx.CreateInBatches(batch[i:j], chunk)
22     }
23
24     tx.Commit()
25     return uint64(h.ID), nil
26 }
27
28 func (s *SnapshotRepository) HeaderAtOrBefore(ctx context.
29     Context,
30     at time.Time) (*d.ScheduleSnapshot, error) {
31
32     var header m.ScheduleSnapshot
33     err := s.db.WithContext(ctx).
34         Where("pulled_at <= ?", at).
35         Order("pulled_at DESC").
36         First(&header).Error
37
38     if err == gorm.ErrRecordNotFound {
39         return nil, nil
40     }
41     return t.ToDomainScheduleSnapshot(header), nil

```

Listing 9.9: Repozytorium snapshotów (snapshot_repo.go)

Zastosowanie [przetwarzanie wsadowe](#) przy zapisie snapshotów jest istotne ze względu na ilość danych, pojedynczy snapshot może zawierać kilka tysięcy wierszy reprezentujących wszystkie zajęcia w systemie. Podział na partie po 1000 rekordów zapobiega przekroczeniu limitów transakcji MySQL oraz pozwala na monitorowanie postępu operacji.

Harmonogram i automatyzacja

Moduł DB wykorzystuje wewnętrzny harmonogram - scheduler, który automatycznie wykonuje cykliczne zadania:

- **Synchronizacja z GAKKO** - co 30 minut system pobiera aktualny plan zajęć i tworzy nowy [snapshot](#).

- **Aktualizacja czytników** - po każdej synchronizacji system wypycha zaktualizowane plany do wszystkich aktywnych czytników przez WebSocket.

Okres 30 minut został dobrany jako kompromis między aktualnością danych a obciążeniem systemu GAKKO. W praktyce zmiany w planie zajęć, podczas trwania semestru, są stosunkowo rzadkie, więc większość synchronizacji nie wymaga zapisu nowych danych.

Interfejs REST API

Moduł DB udostępnia REST API podzielone na kilka sekcji funkcjonalnych:

- **Access / Dostęp:** Logowanie użytkowników oraz weryfikacja zgodności prośby o dostęp do sali z planem zajęć.
- **Timetable / Dane:** Operacje na danych dotyczących planu zajęć, głównie odczyt oraz ręczna synchronizacja z GAKKO.
- **Snapshot / Migawka:** Uzyskiwanie informacji o aktualności danych w systemie oraz przeglądanie historii snapshotów.

Najważniejsze endpointy API:

- `POST /api/access/classroom` — weryfikacja dostępu do sali na podstawie planu zajęć,
- `POST /api/access/login` — logowanie użytkownika do systemu,
- `GET /api/timetables` — ręczna synchronizacja danych z GAKKO,
- `GET /api/snapshots/latest` — pobranie informacji o najnowszym snapshotcie.

Pełna dokumentacja API dostępna jest w formacie Swagger pod adresem `http://localhost:808`

9.2.3 Moduł raportów (digikey-reports)

Moduł raportowania (**digikey-reports**) stanowi autonomiczny komponent systemu, odpowiedzialny za przetwarzanie dużych zbiorów danych dotyczących harmonogramu zajęć i logów wejść. Ze względu na specyfikę procesu, konieczność pobrania tysięcy rekordów, ich analizę oraz wygenerowanie dokumentów PDF, kluczowym wyzwaniem było zapewnienie wydajności i responsywności systemu. Aby sprostać tym wymaganiom, wdrożono mechanizm przetwarzania asynchronicznego oraz autorskie algorytmy weryfikacji obecności.

Asynchroniczny model przetwarzania zadań

W celu uniknięcia blokowania interfejsu użytkownika podczas generowania złożonych raportów, zastosowano architekturę kolejkową [producer consumer pattern](#). Interfejs API przyjmuje zlecenie raportu, zapisuje je w bazie danych ze statusem `queued` i natychmiast zwraca użytkownikowi identyfikator zadania. Właściwe przetwarzanie odbywa się w tle przez niezależne procesy typu Worker.

Kluczowym elementem tego mechanizmu jest funkcja `ClaimQueuedJob`, która zapewnia bezpieczne pobieranie zadań w środowisku współbieżnym. Wykorzystano w niej mechanizm transakcji bazodanowej oraz blokowania wierszy (`SELECT ... FOR UPDATE`), co gwarantuje, że ten sam raport nie zostanie pobrany przez dwa workery jednocześnie. Poniższy fragment kodu prezentuje implementację tej blokady naszym systemie:

```
1 func (r *Repo) ClaimQueuedJob(ctx context.Context) (*ReportJob,
2     error) {
3     // Rozpoczęcie transakcji
4     tx := r.ReportDB.WithContext(ctx).Begin()
5     var job ReportJob
6
7     // Pobranie pierwszego zadania ze statusem 'queued'
8     // Zastosowanie klauzuli blokującej UPDATE, aby inny worker
9     // nie mógł go pobrać
10    if err := tx.
11        Clauses(clause.Locking{Strength: "UPDATE"}).
12        Where("status = ?", "queued").
13        Order("created_at, id").
14        First(&job).Error; err != nil {
15        tx.Rollback()
16        return nil, err
17    }
18
19    // Natychmiastowa zmiana statusu na 'running'
20    if err := tx.Model(&job).Updates(map[string]any{
21        "status": "running",
22        "updated_at": time.Now(),
23    }).Error; err != nil {
24        tx.Rollback()
25        return nil, err
26    }
27
28    // Zatwierdzenie transakcji
29    if err := tx.Commit().Error; err != nil {
30        return nil, err
31    }
32    return &job, nil
33 }
```

Listing 9.10: Bezpieczne pobieranie zadania z kolejki z wykorzystaniem blokowania wierszy

Dzięki takiemu podejściu system jest odporny na zjawisko `race condition` i pozwala na łatwe skalowanie poprzez uruchomienie wielu instancji workerów, co znacząco przyspiesza generowanie raportów w okresach duże obciążenia np. pod koniec semestru.

Algorytm weryfikacji obecności (Streak Matching)

Sercem logiki biznesowej modułu raportującego jest autorski algorytm dopasowywania logów wejść do harmonogramu zajęć. W przeciwieństwie do prostszych systemów, które wymagają logu wejścia dla każdej aktywności, system `EkeyPJATK` uwzględnia specyfikę pracy akademickiej, wprowadzając pojęcie `ciąg obecności`.

Algorytm zakłada, że jeżeli wykładowca prowadzi kilka bloków zajęć z rzędu w tej samej sali, np. kilka grup mających ten sam przedmiot jedna po drugiej, fizyczne otwarcie drzwi wymagane jest jedynie przed rozpoczęciem pierwszego bloku. Każde kolejne zajęcia są automatycznie uznawane za zrealizowane, o ile zachowana jest ciągłość przebywania w sali.

Struktura danych `MatchedEntry` stanowi rozszerzenie standardowego wpisu w planie zajęć o status realizacji oraz powiązany log wejścia.

```
1  // ScheduleEntry to ujednolicony format zajęć do przetwarzania
2  type ScheduleEntry struct {
3      StartTime    time.Time
4      EndTime      time.Time
5      Classroom    string
6      Lecturer     string    // login wykładowcy
7      LecturerName string
8      Group        string
9      Course       string
10     ClassType    string
11 }
12
13 // MatchedEntry to zajęcia z przypisanym statusem realizacji
14 // Obiekt ten jest wynikiem działania algorytmu dopasowania
15 type MatchedEntry struct {
16     ScheduleEntry
17     // Status: "Realized" (Zrealizowane) lub "Missed" (Brak
18     // obecności)
19     Status      string
20     // ActualStart: Godzina wejścia lub "-"
21     ActualStart string
22     // MatchedLog: Wskaźnik na log, który autoryzował te zajęcia
23     MatchedLog  *ClassroomLog
24 }
```

Listing 9.11: Struktury danych obsługujące logikę ciągów (matching.go)

Istotnym elementem algorytmu jest funkcja pomocnicza `canContinueStreak`, która decyduje, czy przerwa między zajęciami nie narusza ciągłości uprawnień. Logika

ta sprawdza, czy poprzednie zajęcia zostały poprawnie zrealizowane przez tego samego wykładowcę oraz czy w międzyczasie nie zarejestrowano w sali innej osoby.

```
1 // canContinueStreak sprawdza, czy bieżące zajęcia są
   kontynuacją
2 // bloku zajęć tego samego wykładowcy w tej samej sali
3 func (s *MatcherService) canContinueStreak(
4     current, prev ScheduleEntry,
5     prevMatched *MatchedEntry,
6     logs []ClassroomLog,
7 ) bool {
8     // 1. Musi to być ten sam wykładowca
9     if current.Lecturer != prev.Lecturer {
10         return false
11     }
12
13     // 2. Poprzednie zajęcia musiały być zrealizowane (Status
14     == "Realized")
15     if prevMatched == nil || prevMatched.Status != "Realized" {
16         return false
17     }
18
19     // 3. Sprawdzenie czy ktoś inny nie wszedł w międzyczasie
20     // Pobieramy logi z okresu przerwy między zajęciami
21     interimLogs := s.getLogsBetween(prev.EndTime, current.
22     StartTime, logs)
23
24     for _, l := range interimLogs {
25         // Jeśli w sali pojawił się ktoś inny, ciąg zostaje
26         // przerwany
27         if l.UserLogin != current.Lecturer {
28             return false
29         }
30     }
31
32     return true
33 }
```

Listing 9.12: Weryfikacja ciągłości uprawnień (matching.go)

Uzupełnieniem mechanizmu dopasowania jest funkcja `findMatchingLog`, która implementuje logikę "okna czasowego". Zamiast szukać logu o idealnie zgodnej godzinie, system akceptuje wejścia zarejestrowane w konfigurowalnym buforze czasowym w okolicy planowego czasu rozpoczęcia zajęć ustalonym razem z pomocą dziekanu i pracowników uczelni tak żeby pasował do realiów prowadzenia zajęć.

```
1 func (s *MatcherService) findMatchingLog(
2     entry ScheduleEntry,
3     logs []ClassroomLog,
4 ) *ClassroomLog {
5     // Definicja okna: np. 20 minut przed startem do końca zajęć
6     windowStart := entry.StartTime.Add(-20 * time.Minute)
7     windowEnd := entry.EndTime
8 }
```



```

9      for _, l := range logs {
10         // Log musi pochodzić z tej samej sali
11         if l.Room != entry.Classroom {
12             continue
13         }
14
15         // Log musi mieścić się w oknie czasowym
16         logTime := l.Timestamp
17         if (logTime.After(windowStart) || logTime.Equal(
18 windowStart)) &&
19             (logTime.Before(windowEnd) || logTime.Equal(
20 windowEnd)) {
21
22             // Log musi należeć do wykładowcy prowadzącego zaję
23             cia
24             if l.UserLogin == entry.Lecturer {
25                 return &l
26             }
27         }
28     }
29     return nil
30 }

```

Listing 9.13: Wyszukiwanie logu w oknie czasowym

Proces weryfikacji realizowany przez funkcję `MatchScheduleWithLogs` przebiega w następujących krokach:

1. **Normalizacja:** System pobiera plan zajęć i logi wejść dla zadanego przedziału czasu.
2. **Analiza okna czasowego:** Dla każdego zajęcia algorytm poszukuje logu wejścia w zdefiniowanym oknie tolerancji (domyślnie od 20 minut przed rozpoczęciem zajęć do momentu ich zakończenia).
3. **Obsługa ciągów (Streak Logic):** Jeżeli bezpośredni log nie zostanie znaleziony, uruchamiana jest funkcja pomocnicza `canContinueStreak`. Sprawdza ona, czy:
 - Poprzednie zajęcia w tej samej sali należały do tego samego wykładowcy.
 - Poprzednie zajęcia zostały uznane za zrealizowane (posiadają ważny log).
 - W międzyczasie nie nastąpiło wejście innej osoby, co przerwałoby ciąg uprawnień.

Dzięki takiemu podejściu system eliminuje fałszywe alarmy w sytuacjach, gdy

wykładowca przebywa w sali przez wiele godzin, autoryzując się tylko raz na początku dnia.

Model integracji systemów i agregacja danych

Moduł raportowy zaprojektowany został z myślą o byciu częścią mikroservisowej architektury systemu [EkeyPJATK](#), jako jeden z monolitów współpracujących dla dobra całości, nie posiada bezpośredniego dostępu do baz danych innych systemów uczelnianych (co stanowiłoby ryzyko bezpieczeństwa), lecz komunikuje się z nimi za pośrednictwem dedykowanych interfejsów API.

Architektura przepływu danych opiera się na trzech filarach:

1. **Zewnętrzne źródła danych (API):** Aplikacja pobiera harmonogram zajęć z mikroservisu planistycznego (`ScheduleService`) oraz historię wejść z systemu kontroli dostępu (`LogService`). Komunikacja odbywa się protokołem HTTP/HTTPS, co zapewnia ścisłą kontrolę uprawnień na poziomie endpointów. Adresy usług są konfigurowalne dynamicznie poprzez zmienne środowiskowe.
2. **Lokalny stan operacyjny (SQLite):** Aplikacja wykorzystuje wbudowaną bazę SQLite (`ReportDB`) wyłącznie do przechowywania własnego stanu: kolejki zadań asynchronicznych oraz historii wygenerowanych raportów. Zapewnia to pełną autonomię modułu, awaria zewnętrznego API uniemożliwi wygenerowanie nowego raportu, ale nie zablokuje dostępu do już zarchiwizowanych dokumentów.
3. **Klienty HTTP i normalizacja:** Dedykowane serwisy klienckie (`log_client.go`, `schedule_client.go`) odpowiadają za wykonanie żądań, obsługę błędów sieciowych oraz mapowanie surowych danych JSON na wewnętrzne struktury aplikacji.

Poniższy fragment kodu prezentuje implementację klienta pobierającego logi wejść. Zapytanie budowane jest z filtrowaniem parametrów (GET query params), co pozwala na pobieranie tylko niezbędnego podzbioru danych.

```
1 func FetchClassroomLogs(teacher, lab, from string) ([]  
    ClassroomLog, error) {  
2     // Budowanie parametrów zapytania (Query Params)  
3     params := url.Values{}  
4     if teacher != "" {  
5         params.Add("teacher", teacher)  
6     }  
7     if lab != "" {  
8         params.Add("lab", lab)
```

```

9      }
10
11      // Pobranie adresu API z konfiguracji
12      fullURL := config.AppConfig.LogServiceURL
13      if len(params) > 0 {
14          fullURL += "?" + params.Encode()
15      }
16
17      // Wykonanie ządania HTTP GET
18      resp, err := http.Get(fullURL)
19      if err != nil {
20          return nil, err
21      }
22      defer resp.Body.Close()
23
24      // Obsługa kodów błędów HTTP
25      if resp.StatusCode != http.StatusOK {
26          return nil, fmt.Errorf("log service status: %d", resp.
27          StatusCode)
28      }
29
30      // Dekodowanie odpowiedzi JSON do struktury Go
31      var logs []ClassroomLog
32      if err := json.NewDecoder(resp.Body).Decode(&logs); err !=
33      nil {
34          return nil, err
35      }
36      return logs, nil
37  }

```

Listing 9.14: Klient API pobierający logi wejść (log_client.go)

Podjęcie to realizuje zasadę [separation of concerns](#). Moduł raportowy nie musi znać struktury bazy danych systemu [Cyfrowe Klucze](#) ani systemu dziekanatu, polega na kontrakcie interfejsu API, co ułatwia niezależny rozwój i utrzymanie poszczególnych komponentów systemu.

Silnik generowania dokumentów PDF

Za ostateczną prezentację danych odpowiada dedykowany serwis oparty na bibliotece `gopdf`. Ze względu na wymóg generowania oficjalnych dokumentów dla Dziekanatu, kluczowym wyzwaniem implementacyjnym było dynamiczne dostosowywanie treści do sztywnego układu tabeli (A4) przy zachowaniu wysokiej czytelności.

W module `pdf_renderer.go` zaimplementowano mechanizmy, które automatyzują proces składu tekstu:

1. **Dynamiczne przycinanie tekstu:** Zastosowano autorską metodę `truncateToWidth`, która oblicza szerokość ciągu znaków w wybranej czcionce i w razie potrzeby

skraca go, dodając elipsę ("..."). Zapobiega to rozjeżdżaniu się tabel przy wyjątkowo długich nazwach przedmiotów.

2. **Warunkowe formatowanie wizualne:** System automatycznie dobiera kolor czcionki w podsumowaniach w zależności od statusu realizacji (np. kolor czerwony dla braków, zielony dla sukcesów). Ułatwia to pracownikom administracyjnym szybką ocenę raportu bez konieczności wczytywania się w szczegóły.

```
1  const (
2      // Definicje kolorów (RGB) zgodne z identyfikacją wizualną
3      colorPrimaryR, colorPrimaryG, colorPrimaryB = 41, 65, 114
4      // Granatowy
5      colorSuccessR, colorSuccessG, colorSuccessB = 34, 139, 34
6      // Zielony
7      colorDangerR, colorDangerG, colorDangerB    = 178, 34, 34
8      // Czerwony
9  )
10
11 // Pętla renderująca wiersze z dynamicznym przycinaniem treści
12 for j, cell := range cells {
13     // truncateToWidth dopasowuje tekst do szerokości kolumny (
14     widths[j])
15     p.pdf.CellFormat(
16         widths[j],
17         7,
18         p.truncateToWidth(cell, widths[j]),
19         "1", 0, "L", i%2 == 1, 0, ""
20     )
21 }
```

Listing 9.15: Konfiguracja stylów i renderowanie komórek tabeli (pdf_renderer.go)

Wzorzec Rejestru i przetwarzanie wsadowe

System został zaprojektowany z myślą o przyszłym rozwoju. Aby umożliwić łatwe dodawanie nowych typów raportów bez modyfikacji logiki sterującej, zastosowano wzorzec [wzorzec Registry](#).

W pliku `router.go` zdefiniowano centralny rejestr raportów, który mapuje tekstowy identyfikator typu raportu (np. "lecturer_detail") na odpowiednią strukturę obsługującą logikę biznesową. Rejestr ten obsługuje również definicje dla przetwarzania wsadowego [przetwarzanie wsadowe](#), co pozwala na generowanie wielu raportów jednym zapytaniem API iterując np. po liście sal lub wykładowców.

```
1  // Inicjalizacja routera i rejestru raportów
2  func NewRouter(repo *repositories.Repo) *gin.Engine {
3      r := gin.New()
4
5      // Konfiguracja dostępnych typów raportów
```

```

6     reg := NewRegistry(
7         // Raport ogólny
8         ReportDef{Type: "overview_basic", Binder:
overviewBinder{}}},
9
10        // Raport szczegółowy wykładowcy (obsługa batch:
iteracja po "leads")
11        ReportDef{
12            Type: "lecturer_detail",
13            Binder: lecturerBinder{},
14            BatchField: "leads",
15            ItemField: "lecturer_lastname"
16        },
17
18        // Raport zajętości sal
19        ReportDef{
20            Type: "classroom_occupancy",
21            Binder: classroomBinder{},
22            BatchField: "classrooms",
23            ItemField: "classroom_code"
24        },
25    )
26    // ... obsługa endpointów
27 }

```

Listing 9.16: Rejestracja definicji raportów i konfiguracja batchingu (router.go)

Dzięki takiemu rozwiązaniu, dodanie nowego rodzaju zestawienia wymaga jedynie implementacji interfejsu i zarejestrowania go w jednej linijce kodu, co znacząco ułatwia dalszy rozwój systemu.

9.2.4 Frontend

Frontend systemu EkeyPJATK stanowi warstwę prezentacji dedykowaną pracownikom Dziekanatu. Zgodnie z założeniami przyjętymi na etapie projektowania, interfejs został zaprojektowany z naciskiem na minimalizację liczby kroków potrzebnych do wykonania zadania oraz czytelność prezentowanych informacji. System nie wymaga długotrwałego szkolenia ani nie wymusza zmiany przyzwyczajeń, funkcje są łatwo dostępne, a nawigacja intuicyjna.

Technologie i architektura frontendu

Warstwa prezentacji została zbudowana w oparciu o framework [SvelteKit](#), który zapewnia reaktywność interfejsu oraz wydajne zarządzanie stanem aplikacji. Do stylizacji wykorzystano TailwindCSS, co pozwoliło na szybkie prototypowanie i zachowanie spójności wizualnej między widokami. Architektura frontendu opiera się na komponentowym podejściu charakterystycznym dla Svelte, gdzie każdy widok składa się z izolowanych, wielokrotnego użytku komponentów. Komunikacja

cja z backendem odbywa się poprzez REST API modułu DB oraz dedykowane endpointy modułu raportów.

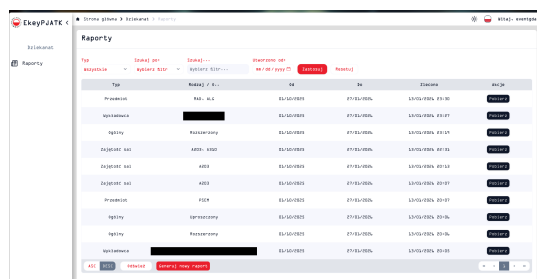
Struktura nawigacyjna i układ widoku

Interfejs aplikacji został podzielony na trzy stałe strefy, które pozostają niezmiennie pomiędzy poszczególnymi widokami:

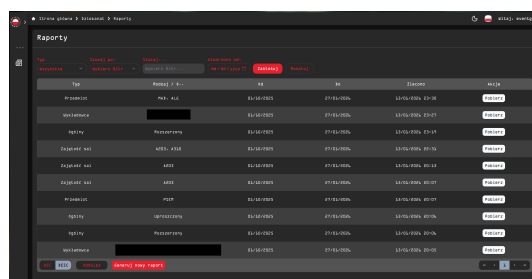
1. **Pasek boczny (Sidebar):** Umieszczony po lewej stronie ekranu, zawiera hierarchiczne menu nawigacyjne pogrupowane w logiczne sekcje funkcjonalne. Sekcja "Dziekanat" agreguje funkcje związane z harmonogramem i synchronizacją danych, "Raporty" zapewnia dostęp do modułu raportowania, natomiast "Wydarzenia" służą do przeglądania zdarzeń systemowych.
2. **Pasek górny (Header):** Zawiera ścieżkę nawigacyjną informującą użytkownika o aktualnej lokalizacji w strukturze aplikacji, np. *Strona główna > Dziekanat > Raporty*. W prawej części paska umieszczono elementy personalizacji: przełącznik motywu kolorystycznego, wybór języka interfejsu oraz przycisk logowania z informacją o zalogowanym użytkowniku.
3. **Obszar roboczy (Main Content):** Centralna część ekranu, w której dynamicznie renderowane są właściwe formularze, tabele i komponenty danych. Obszar ten zajmuje większość dostępnej przestrzeni i dostosowuje się do rozmiaru okna przeglądarki.

Personalizacja interfejsu

System oferuje możliwość dostosowania wyglądu do preferencji użytkownika poprzez wybór motywu kolorystycznego. Dostępne są dwa warianty: motyw jasny oraz motyw ciemny, między którymi można przełączać się za pomocą ikony w pasku górnym. Rysunek poniżej przedstawia porównanie obu wariantów na przykładzie widoku raportów.



(a) Motyw jasny



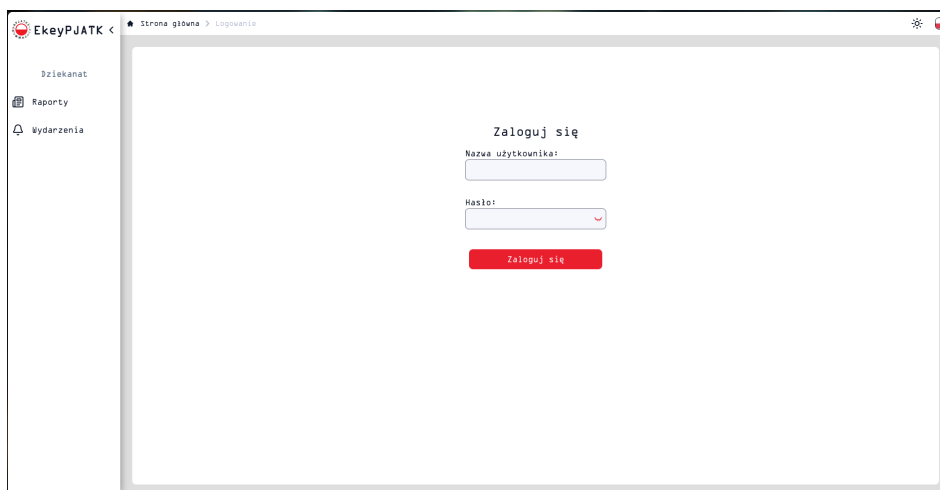
(b) Motyw ciemny

Rysunek 9.1: Porównanie motywów kolorystycznych interfejsu na przykładzie widoku raportów.

Niezależnie od wybranego motywu, kolorystyka akcentów (przyciski akcji, elementy interaktywne) pozostaje spójna i nawiązuje do identyfikacji wizualnej PJATK - dominuje kolor czerwony charakterystyczny dla uczelni. Interfejs wspiera również wielojęzyczność, umożliwiając przełączanie między językiem polskim a angielskim.

Ekran logowania

Punktem wejścia do systemu jest minimalistyczny ekran logowania. Formularz zawiera wyłącznie niezbędne pola: nazwę użytkownika oraz hasło, uzupełnione o przycisk akcji "Zaloguj się".

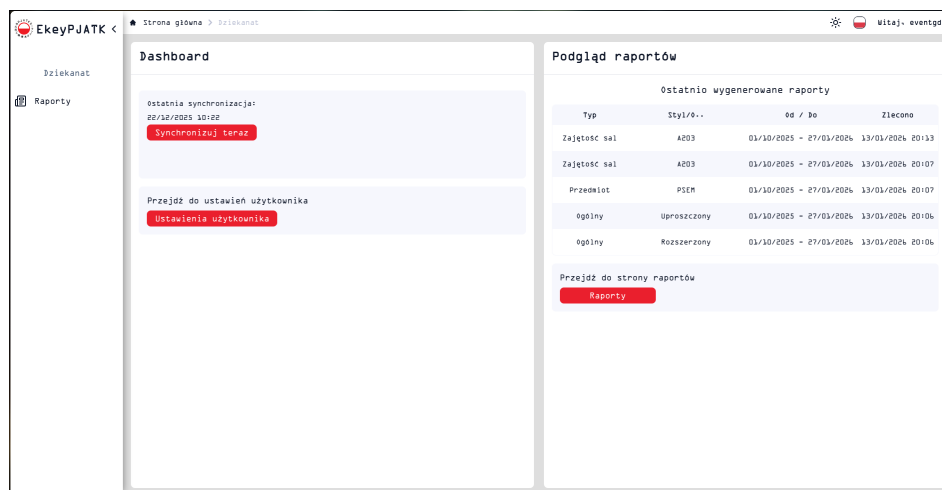


Rysunek 9.2: Ekran logowania do systemu EkeyPJATK.

Prosty układ formularza wycelowanego na ekranie minimalizuje ryzyko pomyłki i pozwala użytkownikowi skupić się na zadaniu. Po pomyślnym uwierzytelnieniu użytkownik zostaje przekierowany do widoku głównego, a jego nazwa wyświetlana jest w prawym górnym rogu interfejsu.

Widok Główny

Po zalogowaniu użytkownik trafia do widoku głównego, który skupia najważniejsze informacje w dwóch panelach:



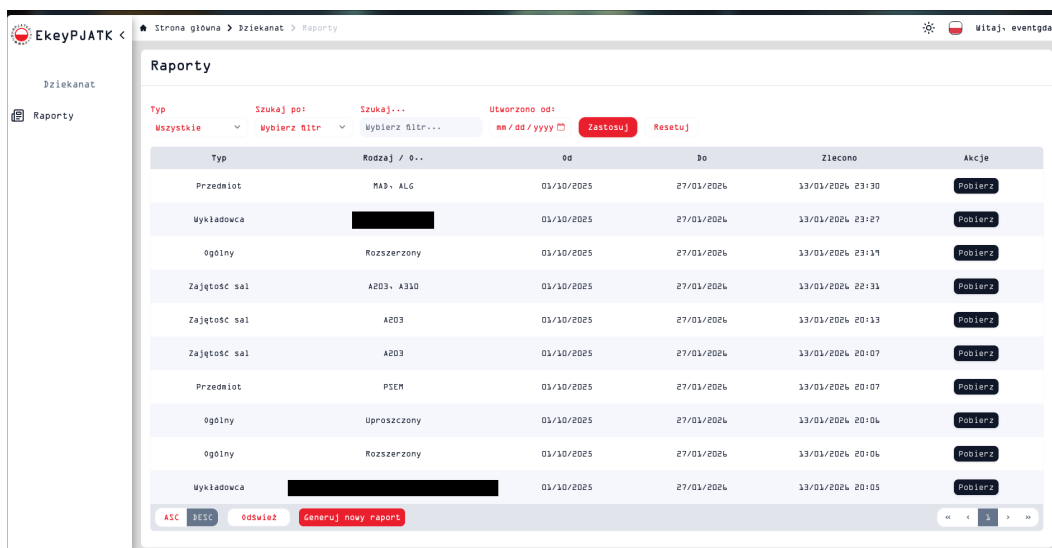
Rysunek 9.3: Widok główny z informacją o synchronizacji i przeglądem raportów.

- **Panel Dashboard:** Wyświetla informację o ostatniej synchronizacji danych z systemem GAKKO oraz udostępnia przycisk "Synchronizuj teraz" umożliwiający ręczne wymuszenie aktualizacji. Poniżej znajdują się skróty do głównych funkcji: wyszukiwarka grup, powiadomienia oraz ustawienia użytkownika.
- **Panel Reports overview:** Prezentuje listę ostatnich pięciu wygenerowanych raportów do szybkiego pobrania oraz link prowadzący do pełnego panelu raportów.

Taki układ pozwala pracownikowi Dziekanatu na błyskawiczną ocenę stanu systemu oraz dostęp do najczęściej wykonywanych operacji bez konieczności nawigowania przez menu.

Moduł raportów

Centralnym elementem interfejsu z perspektywy użytkowników końcowych jest widok zarządzania raportami. Zgodnie z założeniami projektowymi, oszczędzono użytkownikom tworzenia zapytań SQL, zamiast tego otrzymują intuicyjny interfejs z filtrowaniem i predefiniowanymi opcjami.



Rysunek 9.4: Widok listy raportów z filtrami i opcjami zarządzania.

Widok składa się z następujących elementów:

- **Panel filtrów:** Umieszczony w górnej części widoku, umożliwia zawężenie listy raportów według typu, kryterium wyszukiwania, szukania po tekście oraz daty utworzenia. Przyciski "Zastosuj" i "Resetuj" pozwalają na szybkie zarządzanie aktywnym filtrem.
- **Tabela raportów:** Prezentuje listę wygenerowanych raportów w formie tabelarycznej z kolumnami: Typ (rodzaj raportu), Rodzaj/O.. (szczegóły, np. kod przedmiotu lub nazwa wykładowcy), Od/Do (zakres dat), Zlecono (data i godzina utworzenia) oraz Akcje. Każdy wiersz zawiera przycisk "Pobierz" umożliwiający pobranie wygenerowanego pliku PDF.
- **Pasek akcji:** W dolnej części widoku znajdują się przyciski sterujące sortowaniem (ASC/DESC), odświeżaniem listy oraz generowaniem nowego raportu. Po prawej stronie umieszczono kontrolki paginacji umożliwiające nawigację między stronami wyników.

Widoczne na rysunku 9.4 dane przykładowe ilustrują różnorodność dostępnych typów raportów: raporty przedmiotowe, raporty wykładowców, raporty ogólne w dwóch wersjach, uproszczony i rozszerzony, oraz raporty zajętości sal.

9.3 Moduł QR

Moduł QR stanowi komponent systemu odpowiedzialny za alternatywną metodę uwierzytelniania użytkowników poprzez dynamicznie generowane kody QR. Roz-

wiązanie to zostało zaprojektowane w współpracy z zespołem [Cyfrowe Klucze Mobilne](#). W przeciwieństwie do statycznych kodów QR, system generuje tokeny krótko-
trwale, jednorazowe i zaszyfrowane algorytmem kryptograficznym. Takie podejście
zapewnia ochronę przed ponownym użyciem przechwyconego kodu oraz uniemożliwia
modyfikację danych zawartych w kodzie bez znajomości klucza szyfrującego.

Architektura i przepływ danych

Moduł QR funkcjonuje jako niezależny mikroservis komunikujący się z pozosta-
łymi komponentami systemu poprzez kolejkę wiadomości [RabbitMQ](#). Planowany
oryginalnie przepływ danych podczas generowania kodu QR przebiega następująco:

1. Użytkownik aplikacji mobilnej żąda wygenerowania kodu dostępu do kon-
kretnej sali.
2. Moduł QR wysyła zapytanie RPC do modułu DB w celu pobrania planu
zajęć prowadzącego.
3. Po otrzymaniu odpowiedzi system weryfikuje, czy żądany dostęp jest zgodny
z harmonogramem.
4. Generowany jest unikalny token zawierający identyfikator dostępu i kod we-
ryfikacyjny.
5. Token zostaje zaszyfrowany i zakodowany w postaci kodu QR.
6. Kod QR jest zwracany do aplikacji mobilnej z określonym czasem ważności.

Komunikacja przez RabbitMQ

Kluczowym elementem architektury modułu jest implementacja asynchronicznej
kolejki wiadomości.

```
1 func (c *rabbitTimetableClient) GetLeadSchedule(name string)
   ([]models.Schedule, error) {
2     ch, err := c.conn.Channel()
3     if err != nil {
4         return nil, errors.ExternalSourceUnavailableError()
5     }
6     defer ch.Close()
7
8     // Utworzenie tymczasowej kolejki zwrotnej
9     replyQueue, err := ch.QueueDeclare(
10        "", // RabbitMQ generuje unikalną nazwę
11        false, // durable
12        true, // autoDelete - usuwana po rozłączeniu
13        true, // exclusive - dostępna tylko dla tego
           polaczenia
```

```

14         false, // noWait
15         nil,
16     )
17
18     // Unikalny identyfikator korelacji
19     corrID := uuid.New().String()
20
21     reqBody, _ := json.Marshal(map[string]string{"name": name})
22
23     // Wysłanie zadania z adresem kolejki zwrotnej
24     ch.Publish(
25         "",
26         timetableQueueName,
27         false, false,
28         amqp091.Publishing{
29             ContentType: "application/json",
30             CorrelationId: corrID,
31             ReplyTo:      replyQueue.Name,
32             Body:          reqBody,
33         },
34     )
35
36     msgs, _ := ch.Consume(replyQueue.Name, "", true, true,
37         false, false, nil)
38     timeout := time.After(c.timeout)
39
40     for {
41         select {
42             case m := <-msgs:
43                 if m.CorrelationId != corrID {
44                     continue // Ignoruj wiadomości z innym
45                     CorrelationId
46                 }
47                 var rpcResp timetableRPCResponse
48                 json.Unmarshal(m.Body, &rpcResp)
49                 return rpcResp.Data, nil
50             case <-timeout:
51                 return nil, errors.ExternalSourceUnavailableError()
52         }
53     }
54 }

```

Listing 9.17: Klient RPC do pobierania planu zajęć (rabbitmq_client.go)

Implementacja wykorzystuje kluczowe mechanizmy wzorca [RPC](#) na kolejkach:

- **Tymczasowa kolejka zwrotna (ReplyTo):** Dla każdego żądania tworzona jest dedykowana kolejka, na którą serwer odsyła odpowiedź. Flagi `autoDelete` i `exclusive` zapewniają usunięcie kolejki po zakończeniu operacji.
- **Identyfikator korelacji (CorrelationId):** Unikalny UUID przypisywany każdemu żądaniu umożliwia jednoznaczne sparowanie odpowiedzi z żąda-

niem. Jest to niezbędne w systemach gdzie wiele ządań może być przetwarzanych równolegle.

- **Obsługa timeoutów:** Konstrukcja `select` z kanałem `time.After` zapobiega nieskończonemu oczekiwaniu na odpowiedź w przypadku problemów.

Generowanie kodów QR

Proces generowania kodu realizowany jest przez funkcję `GenerateAccessQRCode`, która stanowi centralny element logiki modułu. Funkcja ta łączy w sobie walidację danych, komunikację z backendem, generowanie tokena oraz jego zabezpieczenie kryptograficzne.

```
1 func (s *qrService) GenerateAccessQRCode(user, classCode,
2   classroom,
3   classStartTime string) (string, error) {
4
5   // Walidacja danych wejściowych
6   if user == "" || classCode == "" || classroom == "" ||
7   classStartTime == "" {
8       return "", errors.MissingRequestBodyError()
9   }
10
11   // Generowanie unikalnych identyfikatorów
12   accessID := uuid.New().String()
13   accessCode := utils.GenerateRandomCode()
14
15   // Pobranie planu zajęć
16   schedules, err := s.GetLeadSchedule(user)
17   if err != nil {
18       return "", err
19   }
20
21   // Weryfikacja zgodności z harmonogramem
22   matchingSchedule, err := utils.FindMatchingSchedule(
23       schedules, classCode, classroom, classStartTimeParsed)
24   if err != nil {
25       return "", err
26   }
27
28   // Utworzenie tokena z ograniczonym czasem życia
29   token := models.AccessToken{
30       AccessID:    accessID,
31       AccessCode:  accessCode,
32       UserName:    user,
33       ClassCode:   classCode,
34       Classroom:   classroom,
35       StartTime:   startime,
36       EndTime:     endime,
37       Expires:     time.Now().Add(2 * time.Minute),
38       Used:        false,
39   }
40
41   // Zapis tokena w Redis
42   utils.SaveToken(s.redis, token)
```

```

41
42 //Szyfrowanie payloadu
43 payload := models.AccessPayload{AccessID: accessID,
AccessCode: accessCode}
44 jsonData, _ := json.Marshal(payload)
45 secret := encryption.GetSecret()
46 encryptedData, _ := utils.EncryptAEGCM(jsonData, secret)
47 encoded := base64.StdEncoding.EncodeToString(encryptedData)
48
49 // Zapis kodu QR do pliku
50 filename := fmt.Sprintf("shared/qr_%d.png", time.Now().
UnixMilli())
51 utils.SaveQRCodeToFile(encoded, 256, filename)
52
53 // Asynchroniczne usunięcie pliku po wygasnięciu
54 go func(tokenID string, filename string) {
55     time.Sleep(2 * time.Minute)
56     os.Remove(filename)
57 }(accessID, filename)
58
59 return filename, nil
60 }

```

Listing 9.18: Generowanie zaszyfrowanego kodu QR (qr_service.go)

Token dostępu charakteryzuje się następującymi właściwościami bezpieczeństwa:

- **Krótki czas życia (TTL):** Token wygasa po 2 minutach od wygenerowania, co minimalizuje okno czasowe dla potencjalnego ataku.
- **Jednorazowość:** Pole Used oznaczane jest jako `true` po pierwszym użyciu, uniemożliwiając ponowne wykorzystanie kodu.
- **Powiązanie z harmonogramem:** Token jest ważny tylko w oknie czasowym odpowiadającym zaplanowanym zajęciom.

Po upływie czasu ważności tokenu plik z kodem QR jest automatycznie usuwany z serwera, co zapobiega jego późniejszemu wykorzystaniu i gromadzeniu niepotrzebnych danych.

Szyfrowanie AES-GCM

Dane zawarte w kodzie QR są chronione przy użyciu algorytmu [AES-GCM](#). Algorytm ten wybrany został z uwagi na jego cechy takie jak szybkość działania, bezpieczeństwo oraz wbudowany mechanizm uwierzytelniania danych.

```

1 func EncryptAEGCM(plaintext []byte, key []byte) ([]byte, error)
  {
2     if len(key) != 32 {

```

```

3         return nil, errors.New("key must be 32 bytes long")
4     }
5
6     block, err := aes.NewCipher(key)
7     if err != nil {
8         return nil, err
9     }
10
11     aesgcm, err := cipher.NewGCM(block)
12     if err != nil {
13         return nil, err
14     }
15
16     // Generowanie losowego nonce dla kazdego szyfrowania
17     nonce := make([]byte, aesgcm.NonceSize())
18     if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
19         return nil, err
20     }
21
22     // Szyfrowanie z dolaczeniem tagu autentycznosci
23     ciphertext := aesgcm.Seal(nil, nonce, plaintext, nil)
24
25     // Zwrocenie nonce + zaszyfrowanych danych
26     return append(nonce, ciphertext...), nil
27 }
28
29 func DecryptAESGCM(ciphertextWithNonce []byte, key []byte) ([]
byte, error) {
30     if len(key) != 32 {
31         return nil, errors.New("invalid key length, must be 32
bytes")
32     }
33
34     block, _ := aes.NewCipher(key)
35     aesgcm, _ := cipher.NewGCM(block)
36
37     nonceSize := aesgcm.NonceSize()
38     if len(ciphertextWithNonce) < nonceSize {
39         return nil, errors.New("ciphertext too short")
40     }
41
42     // Rozdzielenie nonce od zaszyfrowanych danych
43     nonce := ciphertextWithNonce[:nonceSize]
44     ciphertext := ciphertextWithNonce[nonceSize:]
45
46     // Deszyfrowanie z weryfikacja tagu autentycznosci
47     plaintext, err := aesgcm.Open(nil, nonce, ciphertext, nil)
48     if err != nil {
49         return nil, err // Tag nie pasuje - dane zostaly
zmodyfikowane
50     }
51
52     return plaintext, nil
53 }

```

Listing 9.19: Implementacja szyfrowania AES-GCM (encryption.go)

Kluczowe aspekty implementacji szyfrowania:

- **Klucz 256-bitowy:** Wymagana długość klucza to 256 bitów, co zapewnia odporność na ataki siłowe zgodnie z aktualnymi standardami kryptograficznymi.
- **Losowy nonce:** Dla każdej operacji szyfrowania generowany jest unikalny **nonce** o rozmiarze 12 bajtów. Ponowne użycie tego samego nonce z tym samym kluczem stanowiłoby krytyczną lukę bezpieczeństwa, taka implementacja zapobiega temu poprzez generowanie nonce z kryptograficznie bezpiecznego źródła losowości (**crypto/rand**).
- **Tag autentyczności:** Tryb **GCM** automatycznie dołącza do zaszyfrowanych danych tag uwierzytelniający. Podczas deszyfrowania tag jest weryfikowany i jakkolwiek modyfikacja zaszyfrowanych danych spowoduje niepowodzenie operacji **Open**. Mechanizm ten chroni przed atakami polegającymi na manipulacji zaszyfrowanym payloadem kodu QR.
- **Format wyjściowy:** Zaszyfrowane dane zwracane są w formacie **nonce + ciphertext**, co ułatwia późniejszą deszyfrowanie bez konieczności przechowywania nonce osobno.

Weryfikacja tokena dostępowego

Proces weryfikacji kodu QR realizowany jest przez funkcję **VerifyAccessToken**. Funkcja ta wykonuje wieloetapową walidację, sprawdzając zarówno poprawność kryptograficzną tokena, jak i jego zgodność z ograniczeniami czasowymi.

```
1 func (s *qrService) VerifyAccessToken(accessID, accessCode
   string) (bool, error) {
2     //Pobranie tokena z Redis
3     token, err := utils.GetToken(s.redis, accessID)
4     if err != nil {
5         return false, errors.NewBadRequestError("Invalid access
   ID")
6     }
7
8     // Sprawdzenie czy token nie został już użyty
9     if token.Used {
10        return false, errors.UsedAccessTokenError()
11    }
12
13    // Walidacja okna czasowego
14    now := time.Now()
15    startWindow := token.StartTime.Add(-15 * time.Minute)
16    endWindow := token.EndTime
17
18    if now.Before(startWindow) || now.After(endWindow) {
```

```

19         return false, errors.WrongTimeError()
20     }
21
22     // Weryfikacja kodu dostępu
23     if token.AccessCode != accessCode {
24         return false, errors.InvalidAccessCodeError()
25     }
26
27     // Sprawdzenie wygasnięcia tokena
28     if token.Expires.Before(time.Now()) {
29         return false, errors.ExpiredAccessTokenError()
30     }
31
32     // Atomowe oznaczenie tokena jako zużyty
33     if err := utils.MarkTokenUsed(s.redis, accessID); err !=
34     nil {
35         return false, err
36     }
37
38     return true, nil
39 }

```

Listing 9.20: Weryfikacja tokena dostępowego (qr_service.go)

Proces weryfikacji obejmuje następujące kontrole bezpieczeństwa:

- **Ochrona przed atakiem Replay:** Sprawdzenie flagi Used jako pierwsza operacja zapobiega ponownemu użyciu przechwyconego kodu QR.
- **Walidacja okna czasowego:** Token jest ważny w przedziale od 15 minut przed planowanym rozpoczęciem zajęć do momentu ich zakończenia. Okno 15-minutowe przed zajęciami zostało ustalone we współpracy z pracownikami uczelni i odpowiada średniemu czasowi potrzebnemu do przygotowania sali przez prowadzącego.
- **Weryfikacja kodu dostępu:** Oprócz identyfikatora tokena (accessID) sprawdzany jest również losowo wygenerowany kod (accessCode). Dwuczęściowa struktura uwierzytelnienia zwiększa odporność na ataki [brute force](#).
- **Atomowość oznaczenia:** Operacja MarkTokenUsed wykonywana jest w Redis, który zapewnia atomowość operacji. W przypadku wielu żądań gwarantuje to, że tylko jedno zostanie zaakceptowane, nawet jeśli kilka osób zeskanuje ten sam kod w tej samej chwili.

Przechowywanie tokenów w Redis

Moduł QR wykorzystuje bazę [Redis](#) do przechowywania tokenów dostępowych. Wybraliśmy [Redis](#) z powodu okoliczności w jakich wykorzystywany jest kod QR:

krótki czasie życia tokenów (2 minuty), konieczność szybkiego dostępu (weryfikacja musi być praktycznie natychmiastowa) oraz potrzeba automatycznego usuwania starych danych. [Redis](#) oferuje wbudowany mechanizm [TTL](#), który automatycznie usuwa klucze po upływie określonego czasu.

Struktura tokena przechowywana w [Redis](#) zawiera wszystkie informacje niezbędne do weryfikacji:

- Identyfikatory: `AccessID`, `AccessCode`
- Dane użytkownika: `UserName`
- Kontekst zajęć: `ClassCode`, `Classroom`
- Ograniczenia czasowe: `StartTime`, `EndTime`, `Expires`
- Stan: `Used`

Integracja z aplikacją mobilną

Moduł QR został zaprojektowany jako współpraca z zespołem [Cyfrowe Klucze Mobile](#). Planowana była komunikacja między aplikacją mobilną a modułem QR poprzez [REST API](#), natomiast moduł QR komunikuje się z modułem DB poprzez [RabbitMQ](#). O tym jak to się skończyło przeczytać można w rozdziale 13- podsumowanie, wnioski, i porażki.

Rozdział 10

Testy

Celem testowania było zapewnienie poprawności, niezawodności oraz bezpieczeństwa systemu [EkeyPJATK](#). Ze względu na złożoność rozwiązania oraz jego integrację z [Systemem Cyfrowego Dostępu PJATK](#), proces testowania został podzielony na cztery kategorie:

- testy jednostkowe,
- testy akceptacyjne (integracyjne),
- testy systemowe,
- testy penetracyjne.

Każdy z tych poziomów pełni odmienną funkcję i wykorzystuje inne techniki, co pozwala na kompleksową ocenę jakości systemu.

W projekcie przyjęto podejście, w którym rodzaj testu był dobierany w zależności od charakteru testowanej funkcji. Proste elementy logiki biznesowej były weryfikowane za pomocą testów jednostkowych, natomiast bardziej złożone fragmenty kodu - szczególnie te, które wymagały interakcji z bazą danych, cache lub warstwą komunikacji - były testowane przy użyciu testów integracyjnych. Dzięki temu możliwe było precyzyjne dopasowanie rodzaju testu do zakresu odpowiedzialności danej funkcji oraz uzyskanie wysokiej wiarygodności wyników testowania.

W testach jednostkowych wykorzystywano bibliotekę **testify**, zapewniającą czytelne asercje oraz wsparcie dla tworzenia [mocków](#). Testy integracyjne opierały się natomiast na narzędziu **testcontainers**, które umożliwiało uruchamianie rzeczywistych instancji MySQL i [Redis](#) w izolowanych kontenerach. Pozwalało to odtworzyć środowisko zbliżone do produkcyjnego i testować pełne przepływy danych, migracje oraz zachowanie systemu w interakcji z realną infrastrukturą.

Takie podejście - łączące testy jednostkowe, integracyjne oraz manualne testy systemowe i penetracyjne - zapewniło wszechstronną ocenę działania systemu [EkeyPJATK](#) oraz pozwoliło na wczesne wykrycie błędów zarówno w logice biznesowej, jak i w integracji z infrastrukturą oraz usługami zewnętrznymi.

10.1 Testy jednostkowe

Testy jednostkowe stanowiły podstawowy mechanizm weryfikacji poprawności logiki biznesowej systemu. Ich głównym celem było sprawdzenie działania pojedynczych funkcji i metod w pełnej izolacji od pozostałych komponentów. Podejście do projektowania testów zależało od stopnia złożoności testowanego kodu. W przypadku funkcji prostych, posiadających niewielką liczbę możliwych ścieżek wykonania, stosowano klasyczne testy jednostkowe, które nie wymagały struktury [table-driven](#). Pozwalało to zachować przejrzystość kodu testowego oraz uniknąć nadmiernej formalizacji tam, gdzie nie była ona konieczna.

W sytuacjach, w których funkcje były bardziej rozbudowane, a ich logika obejmowała wiele wariantów danych wejściowych lub różne typy błędów, stosowano podejście [table-driven tests](#). Każdy przypadek testowy był wówczas definiowany jako osobna struktura zawierająca dane wejściowe, oczekiwany wynik oraz ewentualne błędy. Dzięki temu możliwe było pełne pokrycie wszystkich scenariuszy wykonania, a także łatwe rozszerzanie zestawu testów w miarę rozwoju systemu [Ekey](#). Taka forma testów sprzyjała również jednoznaczniemu dokumentowaniu zachowania funkcji.

Zakres testowania poszczególnych pakietów zależał od ich wewnętrznej struktury. Jeżeli pakiet zawierał niewiele metod prywatnych, testy były wykonywane wyłącznie poprzez jego publiczny interfejs, co pozwalało zachować hermetyzację kodu oraz odzwierciedlało rzeczywisty sposób korzystania z modułu. Testowanie metod prywatnych było stosowane jedynie wtedy, gdy ich logika była na tyle złożona, że pominięcie ich w testach mogłoby prowadzić do niepełnego pokrycia funkcjonalności. W takich przypadkach testy umieszczano w tym samym pakiecie co kod produkcyjny, co umożliwiało dostęp do elementów nieeksportowanych.

Aby zapewnić pełną izolację testowanej logiki, wszystkie zależności zewnętrzne były [mockowane](#). Dotyczyło to między innymi warstwy bazy danych, komunikacji z usługami zewnętrznymi oraz warstwy cache opartej na [Redisie](#). Mockowanie umożli-

liwiało precyzyjne kontrolowanie zachowania zależności oraz symulowanie zarówno scenariuszy poprawnych, jak i błędnych, bez konieczności uruchamiania dodatkowej infrastruktury.

Do implementacji testów jednostkowych wykorzystywano bibliotekę **testify**, która dostarczała czytelne i wygodne w użyciu asercje oraz narzędzia do tworzenia **mocków**. Zastosowanie tej biblioteki znacząco poprawiło czytelność kodu testowego i skróciło czas potrzebny na przygotowanie testów, szczególnie w przypadkach wymagających intensywnego **mockowania**.

10.2 Testy akceptacyjne

Testy akceptacyjne (integracyjne) miały na celu zweryfikowanie poprawności współdziałania modułów systemu **Ekey** w warunkach możliwie zbliżonych do środowiska produkcyjnego. W przeciwieństwie do testów jednostkowych, które koncentrowały się na izolowanej logice funkcji, testy akceptacyjne obejmowały pełne przepływy danych pomiędzy komponentami oraz ich interakcje z infrastrukturą i zewnętrznymi usługami. Dzięki temu możliwe było wykrycie błędów wynikających z konfiguracji, niezgodności formatów danych, niepoprawnych migracji czy różnic w zachowaniu poszczególnych warstw systemu.

W ramach tych testów odtworzono realną infrastrukturę wykorzystywaną przez system **EkeyPJATK**. Przy użyciu biblioteki **testcontainers** uruchamiano instancję bazy danych **MySQL** oraz serwer **Redis**, co pozwalało testować rzeczywiste zapytania **SQL**, migracje schematu, transakcje, mechanizmy **TTL** oraz inne elementy, których nie da się wiarygodnie zweryfikować przy użyciu **mocków**. Dzięki temu testy mogły odzwierciedlać zachowanie systemu w warunkach produkcyjnych, jednocześnie zachowując pełną powtarzalność i izolację środowiska.

Komunikacja z usługami zewnętrznymi była w testach akceptacyjnych **mockowana**, pozwoliło to symulować różne scenariusze odpowiedzi - zarówno poprawnych, jak i błędnych - przy jednoczesnym wykorzystaniu pełnej logiki komunikacji zaimplementowanej w **EkeyPJATK**. Oznacza to, że wszystkie metody odpowiedzialne za integrację z usługami zewnętrznymi były rzeczywiście wykonywane, a testy obejmowały pełny przepływ danych przez warstwy odpowiedzialne za komunikację, walidację i przetwarzanie. Jedynie finalne połączenia sieciowe były zastępowane kontrolowanymi **mockami**, co pozwalało zachować deterministyczność

testów.

Takie podejście umożliwiło kompleksową weryfikację zachowania systemu w realistycznym środowisku, przy jednoczesnym zachowaniu pełnej kontroli nad odpowiedziami zewnętrznych usług. Testy akceptacyjne ujawniły błędy, które nie mogły zostać wykryte w testach jednostkowych, takie jak problemy z migracjami, niezgodności formatów danych, błędna konfiguracja połączeń czy niepoprawna obsługa komunikacji asynchronicznej. Dzięki temu stanowiły kluczowy etap potwierdzający gotowość systemu [Ekey](#) do działania w [SCD](#).

10.3 Testy systemowe

Testy systemowe miały charakter manualny i stanowiły końcowy etap weryfikacji działania [EkeyPJATK](#) w pełnym, rzeczywistym przepływie użytkowym. Ich celem było potwierdzenie, że system [Ekey](#) współpracuje z wszystkimi modułami [Systemu Cyfrowego Dostępu PJATK](#) poprawnie, a logika przyznawania dostępu działa zgodnie z założeniami w warunkach zbliżonych do produkcyjnych. W przeciwieństwie do testów akceptacyjnych, które koncentrowały się na technicznych aspektach integracji, testy systemowe obejmowały pełne scenariusze użytkowe, odzwierciedlające sposób, w jaki system będzie wykorzystywany przez użytkowników.

W ramach testów systemowych weryfikowano między innymi poprawność interpretacji harmonogramu zajęć, reakcję [SCD](#) na próby uzyskania dostępu poza czasem rezerwacji oraz zachowanie [modułów sprzętowych](#) w typowych i nietypowych sytuacjach. Każdy scenariusz testowy był przygotowany w formie tabelarycznej, obejmującej opis celu testu, warunki początkowe, kroki do wykonania, oczekiwany rezultat oraz końcowy wynik. Taka forma pozwalała na jednoznaczną ocenę poprawności działania systemu [EkeyPJATK](#) oraz ułatwiała powtarzalność testów.

Testy systemowe były szczególnie istotne w kontekście interakcji z fizycznymi urządzeniami oraz procesów, których nie da się w pełni odwzorować w środowisku testowym, takich jak zachowanie użytkownika, opóźnienia sieciowe czy specyficzne sytuacje związane z obsługą sprzętu. Dzięki nim możliwe było potwierdzenie, że system działa zgodnie z założeniami nie tylko na poziomie logiki i integracji, lecz również w praktycznym, realnym użyciu.

10.4 Testy penetracyjne

Testy penetracyjne miały na celu ocenę odporności systemu na próby nieautoryzowanego dostępu oraz identyfikację potencjalnych słabości, które mogłyby zostać wykorzystane przez osoby próbujące obejść zabezpieczenia. Podobnie jak testy systemowe, były wykonywane manualnie, jednak ich charakter był odmienny, koncentrowały się na symulowaniu zachowań typowych dla atakujących, a nie zwykłych użytkowników.

W ramach testów penetracyjnych analizowano między innymi reakcję systemu na manipulację danymi wejściowymi, próby użycia nieprawidłowych lub sfałszowanych tokenów, nieautoryzowane zapytania do [API](#) oraz sytuacje, w których błędy mogłyby ujawniać wrażliwe informacje. Testy obejmowały również weryfikację poprawności walidacji danych, odporności na błędne formaty komunikatów oraz zachowania systemu w przypadku nietypowych lub skrajnych scenariuszy komunikacyjnych.

Celem testów penetracyjnych nie było testowanie zewnętrznych usług ani infrastruktury uczelni, lecz potwierdzenie, że [EkeyPJATK](#) poprawnie zabezpiecza swoje własne interfejsy, nie dopuszcza do eskalacji uprawnień oraz nie ujawnia informacji, które mogłyby zostać wykorzystane do obejścia mechanizmów kontroli dostępu. Dzięki temu testy te stanowiły kluczowy element oceny bezpieczeństwa systemu przed wdrożeniem go do środowiska produkcyjnego.

10.4.1 Rekonesans i analiza powierzchni ataku

Pierwszą fazę testów penetracyjnych stanowił rekonesans, czyli systematyczne rozpoznanie infrastruktury w celu identyfikacji potencjalnych wektorów ataku. Etap ten jest kluczowy pomimo naszej znajomości systemu, ponieważ pozwala na określenie, które elementy są widoczne z perspektywy osoby atakującej.

Skanowanie portów i identyfikacja usług

W celu identyfikacji usług uruchomionych w badanym środowisku przeprowadzono skanowanie portów z wykorzystaniem narzędzia [Nmap](#). Skaner został skonfigurowany z flagami umożliwiającymi kompleksową analizę: `-p-` (skanowanie wszystkich 65535 portów), `-sV` (detekcja wersji usług) oraz `-sC` (uruchomienie domyślnych skryptów NSE służących do rozpoznania szczegółów konfiguracji).

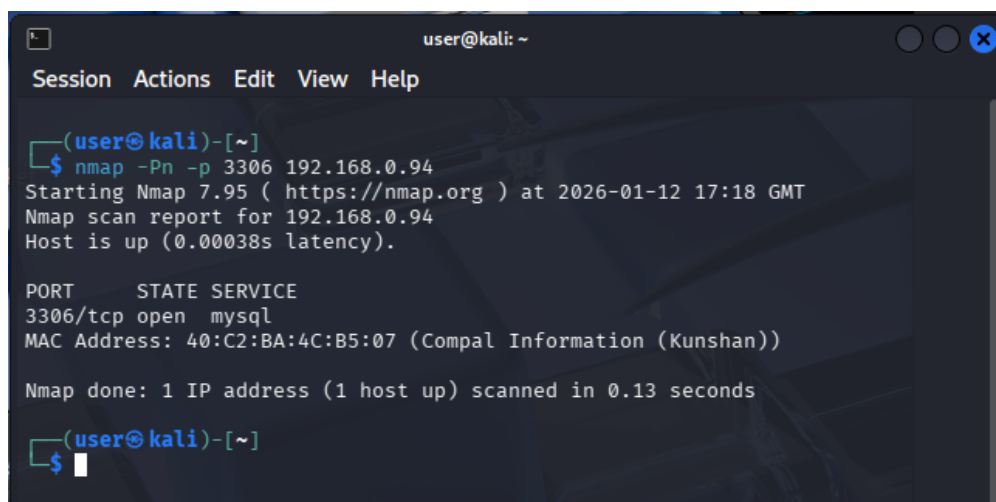
```
1 $ nmap -p- -sV -sC 172.18.48.1
2 Starting Nmap 7.98 at 2026-01-12 16:06
3 PORT      STATE SERVICE
4 3306/tcp  open  mysql
5 5432/tcp  open  postgresql
```

```

6 5672/tcp open  amqp
7 15672/tcp open  http (RabbitMQ Management)
8 ...

```

Listing 10.1: Skanowanie portów narzędziem Nmap



```

user@kali: ~
Session Actions Edit View Help

(user@kali)-[~]
$ nmap -Pn -p 3306 192.168.0.94
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-12 17:18 GMT
Nmap scan report for 192.168.0.94
Host is up (0.00038s latency).

PORT      STATE SERVICE
3306/tcp  open  mysql
MAC Address: 40:C2:BA:4C:B5:07 (Compal Information (Kunshan))

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds

(user@kali)-[~]
$

```

Rysunek 10.1: Wynik skanowania Nmap ujawniający otwarte porty baz danych i RabbitMQ.

Analiza wyników: Skanowanie ujawniło, że kluczowe elementy infrastruktury backendowej: baza danych MySQL (port 3306), PostgreSQL (port 5432) oraz system kolejkowy RabbitMQ (porty 5672, 15672) są dostępne bezpośrednio z poziomu sieci hosta. W bezpiecznej architekturze produkcyjnej usługi te powinny nasłuchiwać wyłącznie wewnątrz prywatnej sieci kontenerów (Docker Network) i nie być mapowane na interfejsy zewnętrzne.

Szczególne ryzyko stwarza otwarcie portu **15672**, który odpowiada za panel administracyjny RabbitMQ. Domyślna konfiguracja tego serwisu często pozwala na logowanie przy użyciu poświadczeń `guest:guest`, co wymagało weryfikacji w kolejnym kroku testów. Warto zaznaczyć, że testy przeprowadzane są na środowisku testowym i porty te nigdy nie zostały udostępnione w sieci publicznej, odpowiednie zmiany zawsze były w planach przed wdrożeniem produkcyjnym.

10.4.2 Weryfikacja siły haseł usług pomocniczych

W celu weryfikacji odporności usług pomocniczych na ataki słownikowe przeprowadzono próbę siłowego łamania haseł ([brute force](#)) dla panelu zarządzania RabbitMQ. Wykorzystano narzędzie [Hydra](#) oraz popularną listę najczęściej używanych haseł **Pwdb-top-1000**. Podczas testu zaobserwowano istotne zjawisko tzw. [false](#)

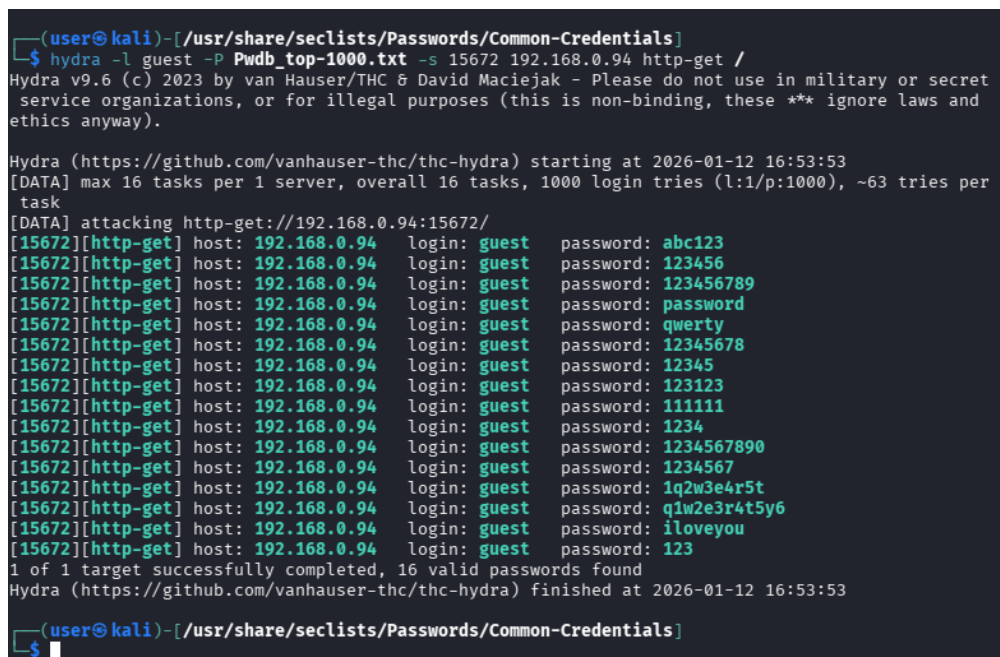
positive, które wymagało pogłębionej analizy manualnej i stanowi przykład konieczności weryfikacji wyników narzędzi automatycznych.

Automatyczne skanowanie i błędna interpretacja wyników

W pierwszej fazie testu skonfigurowano atak na główny adres panelu webowego (/). Narzędzie Hydra zwróciło sukces dla każdego testowanego hasła, sugerując odnalezienie 16 poprawnych par login/hasło.

```
1 $ hydra -l guest -P Pwdb_top-1000.txt -s 15672 192.168.0.94
   http-get /
2 [DATA] attacking http-get://192.168.0.94:15672/
3 [15672][http-get] host: 192.168.0.94 login: guest password:
   123456
4 [15672][http-get] host: 192.168.0.94 login: guest password:
   password
5 ...
6 1 of 1 target successfully completed, 16 valid passwords found
```

Listing 10.2: Narzędzie Hydra błędnie raportujące sukces ataku



```
(user@kali)-[/usr/share/seclists/Passwords/Common-Credentials]
$ hydra -l guest -P Pwdb_top-1000.txt -s 15672 192.168.0.94 http-get /
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-01-12 16:53:53
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1000 login tries (l:1/p:1000), ~63 tries per
task
[DATA] attacking http-get://192.168.0.94:15672/
[15672][http-get] host: 192.168.0.94 login: guest password: abc123
[15672][http-get] host: 192.168.0.94 login: guest password: 123456
[15672][http-get] host: 192.168.0.94 login: guest password: 123456789
[15672][http-get] host: 192.168.0.94 login: guest password: password
[15672][http-get] host: 192.168.0.94 login: guest password: qwerty
[15672][http-get] host: 192.168.0.94 login: guest password: 12345678
[15672][http-get] host: 192.168.0.94 login: guest password: 12345
[15672][http-get] host: 192.168.0.94 login: guest password: 123123
[15672][http-get] host: 192.168.0.94 login: guest password: 111111
[15672][http-get] host: 192.168.0.94 login: guest password: 1234
[15672][http-get] host: 192.168.0.94 login: guest password: 1234567890
[15672][http-get] host: 192.168.0.94 login: guest password: 1234567
[15672][http-get] host: 192.168.0.94 login: guest password: 1q2w3e4r5t
[15672][http-get] host: 192.168.0.94 login: guest password: q1w2e3r4t5y6
[15672][http-get] host: 192.168.0.94 login: guest password: iloveyou
[15672][http-get] host: 192.168.0.94 login: guest password: 123
1 of 1 target successfully completed, 16 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-01-12 16:53:53
(user@kali)-[/usr/share/seclists/Passwords/Common-Credentials]
```

Rysunek 10.2: Zrzut ekranu z narzędzia Hydra potwierdzający błędną interpretację (False Positive).

Analiza techniczna błędu: Panel logowania RabbitMQ zwraca kod odpowiedzi HTTP **200 OK** dla strony głównej niezależnie od tego, czy użytkownik jest zalogowany, czy dopiero widzi formularz logowania. Hydra, skonfigurowana w trybie **http-get**, zinterpretowała kod 200 jako udane uwierzytelnienie, co było założeniem błędnym. Sytuacja ta ilustruje ograniczenia narzędzi automatycznych,

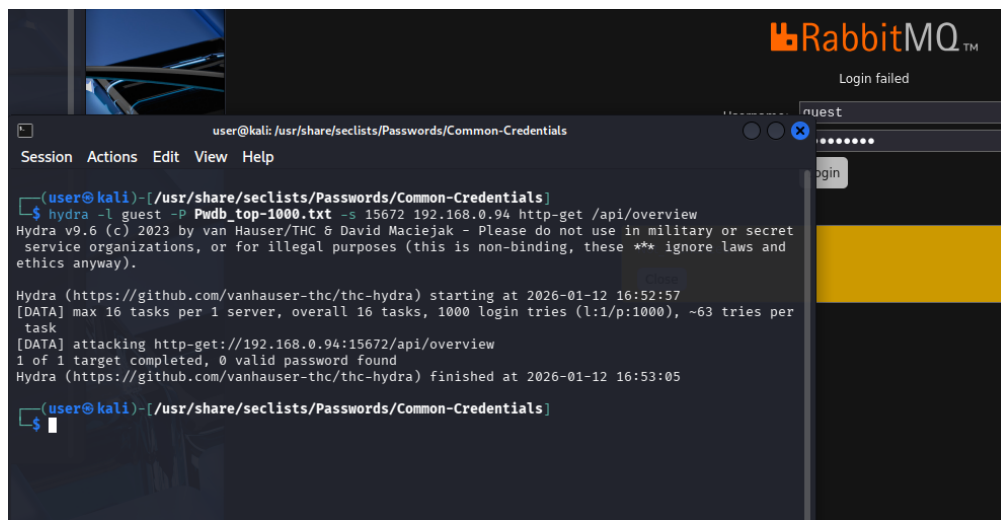
które opierają swoją logikę na prostych zasadach i nie są w stanie zrozumieć kontekstu autorskiej aplikacji webowej.

Weryfikacja manualna i korekta metodologii

W celu weryfikacji wyników podjęto próbę ręcznego zalogowania przy użyciu poświadczeń `guest:guest`, wskazanych przez narzędzie jako poprawne. Serwer odrzucił połączenie, zwracając komunikat o braku autoryzacji. Potwierdza to, że testowana instancja RabbitMQ posiada bezpieczną konfigurację, która blokuje logowanie na konto `guest` z jakiegokolwiek adresu IP innego niż `localhost`. Jest to zgodne z zaleceniami bezpieczeństwa producenta oprogramowania. Skorygowany test automatyczny został wycelowany w endpoint API (`/api/overview`), który poprawnie zwraca kody błędów **401 Unauthorized** w przypadku nieudanego uwierzytelnienia:

```
1 $ hydra -l guest -P Pwdb_top-1000.txt -s 15672 192.168.0.94
   http-get /api/overview
2 [DATA] attacking http-get://192.168.0.94:15672/api/overview
3 ...
4 1 of 1 target successfully completed, 0 valid passwords found
```

Listing 10.3: Poprawny wynik Hydry po zmianie celu na endpoint API



Rysunek 10.3: Poprawna konfiguracja RabbitMQ blokująca zdalne logowanie na konto domyślne.

Wnioski: Usługa RabbitMQ jest odporna na zdalne ataki słownikowe na konto domyślne. Test wykazał również, że poleganie wyłącznie na wynikach skanerów automatycznych bez weryfikacji kogoś z doświadczeniem często prowadzi do wyników zarówno fałszywie pozytywnych (jak w tym przypadku), jak i fałszywie nega-

tywnych. Każdy wynik narzędzia automatycznego powinien zostać zweryfikowany manualnie, aby potwierdzić jego poprawność w kontekście naszego systemu.

10.4.3 Testy obciążeniowe i wykrycie podatności typu Denial of Service

W celu weryfikacji odporności systemu na zjawisko [race condition](#) oraz ataki typu [DOS](#) przeprowadzono próbę równoległego wysłania serii żądań weryfikacyjnych dla tego samego tokena QR. Testy te miały na celu sprawdzenie, czy system poprawnie obsługuje sytuacje współbieżnego dostępu do zasobów oraz czy jest odporny na celowe przeciążenie. Wykorzystano poprawny payload kodu QR symulując sytuacje przechwycenia kodu i ponownego użycia przez osobę atakującą.

Metodologia i narzędzia testowe

Do przeprowadzenia testu wykorzystano autorski skrypt `race_test.py`, który symuluje zachowanie wielu wątków próbujących jednocześnie uzyskać dostęp do tego samego zasobu. Listing 10.4 przedstawia kluczowy fragment logiki testowej, odpowiedzialnej za wielowątkowe wysyłanie żądań HTTP POST.

```
1 def worker(url, body, headers, idx):
2     global success_count, total
3     try:
4         # Wysyłanie zadania z timeoutem 10s
5         r = requests.post(url, json=body, headers=headers,
6             timeout=10)
7         with lock:
8             total += 1
9             # Analiza odpowiedzi - szukamy potwierdzenia "
10            Access verified"
11            if r.status_code == 200:
12                if 'Access verified' in r.text:
13                    success_count += 1
14            # Rejestrowanie błędów serwera (np. 500 Internal
15            Server Error)
16            elif r.status_code >= 500:
17                responses.append((idx, 'SERVER_ERROR', r.
18                    status_code))
19            except Exception as e:
20                with lock:
21                    responses.append((idx, 'ERR', str(e)))
22
23 # (...) Uruchomienie wątków
24 for i in range(args.n):
25     t = threading.Thread(target=worker, args=(args.url, body,
26         headers, i))
27     t.start()
```

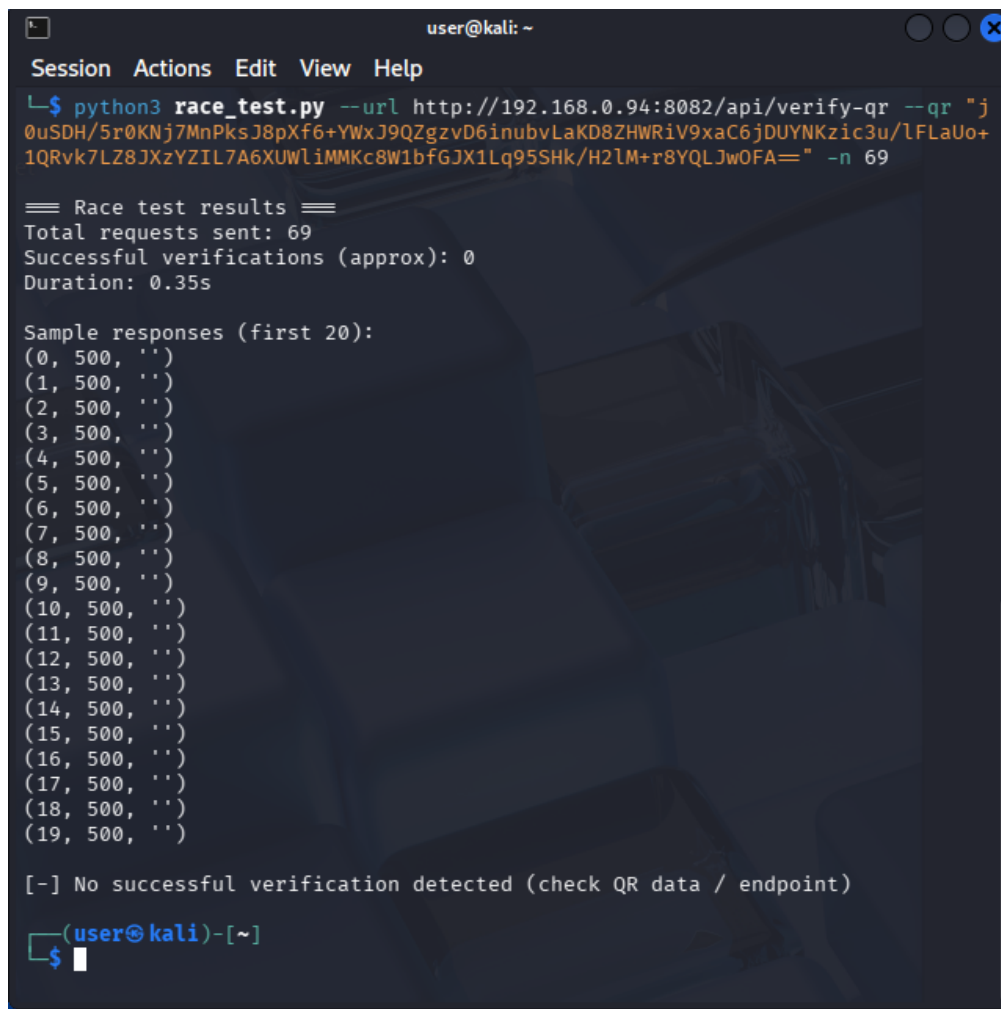
Listing 10.4: Skrypt testujący współbieżność (`race_test.py`)

Przebieg testu i obserwacje

Wygenerowano ważny kod QR, a następnie uruchomiono skrypt z parametrem `-n 20`, co skutkowało wysłaniem 20 równoległych żądań w czasie około 0.10 sekundy.

```
1 $ python3 race_test.py --url http://192.168.0.94:8082/api/  
   verify-qr --qr "..." -n 20  
2  
3 === Race test results ===  
4 Total requests sent: 20  
5 Successful verifications (approx): 0  
6 Duration: 0.10s  
7 Sample responses:  
8 (0, 500, ''), (1, 500, ''), ... (19, 500, '')  
9 [-] No successful verification detected
```

Listing 10.5: Wyniki testu obciążeniowego



```
user@kali: ~  
Session Actions Edit View Help  
$ python3 race_test.py --url http://192.168.0.94:8082/api/verify-qr --qr "j  
0uSDH/5r0KNj7MnPkSj8pXf6+YwXJ9QZgzvD6inubvLaKD8ZHWRiV9xaC6jDUYNKzic3u/LFLaUo+  
1QRvk7LZ8JXzYZIL7A6XUWliMMKc8W1bf6JX1Lq95SHk/H2LM+r8YQLJw0FA==" -n 69  
  
=== Race test results ===  
Total requests sent: 69  
Successful verifications (approx): 0  
Duration: 0.35s  
  
Sample responses (first 20):  
(0, 500, '')  
(1, 500, '')  
(2, 500, '')  
(3, 500, '')  
(4, 500, '')  
(5, 500, '')  
(6, 500, '')  
(7, 500, '')  
(8, 500, '')  
(9, 500, '')  
(10, 500, '')  
(11, 500, '')  
(12, 500, '')  
(13, 500, '')  
(14, 500, '')  
(15, 500, '')  
(16, 500, '')  
(17, 500, '')  
(18, 500, '')  
(19, 500, '')  
  
[-] No successful verification detected (check QR data / endpoint)  
  
(user@kali)-[~]  
$
```

Rysunek 10.4: Wyniki testu obciążeniowego wykrywającego podatność na race condition.

Analiza awarii: Wbrew oczekiwaniom, system nie zwrócił poprawnych odpowiedzi, ale dla wszystkich zapytań zwrócono kod błędu **500 Internal Server**

Error. Analiza logów serwera aplikacji ujawniła, że gwałtowny napływ żądań doprowadził do nieobsłużonego błędu krytycznego typu **Panic** i zatrzymania wątku obsługi żądania.

```
1 [GIN] 2026/01/12 - 17:12:44 | 500 | 530.312us | 172.18.0.1 |  
  POST "/api/verify-qr"  
2 panic recovered:  
3 runtime error: invalid memory address or nil pointer  
  dereference  
4 /usr/local/go/src/runtime/panic.go:262  
5 /app/services/qr_service.go:174 (0x6eba4a)  
6 /app/controllers/qr_controller.go:75 (0x7dce62)
```

Listing 10.6: Logi serwera wskazujące na Panic (Nil Pointer Dereference)

Analiza techniczna: Błąd `nil pointer dereference` w pliku `qr_service.go` wskazuje na brak walidacji błędu podczas pobierania tokena z bazy Redis. W warunkach gdy wiele wątków próbuje jednocześnie odczytać lub zmodyfikować ten sam klucz, funkcja pobierająca dane prawdopodobnie zwróciła wskaźnik `nil`, a kolejna linia kodu próbowała odwołać się do pól nieistniejącego obiektu. Jest to klasyczny przykład błędu współbieżności, który nie ujawnia się podczas normalnego użytkowania systemu.

Wnioski: Zidentyfikowano podatność na ataki typu **DOS**. Atakujący może, wysyłając dużą liczbę równoległych zapytań, doprowadzić do awarii wątków serwera, co w skrajnym przypadku może skutkować niedostępnością usługi dla innych użytkowników. Podatność ta wymaga naprawy przed wdrożeniem produkcyjnym poprzez dodanie odpowiedniej walidacji błędów oraz mechanizmów ograniczających liczbę równoczesnych żądań.

10.4.4 Weryfikacja bezpieczeństwa warstwy danych

W ramach testów bezpieczeństwa przeprowadzono testy bezpośredniego dostępu do bazy danych MySQL, która w badanym środowisku kontenerowym nasłuchiwała na porcie 3306. Celem testu było sprawdzenie, czy skonfigurowane poświadczenia są odporne na ataki słownikowe.

Atak słownikowy na bazę danych MySQL

Wykorzystując narzędzie **Hydra** oraz rozszerzoną listę najczęstszych haseł **Pwdb-top-10000.txt**, przeprowadzono atak na konto administracyjne **root**.

```
1 $ hydra -l root -P Pwdb_top-10000.txt 192.168.0.94 mysql  
2 [DATA] attacking mysql://192.168.0.94:3306/  
3 [3306][mysql] host: 192.168.0.94 login: root password:  
  password123  
4 1 of 1 target successfully completed, 1 valid password found
```

Listing 10.7: Atak słownikowy na bazę danych MySQL

```
(user@kali)-[/usr/share/seclists/Passwords/Common-Credentials]
$ hydra -l root -P Pwdb_top-10000.txt 192.168.0.94 mysql
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-01-12 17:20:10
[INFO] Reduced number of tasks to 4 (mysql does not like many parallel connections)
[DATA] max 4 tasks per 1 server, overall 4 tasks, 10000 login tries (l:1/p:10000), ~2500 tries p
er task
[DATA] attacking mysql://192.168.0.94:3306/
[3306][mysql] host: 192.168.0.94 login: root password: password123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-01-12 17:20:14

(user@kali)-[/usr/share/seclists/Passwords/Common-Credentials]
$
```

Rysunek 10.5: Skuteczny atak słownikowy na bazę danych i wykrycie słabego hasła deweloperskiego.

Analiza wyników: Atak zakończył się sukcesem w czasie poniżej 5 sekund. Wykryto hasło `password123`, które jest klasycznym przykładem [artefakt deweloperski](#) czyli tymczasowego rozwiązania stosowanego podczas fazy prototypowania, które nie zostało zmienione przed etapem testów.

Ocena ryzyka: Uzyskanie dostępu do konta `root` w MySQL daje atakującemu pełną kontrolę nad danymi przechowywanymi w bazie, włącznie z możliwością ich odczytu, modyfikacji i usunięcia. W kontekście systemu kontroli dostępu oznacza to potencjalną możliwość manipulacji uprawnieniami użytkowników, historią wejść oraz konfiguracją systemu.

Wnioski: Wykrycie tego zaniedbania na etapie testów penetracyjnych pozwoliło na wyeliminowanie krytycznej luki, która mogłaby doprowadzić do kompromitacji całego systemu. Incydent ten pokazuje wartość testów penetracyjnych przeprowadzanych nawet przed samym wdrożeniem produkcyjnym, umożliwiając identyfikację i naprawę błędów konfiguracyjnych, które mogłyby uciec uwadze deweloperów. Po przekazaniu informacji osobom pracującym nad tą częścią systemu, były one świadome hasła i planowały jego zmianę przed wdrożeniem produkcyjnym, ale teraz możemy być pewni że ta ani podobna sytuacja już nam nie umkną.

10.4.5 Weryfikacja odporności na wstrzykiwanie kodu SQL

Równolegle do testów infrastrukturalnych przeprowadzono weryfikację aplikacji pod kątem podatności w warstwie logicznej, ze szczególnym uwzględnieniem ataków typu [SQL Injection](#). Ataki te polegają na manipulacji parametrami wejściowymi w celu wstrzyknięcia złośliwych fragmentów zapytań SQL, co może prowadzić

do nieautoryzowanego odczytu, modyfikacji lub usunięcia danych. Testy miały na celu sprawdzenie, czy mechanizmy zastosowane w backendzie (biblioteka GORM) poprawnie izolują dane wejściowe od zapytań bazodanowych.

Automatyczne skanowanie podatności

Do przeprowadzenia testów wykorzystano narzędzie **sqlmap**, które jest popularnym wyborem w zakresie automatycznego wykrywania i eksploatacji podatności SQL Injection. Skaner został skierowany na endpoint raportowy API, przyjmujący parametry filtrujące w adresie URL.

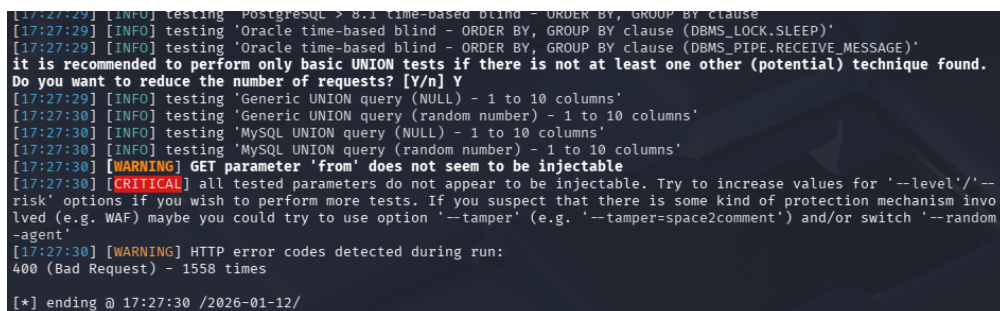
```
1 $ sqlmap -u "http://192.168.0.94:8081/api/reports/general?from=2024-01-01" \
2 --level 3 --risk 2 --batch
```

Listing 10.8: Uruchomienie skanera sqlmap z konfiguracją agresywną

Konfiguracja testu:

- **Cel:** `http://192.168.0.94:8081/api/reports/general`
- **Testowany parametr:** `from` (data początkowa zakresu raportu)
- **Poziom agresywności:** `-level 3` (rozszerzona liczba testowanych payloadów)
- **Poziom ryzyka:** `-risk 2` (uwzględnienie technik mogących modyfikować dane)

Skaner wykonał serię prób wstrzyknięcia, obejmujących między innymi techniki [time-based blind SQL Injection](#), [boolean-based blind SQL Injection](#), [error-based SQL Injection](#) oraz [UNION query-based SQL Injection](#).



```
[17:27:29] [INFO] testing 'PostgreSQL > 8.1 time-based blind - ORDER BY, GROUP BY clause'
[17:27:29] [INFO] testing 'Oracle time-based blind - ORDER BY, GROUP BY clause (DBMS_LOCK.SLEEP)'
[17:27:29] [INFO] testing 'Oracle time-based blind - ORDER BY, GROUP BY clause (DBMS_PIPE.RECEIVE_MESSAGE)'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found.
Do you want to reduce the number of requests? [Y/n] Y
[17:27:29] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[17:27:30] [INFO] testing 'Generic UNION query (random number) - 1 to 10 columns'
[17:27:30] [INFO] testing 'MySQL UNION query (NULL) - 1 to 10 columns'
[17:27:30] [INFO] testing 'MySQL UNION query (random number) - 1 to 10 columns'
[17:27:30] [WARNING] GET parameter 'from' does not seem to be injectable
[17:27:30] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
[17:27:30] [WARNING] HTTP error codes detected during run:
400 (Bad Request) - 1558 times
[*] ending @ 17:27:30 /2026-01-12/
```

Rysunek 10.6: Raport narzędzia sqlmap potwierdzający brak podatności SQL Injection w API.

Analiza wyników: Mimo wykonania ponad 1500 prób wstrzyknięcia złośliwego kodu, narzędzie zakończyło skanowanie komunikatem: **[CRITICAL] all tested parameters do not appear to be injectable**. Oznacza to, że żadna z

zastosowanych technik ataku nie pozwoliła na manipulację zapytaniami kierowanymi do bazy danych.

Wnioski: Architektura aplikacji skutecznie chroni przed atakami wstrzykiwania kodu SQL. Zastosowanie biblioteki GORM wymusza automatyczne stosowanie mechanizmu [Prepared Statements](#), który separuje strukturę zapytania od danych wejściowych dostarczanych przez użytkownika. Dzięki temu nawet celowo spreparowane ciągi znaków zawierające fragmenty składni SQL są traktowane wyłącznie jako dane tekstowe, a nie jako część wykonywanego zapytania.

10.4.6 Podsumowanie testów penetracyjnych

Przeprowadzone testy penetracyjne pozwoliły na kompleksową ocenę stanu bezpieczeństwa systemu [EkeyPJATK](#) przed wdrożeniem produkcyjnym. Tabela poniżej przedstawia zestawienie zidentyfikowanych podatności wraz z ich oceną krytyczności.

Tabela 10.1: Podsumowanie wyników testów penetracyjnych

Obszar testu	Wynik	Krytyczność
Ekspozycja portów usług wewnętrznych	Wykryto	Średnia
Domyślne hasła RabbitMQ	Brak podatności	Brak
Odporność na race condition	Wykryto podatność DoS	Średnia
Hasło bazy danych MySQL	Wykryto słabe hasło	Krytyczna
SQL Injection w API	Brak podatności	Brak

Zidentyfikowane podatności zostały udokumentowane i przekazane zespołowi deweloperskiego w celu naprawy przed wdrożeniem systemu do środowiska produkcyjnego.

10.5 Testy manualne

Testy manualne stanowiły uzupełnienie testów automatycznych, koncentrując się na weryfikacji scenariuszy trudnych do zautomatyzowania oraz interakcji z interfejsem użytkownika. Przeprowadzono je w oparciu o scenariusze, które symulowały rzeczywiste przypadki użycia systemu przez różnych użytkowników. Każdy scenariusz został udokumentowany w formie tabeli, zawierającej identyfikator, opis celu, warunki początkowe, kroki do wykonania oraz oczekiwany rezultat.

10.5.1 Testy modułu sprzętowego i kontroli dostępu

Tabela 10.2: Test manualny TM-01: Standardowe przejście z wykorzystaniem karty magnetycznej

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-01
Nazwa:	Weryfikacja standardowego procesu wejścia i wyjścia z sali
Testowane wymagania:	WF-MAG-1, WF-MAG-2, WF-ACCESS-1, WF-ACCESS-2, IO1, IO2
Warunki początkowe:	• Użytkownik posiada ważną Karta magnetyczna • Moduł sprzętowy jest podłączony do sieci i działa • W systemie GAKKO istnieje rezerwacja dla użytkownika • Test wykonywany 10 minut przed rozpoczęciem zajęć
Kroki testowe:	1. Przyłożyć kartę magnetyczną do czytnika w celu odkluczenia zamka. 2. Poczekać na reakcję systemu. 3. Zweryfikować informację o odkluczeniu na module oraz fizycznie sprawdzić zamek. 4. Przyłożyć kartę ponownie do czytnika w celu zakluczenia zamka. 5. Poczekać na reakcję systemu. 6. Zweryfikować informację o zakluczeniu na module oraz fizycznie sprawdzić zamek.
Oczekiwany rezultat:	• Czytnik poprawnie odczytuje kartę. • System podejmuje decyzję o odkluczeniu w czasie $< 2s$. • Zamek zostaje fizycznie odblokowany. • Na ekranie wyświetla się informacja o przyznaniu dostępu. • Po ponownym przyłożeniu karty, zamek zostaje fizycznie zakluczony. • Na ekranie wyświetla się informacja o zakluczeniu.
Kryteria akceptacji:	Całkowity czas od przyłożenia karty do odblokowania oraz zakluczenia zamka nie przekracza 3 sekund na każdą operację. Moduł sprzętowy operuje zamkiem zgodnie z oczekiwaniami.
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Tabela 10.3: Test manualny TM-02: Przejęcie odpowiedzialności za salę

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-02
Nazwa:	Weryfikacja mechanizmu przejęcia sali
Testowane wymagania:	WF-ACCESS-1, WF-ACCESS-3, WF-ACCESS-5, IO5
Warunki początkowe:	• Sala jest obecnie zajęta przez innego użytkownika • Test przeprowadzany jest kartą, która przypisana jest do wykładowcy, który ma zaplanowane zajęcia bezpośrednio po obecnych • Ekran modułu wyświetla aktualną rezerwację
Kroki testowe:	1. Na 5 minut przed końcem obecnych zajęć przyłożyć kartę 2. TU WPISAĆ MECHANIZM PRZEJĘCIA SALI JEŚLI CHCEMY GO TESTOWAĆ W TEN SPOSÓB 3. Sprawdzić aktualizację informacji na ekranie
Oczekiwany rezultat:	• System rozpoznaje potrzebe przekazania odpowiedzialności za salę • Po potwierdzeniu aktualizuje status sali
Kryteria akceptacji:	Przejęcie sali możliwe tylko w oknie ustalonym oknie czasowym i zgodne z harmonogramem zajęć
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Tabela 10.4: Test manualny TM-03: Weryfikacja wyświetlania informacji na module

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-03
Nazwa:	Test wyświetlania rezerwacji i kodów QR
Testowane wymagania:	IO5, WF-GAKKO-1, WO3
Warunki początkowe:	• Moduł sprzętowy działa i jest połączony z backendem • W harmonogramie są zaplanowane zajęcia
Kroki testowe:	1. Sprawdzić wyświetlanie aktualnej rezerwacji na ekranie 2. Zweryfikować poprawność danych (sala, godzina, prowadzący) 3. Sprawdzić wyświetlanie kolejnych rezerwacji (przewijanie) 4. Zweryfikować automatyczne odświeżanie
Oczekiwany rezultat:	• Ekran pokazuje aktualną i najbliższe rezerwacje • Dane odświeżają się automatycznie po synchronizacji z GAKKO
Kryteria akceptacji:	Ekran wyświetla informacje zgodne z harmonogramem zajęć
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

10.5.2 Testy frontendu i panelu dziekanatu

Tabela 10.5: Test manualny TM-04: Logowanie i autoryzacja w panelu dziekanatu

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-04
Nazwa:	Test logowania
Testowane wymagania:	IO4, NFO6, NFO2
Warunki początkowe:	• Panel administracyjny dostępny przez przeglądarkę • Użytkownik posiada konto w systemie • Połączenie HTTPS aktywne
Kroki testowe:	1. Otworzyć panel w przeglądarce 2. Wprowadzić niepoprawne dane logowania 3. Zweryfikować komunikat błędu 4. Wprowadzić poprawne dane logowania 5. Sprawdzić czy system poprawnie przekierował użytkownika do panelu dziekanatu 6. Wylogować się i sprawdzić czyszczenie sesji
Oczekiwany rezultat:	• Niepoprawne logowanie zwraca jasny komunikat błędu • Po zalogowaniu widoczne funkcje dostępne dla użytkownika
Kryteria akceptacji:	System przewidywalnie odpowiada na próby logowania
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Tabela 10.6: Test manualny TM-05: Generowanie raportów odbytych zajęć

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-05
Nazwa:	Test funkcjonalności raportowania
Testowane wymagania:	IO4, WF-ACCESS-5, WF-MAG-5
Warunki początkowe:	• Użytkownik zalogowany do panelu dziekanatu • W systemie znajdują się dane historyczne dotyczące obecności na zajęciach • Moduł raportowy działa poprawnie
Kroki testowe:	1. Przejść do sekcji "Raporty" 2. Wybrać typ raportu: "Ogólny raport zajęć" 3. Skonfigurować parametry: • Zakres dat: ostatni miesiąc • Wydział/Budunek: IT Targ Węglowy 4. Zapisać profil raportu 5. Wygenerować raport w formacie PDF 6. Wczytać zapisany raport z archiwum i wygenerować ponownie
Oczekiwany rezultat:	• Raport generuje się w czasie < 10 sekund • PDF zawiera nagłówki, stopkę, logo uczelni • Profile raportów są zapisywane i wczytywane poprawnie • Dane są spójne między formatami
Kryteria akceptacji:	Raporty zawierają tylko dane zgodne z wybranymi filtrami, generują się w całości poprawne i pobierają bez błędów
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Tabela 10.7: Test manualny TM-06: Synchronizacja z systemem GAKKO

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-06
Nazwa:	Test aktualizacji harmonogramu zajęć
Testowane wymagania:	WF-GAKKO-1, WF-GAKKO-2, WO3, IO3
Warunki początkowe:	• API GAKKO dostępne i responsywne • Ręcznie wprowadzono zmianę w harmonogramie • Panel sprzętowy działa i jest połączony z backendem
Kroki testowe:	1. Sprawdzić aktualny harmonogram w panelu 2. Ręcznie wprowadzić zmianę w harmonogramu dla danej sali w systemie, aby zasymulować zmianę dodaną przez dziekanat lub GAKKO 3. Poczekać na automatyczną synchronizację 4. Alternatywnie: wywołać ręczną synchronizację 5. Sprawdzić logi synchronizacji 6. Zweryfikować aktualizację na module sprzętowym 7. Sprawdzić zachowanie przy błędzie API (symulacja)
Oczekiwany rezultat:	• Automatyczna synchronizacja następuje co 30 minut • Ręczna synchronizacja działa natychmiast • Zmiany są widoczne w na modułach sprzętowych po synchronizacji • Przy błędzie API system używa cache danych • Wszystkie synchronizacje są logowane z timestampem
Kryteria akceptacji:	System musi działać nawet przy braku dostępu do GAKKO
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

10.5.3 Testy integracyjne i awaryjne

Tabela 10.8: Test manualny TM-07: Obsługa kart tymczasowych i gości

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-07
Nazwa:	Test kart tymczasowych dla gości
Testowane wymagania:	WF-MAG-3, WF-ACCESS-1, IO2
Warunki początkowe:	• Moduł sprzętowy działa • Gość nie posiada rezerwacji w GAKKO
Kroki testowe:	1. Cyfrowe Klucze tworzy kartę tymczasową 2. Ustawia parametry: • Ważność: 2 godziny ?? możemy ustawić na coś krótkiego do testu, nwm jak małe parametry można dać do ck • Sale: wybrana przez nas • Cel: "test gościa" 3. Wydaje kartę gościowi 4. Gość używa karty przy sali 5. Gość próbuje użyć karty przy sali do jej której nie ma dostępu system nie pozwala wejść 6. Sprawdzenie wygaśnięcia po 2 godzinach
Oczekiwany rezultat:	• Karta działa tylko dla określonych sal • Odmowa dostępu do nieautoryzowanych sal • Automatyczna dezaktywacja po czasie
Kryteria akceptacji:	Karta tymczasowa nie może działać poza określonymi parametrami
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Tabela 10.9: Test manualny TM-08: Tryb awaryjny i obsługa błędów

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-08
Nazwa:	Test działania w warunkach awaryjnych
Testowane wymagania:	WO5, WF-GAKKO-3, WF-MAG-4, NFO1
Warunki początkowe:	• System działa normalnie • Przygotowane scenariusze awarii
Kroki testowe:	Test 1: Awaria sieci • Odłączyć moduł od sieci • Sprawdzić komunikat na ekranie • Zweryfikować tryb offline (cache) Test 2: Awaria zasilania • Symulować zanik zasilania PoE • Sprawdzić odblokowanie drzwi (fail-safe) ?? może chociaż dioda albo do usunięcia ten fragment i wymaganie Test 3: Przycisk ewakuacyjny • Nacisnąć przycisk awaryjny ?? do symulacji albo usunięcia • Zweryfikować natychmiastowe otwarcie • Sprawdzić alarm w systemie Test 4: Uszkodzona karta • Użyć uszkodzonej karty magnetycznej • Sprawdzić komunikat błędu • Zweryfikować że system rozpoznaje błąd a nie odmowę dostępu
Oczekiwany rezultat:	• Przy awarii sieci: tryb cache z ostatnimi danymi • Przy braku zasilania: drzwi automatycznie otwarte • Przycisk awaryjny: natychmiastowe otwarcie • Błąd karty: jasny komunikat, nie odmowa dostępu
Kryteria akceptacji:	System fail-safe musi działać niezależnie od stanu backendu
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Tabela 10.10: Test manualny TM-09: Weryfikacja integracji z Cyfrowe Klucze

KARTA TESTU MANUALNEGO	
Identyfikator:	TM-09
Nazwa:	Test komunikacji EkeyPJATK z systemem Cyfrowe Klucze
Testowane wymagania:	IO2, WF-ACCESS-4, NFO5, NFO3
Warunki początkowe:	• WebSocket z Cyfrowe Klucze aktywny • Szyfrowanie TLS włączone • Moduł sprzętowy podłączony
Kroki testowe:	1. Monitorować komunikację WebSocket z użyciem narzędzie developerskich 2. Wykonać próbę wejścia z ważną kartą 3. Sprawdzić format komunikatu JSON do Cyfrowe Klucze 4. Zweryfikować odpowiedź z Cyfrowe Klucze 5. Sprawdzić czas odpowiedzi 6. Symulować timeout połączenia (>5s) 7. Sprawdzić ponowne nawiązanie połączenia 8. Zweryfikować szyfrowanie transmisji
Oczekiwany rezultat:	• Komunikaty zgodne ze specyfikacją Cyfrowe Klucze • Odpowiedź w czasie < 500ms • Automatyczne reconnect po utracie połączenia • Wszystkie transmisje zaszyfrowane (TLS) • Poprawna obsługa timeoutów
Kryteria akceptacji:	Brak niezaszyfrowanej komunikacji, zgodność z protokołem CK
Rezultaty testu:	TUTAJ TESTER WPISUJE CO SIĘ STAŁO LOREM IPSUM GIPSUM SRIPSUM

Rozdział 11

Prezentacja rozwiązań

Rozdział 12

Nakład pracy

W trakcie realizacji projektu [EkeyPJATK](#) przyjęto podział ról, który porządkował odpowiedzialności poszczególnych członków zespołu i odzwierciedlał ich kompetencje. Na tej podstawie zespół został podzielony na trzy podgrupy robocze:

- zespół hardware: Bartosz Bohatyrewicz, Jan Szydłowski,
- zespół dokumentacji i backendu: Łukasz Korycki, Kamil Maliński,
- rola full-stack: Igor Wojciechowski.

Taki podział pozwalał na równoległe prowadzenie prac nad kluczowymi elementami systemu oraz na efektywne wykorzystanie specjalizacji poszczególnych członków.

Podział ten miał jednak charakter orientacyjny. Zgodnie z przyjętą metodyką pracy, inspirowaną zasadami [programowania ekstremalnego](#), priorytetem było sprawne dostarczanie kolejnych działających elementów systemu, a nie sztywne trzymanie się formalnych ról. W praktyce oznaczało to, że o ile nie pojawiały się zadania wymagające natychmiastowej reakcji lub bardzo krótkiego czasu realizacji, każdy członek zespołu wykonywał przede wszystkim obowiązki wynikające z jego roli. Jednocześnie, gdy wymagała tego sytuacja projektowa, członkowie zespołu podejmowali się zadań wykraczających poza ich pierwotny zakres odpowiedzialności - szczególnie w momentach, gdy konieczne było szybkie rozwiązanie problemu, wsparcie innego modułu lub dostosowanie systemu do nowych wymagań.

Takie podejście, łączące specjalizację z elastycznością, pozwoliło utrzymać wysoki poziom równoległości prac oraz zapewniło płynne reagowanie na zmiany wynikające z dynamicznego środowiska projektu. Niniejszy rozdział przedstawia nakład pracy zarówno w ujęciu ogólnym, jak i indywidualnym, opisując rzeczywisty wkład

poszczególnych członków zespołu w powstanie systemu potencjalnie gotowego do wdrożenia w środowisku uczelnianym.

12.1 Ogólny nakład pracy

Praca nad projektem [EkeyPJATK](#) miała charakter intensywny i iteracyjny, a jej przebieg był bezpośrednio związany z przyjętą metodyką opartą na zasadach [Programowanie ekstremalne](#). Realizacja systemu wymagała stałej komunikacji, częstych spotkań oraz gotowości do szybkiego reagowania na zmieniające się wymagania i ograniczenia wynikające ze współpracy z wieloma niezależnymi interesariuszami. W praktyce oznaczało to, że znaczna część nakładu pracy była poświęcona nie tylko implementacji poszczególnych modułów, ale również analizie nowych informacji, dostosowywaniu planów oraz koordynacji działań pomiędzy podzespołami.

Regularne spotkania - zarówno w ramach zajęć projektowych, jak i organizowane dodatkowo - stanowiły fundament pracy zespołu. Każde z nich wносиło nowe ustalenia, które często wymagały natychmiastowego przełożenia na konkretne zadania. Wiele elementów systemu powstawało w krótkich cyklach, obejmujących projektowanie, implementację, testowanie oraz refaktoryzację, co pozwalało na szybkie dostarczanie działających fragmentów rozwiązania. Taki sposób pracy wymagał elastyczności oraz umiejętności płynnego przełączania się między zadaniami, szczególnie w momentach, gdy zmiany po stronie [GAKKO](#), [Cyfrowe Klucze](#) lub [Cyfrowe Klucze Mobile](#) wymuszały korekty w architekturze lub implementacji.

Znaczącą część nakładu pracy stanowiła również współpraca z interesariuszami. Konsultacje z dziekanatem, zespołem [Cyfrowe Klucze](#), osobą rozwijającą [Cyfrowe Klucze Mobile](#), administracją uczelni, firmą obsługującą [GAKKO](#) oraz promotorem były niezbędne do utrzymania spójności systemu i dostosowania go do realnych potrzeb użytkowników. Każda z tych grup wносиła własne wymagania, ograniczenia i perspektywy, co wymagało od zespołu ciągłego porządkowania priorytetów oraz aktualizowania planu prac.

Ogólny nakład pracy obejmował więc nie tylko rozwój oprogramowania i hardware'u, ale również liczne działania organizacyjne, analityczne i integracyjne. Dzięki konsekwentnej współpracy, intensywnej komunikacji oraz elastycznemu podejściu do zmieniających się warunków projektowych udało się doprowadzić sys-

tem do etapu, w którym stanowi on spójną i działającą całość, będąc jednocześnie solidną bazą do dalszego rozwoju i potencjalnego wdrożenia w środowisku produkcyjnym.

12.2 Bartosz Bohatyrewicz

12.3 Łukasz Korycki

W projekcie wykonywałem zadania związane z dokumentacją, testami oraz backendem, zgodnie z ustalonym na początku podziałem obowiązków. Do moich zadań należało opracowywanie i rozwijanie dokumentacji projektowej (`digikey-thesis`, `digikey-SWO`), przygotowywanie testów jednostkowych i implementacyjnych dla kluczowych modułów systemu (`digikey-DB`, `digikey-QR`, `digikey-raport`), a także wsparciem w tworzeniu backendu.

Zakres wykonanych prac

Praca nad backendem

Opracowanie kontraktów backendowych tłumaczących harmonogramy zajęć z [GAKKO](#) na struktury zgodne z tabelami w głównej bazie danych.

Praca nad dokumentacją

Dokument SWO

Przygotowanie kompletnego dokumentu SWO w $\text{\LaTeX}$.

Diagramy

- Stworzenie diagramów kontraktów systemu [EkeyPJATK](#) z [GAKKO](#).
- Stworzenie diagramów przypadków użycia systemu [Ekey](#).
- Współpraca przy tworzeniu diagramów baz danych i diagramów architektury systemu [Ekey](#).

Dokument inżynierski (`digikey-thesis`)

- Wkład w rozwój głównej dokumentacji projektowej, obejmujący następujące rozdziały i sekcje:

- 1. Wstęp

- **2. Słownik pojęć**
- **3. Opis problemu**
- **4. Konteksty projektu**
 - * 4.1 Kontekst systemowy
 - * 4.1.1 Zakres i granice systemu EkeyPJATK
 - * 4.1.2 Aktorzy i systemy zewnętrzne
 - * 4.1.3 Scenariusze interakcji użytkowników z systemem
 - * 4.1.4 Różnice względem prototypu Cyfrowe Klucze
- **5. Planowanie projektu**
- **6. Analiza wymagań**
 - * 6.1 Wymagania ogólne i dziedzinowe
 - * 6.2 Wymagania funkcjonalne
 - * 6.3 Wymagania zewnętrzne
 - * 6.4 Interfejs z otoczeniem
 - * 6.5 Przypadki użycia
- **7. Projektowanie systemu**
 - * 7.1 Architektura systemu
 - * 7.2 Technologie i ewolucja stosu technologicznego
- **10. Testy**
- **12. Nakład pracy** - opracowanie własnej części oraz sekcji dotyczącej ogólnego nakładu pracy.
- **13. Podsumowanie**
- **14. Bibliografia i materiały dodatkowe**

Praca nad implementacją testów

- Przygotowanie testów jednostkowych i implementacyjnych dla modułów:
 - digikey-DB,
 - digikey-QR,
 - digikey-raport.

12.4 Kamil Maliński

Jako dumny członek EkeyPJATK, zajmowałem się tworzeniem modułu raportowego `digikey-raport` jak i tworzeniem dokumentacji projektowej (`digikey-thesis`, `digikey-SW0`). Jestem też odpowiedzialny za stworzenie `mock-Cyfrowe-Klucze` i dodanie do niego infrastruktury logów wejść, co pozwoliło na testowanie modułu raportowego bez potrzeby posiadania fizycznych czytników czy ścisłej współpracy z członkami `Cyfrowe Klucze`, którzy ,zrozumiale, nie zawsze mieli czas na spotkania i pracę nad integracją naszych systemów.

Pełniłem supportową rolę przy tworzeniu testów jednostkowych i implementacyjnych dla modułów `digikey-DB`, ale zaopiekowałem się testami modułu raportowego.

Zaprojektowałem też scenariusz testów penetracyjnych, które sprawdzały cyberbezpieczeństw całego systemu EkeyPJATK.

Przeprowadzę ręcznie te testy na etapie końcowym projektu, a ich wyniki zostaną opisane w rozdziale dotyczącym testów.

Zakres wykonanych prac

Praca nad modułem raportowym `digikey-raport`

- Stworzenie modułu raportowego `digikey-raport`, odpowiedzialnego za generowanie raportów dostępu na podstawie logów wejść i danych z harmonogramu zajęć.
- Implementacja funkcji eksportu raportów w formatach PDF i CSV.
- Integracja modułu raportowego z główną bazą danych systemu EkeyPJATK.

Praca nad dokumentacją

- Wkład w rozwój głównej dokumentacji projektowej, obejmuje przede wszystkim część nietechniczną składającą się na rozdziały:
 - 1. Wstęp
 - 2. Słownik
 - 3. Opis problemu
 - 4. Konteksty projektu
 - 5. Planowanie projektu

- **6. Analiza wymagań**
- **7. Projektowanie systemu**
- **8. Opis realizacji**
- **9. Szczegóły implementacji** -następnie skupiłem się już głównie na fragmentach bezpośrednio związanych z modulem raportowym
- **10. Testy** - opis testów penetracyjnych.
- **12. Nakład pracy** - opracowanie własnej części. przez znaczną część czasu byłem też odpowiedzialny za rozwiązywanie problemów z L^AT_EX-em, które pojawiały się podczas pisania a przede wszystkim formatowanie tabel i innych elementów dokumentu.

Praca nad implementacją testów

- Przez chwilę pomagałem przy tworzeniu testów dla modułów digikey-DB, ale później skupiłem się na module raportowym.
- Przygotowanie testów jednostkowych i implementacyjnych dla modułu digikey-raport.
- Zaprojektowanie scenariusza testów penetracyjnych dla całego systemu Ekey-PJATK.
- Przeprowadzenie testów penetracyjnych na etapie końcowym projektu TODOO.
- Analiza wyników testów penetracyjnych i opracowanie rekomendacji dotyczących poprawy bezpieczeństwa systemu TODOO.

12.5 Jan Szydłowski

12.6 Igor Wojciechowski

Rozdział 13

Podsumowanie

Realizacja projektu [EkeyPJATK](#) obejmowała szeroki zakres prac związanych z integracją systemów, tworzeniem nowych modułów oraz dostosowywaniem istniejących rozwiązań do potrzeb [SCD](#). W trakcie prac konieczne było zarówno rozwijanie warstwy programistycznej i sprzętowej, jak i współpraca z wieloma zewnętrznymi zespołami odpowiedzialnymi za kluczowe elementy ekosystemu. Rozdział ten przedstawia podsumowanie zrealizowanych funkcjonalności, opisuje zadania, których nie udało się ukończyć, oraz omawia najważniejsze doświadczenia i wyzwania napotkane podczas projektu. Zawiera również propozycje dalszych kierunków rozwoju systemu, które pozwolą na jego pełne wdrożenie i dalszą ewolucję.

13.1 Zrealizowane zadania

13.1.1 Moduł raportowy

Moduł raportowy został zrealizowany w pełnym zakresie funkcjonalnym, umożliwiając generowanie zestawień dotyczących odbytych zajęć na podstawie logów pochodzących z [modułów sprzętowych](#) zamontowanych przy salach. W trakcie implementacji konieczne było jednak dostosowanie pierwotnych założeń do ograniczeń wynikających z niepełnych danych dostarczanych przez [GAKKO](#). Pomimo tych zmian moduł spełnia wymagania dziekanatu oraz administracji i stanowi w pełni funkcjonalny element [SCD](#).

Ograniczenia wynikające z danych źródłowych

Brak kontraktu zwracającego informacje o przedmiotach oraz liczbie godzin przewidzianych w sylabusie spowodował, że [Ekey](#) może jedynie zliczać liczbę godzin wynikającą z rezerwacji widniejących w planie zajęć. W praktyce oznacza to, że system nie jest w stanie określić, czy rzeczywista liczba odbytych zajęć odpowiada liczbie godzin przewidzianych dla danego przedmiotu. Ograniczenie to nie wynika

z modułu raportowego, lecz z niekompletności danych wejściowych.

Dodatkowo moduł raportowy dziedziczy ogólne ograniczenia opisane w sekcji [Integracja z Gakko](#). W szczególności:

Weryfikacja uprawnień do sal

Podstawowy mechanizm sprawdzania, czy użytkownik posiada rezerwację na daną salę w określonym czasie, działa poprawnie. Jedynym istotnym ograniczeniem pozostaje brak możliwości rozróżnienia osób o identycznych imionach i nazwiskach, co wynika bezpośrednio z niepełnych danych dostarczonych przez [GAKKO](#). Ograniczenie to wpływa również na raportowanie, ponieważ system nie zawsze może jednoznacznie przypisać log wejścia do konkretnej osoby.

Rodzaje dostępnych raportów

Moduł raportowy umożliwia generowanie dwóch głównych typów raportów:

- **Raport przedmiotowy** - zawiera zestawienie odbytych zajęć dla danego przedmiotu, obejmujące:
 - liczbę przeprowadzonych zajęć,
 - liczbę godzin wynikającą z logów wejść,
 - procentową realizację zajęć względem liczby rezerwacji w planie.
- **Raport wykładowcy** - prezentuje informacje o zajęciach prowadzonych przez danego wykładowcę:
 - liczbę odbytych zajęć dla każdego przedmiotu,
 - procentową realizację zajęć per przedmiot,
 - ogólny procent odbytych zajęć wykładowcy.

Filtrowanie i elastyczność raportów

Moduł został zaprojektowany w sposób modularny, umożliwiający zaawansowane filtrowanie danych. Najważniejsze kategorie filtrowania zostały zaimplementowane w panelu dziekanatu, obejmując m.in.:

- budynek, w którym odbywają się zajęcia,
- grupę studencką,

- kierunek studiów,
- typ zajęć.

W praktyce część filtrów jest ograniczona przez niepełność danych z planu zajęć. Przykładowo, rezerwacje typu „odrabianie” często nie zawierają informacji o kierunku studiów, co uniemożliwia ich poprawne filtrowanie według tej kategorii. Mimo to moduł raportowy umożliwia tworzenie raportów łączonych, takich jak zestawienia obejmujące wielu wykładowców lub wiele przedmiotów jednocześnie.

13.1.2 Moduł sprzętowy

Moduł sprzętowy został zrealizowany zgodnie ze wstępnymi założeniami projektowymi, jednak jego wykonanie wymagało szeregu iteracji oraz eksperymentów wynikających z ograniczonego doświadczenia zespołu w projektowaniu urządzeń elektronicznych. Proces tworzenia modułu obejmował zarówno prace nad warstwą sprzętową, jak i dostosowanie oprogramowania mikrokontrolera do specyfiki nowej konstrukcji.

W trakcie prac konieczne było opracowanie własnego układu rozdzielającego sygnał [PoE](#) na zasilanie oraz ethernet, ponieważ dostępne rozwiązania nie spełniały wymagań systemu ani nie współpracowały poprawnie z docelową konfiguracją modułu. Dodatkowo zmodyfikowano istniejące biblioteki dla mikrokontrolera [ESP32](#), aby zapewnić ich kompatybilność z nową architekturą sprzętową oraz umożliwić stabilną komunikację z serwerem [Ekey](#).

Rezultat prac

Efektem przeprowadzonych działań jest nowa wersja [modułu sprzętowego](#), wyposażona w:

- dotykowy ekran [LCD](#) o przekątnej 7”,
- w pełni działający układ zasilania i komunikacji oparty na jednym przewodzie ethernetowym,
- zmodyfikowane oprogramowanie mikrokontrolera dostosowane do wymagań systemu,
- uproszczony proces instalacji i konfiguracji urządzenia.

Moduł sprzętowy wymaga jedynie jednego kabla do poprawnego działania w ramach [SCD](#), co znacząco upraszcza montaż oraz zwiększa niezawodność całej infrastruktury. Finalna wersja urządzenia spełnia wszystkie kluczowe założenia funkcjonalne i stanowi w pełni operacyjny element systemu.

13.1.3 Moduł QR

Moduł QR został zrealizowany jako jeden z mechanizmów uwierzytelniania użytkowników w ramach [SCD](#). Jego głównym zadaniem jest umożliwienie użytkownikowi wygenerowania kodu QR, który następnie może zostać zeskanowany w aplikacji [Cyfrowe Klucze Mobile](#) w celu potwierdzenia tożsamości oraz uzyskania dostępu do sali. W trakcie implementacji konieczne było dostosowanie pierwotnych założeń do ograniczeń technicznych modułów sprzętowych oraz aplikacji mobilnej.

Charakterystyka procesu uwierzytelniania

Pomimo że kody QR często kojarzone są z bezkontaktowymi metodami uwierzytelniania, w przypadku [Ekey](#) proces ten nie jest w pełni bezkontaktowy. Aby wygenerować kod QR, użytkownik musi fizycznie wybrać odpowiednią opcję na [Moduł Sprzętowy](#), co inicjuje wyświetlenie kodu na ekranie urządzenia. Dopiero po tym etapie możliwe jest zeskanowanie kodu przy użyciu aplikacji [CKM](#). Ograniczenie to wynika z architektury sprzętowej oraz sposobu integracji z istniejącym ekosystemem [CK](#).

Ocena realizacji zadania

Pomimo konieczności dostosowania założeń do ograniczeń technicznych, moduł QR został w pełni zrealizowany i działa zgodnie z wymaganiami funkcjonalnymi. Umożliwia generowanie kodów [QR](#), poprawną komunikację z aplikacją [Cyfrowe Klucze Mobile](#) oraz obsługę scenariuszy wejścia do sal. Moduł stanowi integralną część [SCD](#), zapewniając alternatywny mechanizm uwierzytelniania użytkowników.

13.1.4 Panel zarządzania

Panel zarządzania został zrealizowany jako zestaw trzech odrębnych interfejsów, z których każdy zawiera funkcjonalności dostosowane do konkretnych potrzeb operacyjnych systemu. W ramach prac wdrożono komplet narzędzi umożliwiających obsługę procesów dydaktycznych, zarządzanie infrastrukturą sprzętową oraz kontrolę nad długoterminowymi uprawnieniami dostępu. Poniżej przedstawiono zakres funkcji zrealizowanych w poszczególnych panelach.

Panel dziekanatu

W panelu dziekanatu zaimplementowano funkcje wspierające bieżącą obsługę zajęć oraz analizę ich realizacji. Do najważniejszych elementów należą:

- podgląd ostatnich zdarzeń systemowych związanych z wejściami do sal,

- ręczne odświeżanie rezerwacji pobieranych z [GAKKO](#),
- generowanie raportów odbytych zajęć,
- konfiguracja i zapis profili raportów umożliwiających szybkie przygotowanie powtarzalnych zestawień.

Panel IT/BSS

Panel IT/[BSS](#) został wyposażony w narzędzia umożliwiające nadzór nad infrastrukturą sprzętową oraz diagnozowanie problemów technicznych. W ramach panelu zrealizowano:

- aktywację i konfigurację modułów sprzętowych,
- podgląd listy aktywnych urządzeń wraz z ich stanem,
- dostęp do logów systemowych umożliwiających analizę błędów i zdarzeń technicznych.

Panel administracji

Panel administracji rozszerza funkcjonalności pozostałych paneli o możliwość zarządzania długoterminowymi uprawnieniami dostępu. W ramach tej części systemu zaimplementowano:

- dodawanie, usuwanie i modyfikację długoterminowych [kart magnetycznych](#),
- nadawanie i odbieranie uprawnień dostępu przypisanych do kart,
- przegląd historii operacji związanych z kartami.

Panel zarządzania w finalnej formie obejmuje wszystkie kluczowe funkcjonalności wymagane do obsługi systemu [Ekey](#), zapewniając pełne wsparcie dla procesów administracyjnych, technicznych i organizacyjnych.

13.2 Niezrealizowane zadania

13.2.1 Integracja z systemem Cyfrowe Klucze (CK)

Integracja z systemem [Cyfrowe Klucze \(CK\)](#) została zrealizowana jedynie częściowo. Udało się wdrożyć podstawowy mechanizm komunikacji pomiędzy [CK](#) a [Ekey](#) poprzez połączenie [WebSocket](#), co umożliwiło działanie kluczowego scenariusza

użytkowego w ramach [Systemu Cyfrowego Dostępu PJATK \(SCD\)](#). W tym modelu, gdy użytkownik próbuje uzyskać dostęp do sali, [CK](#) wysyła zapytanie do [Ekey](#) o to, czy dana osoba posiada aktualną rezerwację na daną salę. [Ekey](#) zwraca odpowiedź, a [CK](#) na jej podstawie podejmuje ostateczną decyzję o otwarciu lub zablokowaniu drzwi. Ten element integracji działa poprawnie i stanowi fundament funkcjonalny całego [SCD](#).

Nie udało się jednak zrealizować pełnej integracji, która była wymagana, aby [SCD](#) mógł w pełni rozszerzać i współdziałać z istniejącą architekturą [CK](#). Głównym powodem był brak aktywnego udziału osoby odpowiedzialnej za backend [CK](#), co znacząco ograniczyło wprowadzenie zmian po stronie tego systemu.

Najważniejsze elementy, które wymagały modyfikacji, to:

- **Zmiana systemu logowania zdarzeń.** W obecnej wersji [CK](#) logi są wysyłane do zewnętrznej usługi analitycznej [Betterstack](#). Aby zapewnić spójność operacyjną w ramach [SCD](#), konieczne było przekierowanie logów do systemu [EkeyPJATK](#). Brak dostępu do logów uniemożliwił również generowanie raportów o odbytych zajęciach w panelu dziekanatu, co zmusiło nas do wykorzystania [mockowanej](#) wersji [CK](#) jako źródła danych.
- **Zmiana sposobu inicjalizacji modułów sprzętowych.** Oryginalnie [CK](#) zakłada, że każdy [Moduł Sprzętowy](#) ma zapisane w pamięci lokalnej swoje dane identyfikacyjne (numer sali, numer wejścia, identyfikator modułu). W praktyce oznaczało to konieczność fizycznego przygotowywania modułów, często poprzez montowanie kart microSD z plikiem inicjalizacyjnym, aby system w ogóle był w stanie je rozpoznać. W [SCD](#) planowano zastąpić ten mechanizm zdalną inicjalizacją modułów, co wyeliminowałoby potrzebę ręcznego przygotowywania urządzeń i umożliwiłoby centralne zarządzanie konfiguracją. Realizacja tego celu wymagała jednak zmian w logice [CK](#), których nie można było wykonać bez udziału autora tej części systemu.

Wszystkie stosowane obejścia były projektowane w taki sposób, aby nie odtwarzać istniejących funkcjonalności [CK](#) po stronie [Ekey](#). Jeżeli nie było możliwości pominięcia danego elementu [Cyfrowych Kluczy](#) bez reimplementacji ich logiki, w tym miejscu korzystano z [mockowanej](#) wersji [CK](#). Pozwoliło to zachować przejrzystość architektury oraz uniknąć duplikowania odpowiedzialności pomiędzy systemami. Podczas prac kierowaliśmy się zasadą zachowania jak największej kompatybilności z [CK](#), tak aby w przyszłości możliwe było pełne i bezkonfliktowe zintegrowanie

obu systemów. Dzięki temu obecna architektura [SCD](#) pozostaje otwarta na dalszy rozwój i umożliwia stopniowe zastępowanie obejść właściwą integracją, gdy tylko będzie to organizacyjnie możliwe.

13.2.2 Integracja z Gakko

Integracja z systemem [GAKKO](#) została zrealizowana jedynie w minimalnym zakresie. Udostępniony kontrakt [API](#) pozwalał na pobranie podstawowego harmonogramu zajęć, co umożliwiło działanie kluczowego scenariusza w ramach [Systemu Cyfrowego Dostępu \(SCD\)](#) - weryfikacji, czy dana osoba posiada rezerwację na określoną salę w danym czasie. Ten element integracji funkcjonuje poprawnie i stanowi fundament działania [Ekey](#).

Nie udało się jednak zrealizować pełnej integracji, która była wymagana, aby [SCD](#) mógł w pełni wykorzystywać dane z [GAKKO](#). Głównym ograniczeniem był brak kluczowych kontraktów oraz niepełny zakres danych, w szczególności brak jednoznacznych identyfikatorów prowadzących oraz brak informacji o grupach studenckich i przedmiotach. Ograniczenia te znacząco wpłynęły na możliwość wdrożenia planowanych funkcjonalności.

Najważniejsze elementy, które nie mogły zostać zrealizowane, to:

Aplikacja mobilna Cyfrowe Klucze Mobile

Brak kluczowych kontraktów po stronie [GAKKO](#) wpłynął również na integrację [Ekey](#) z aplikacją mobilną [CKM](#). Ograniczony zakres danych uniemożliwił poprawne powiązanie użytkowników z ich planem zajęć oraz jednoznaczną identyfikację prowadzących. W efekcie:

- aplikacja nie mogła obsługiwać studentów, ponieważ brakowało kontraktu zwracającego informacje o grupach zajęciowych,
- podgląd rezerwacji wykładowców nie mógł zostać poprawnie zintegrowany z [Ekey](#) z powodu braku unikalnych identyfikatorów prowadzących,
- funkcjonalność otwierania sal poprzez skanowanie kodów [QR](#) była ograniczona, ponieważ osoby o identycznych imionach i nazwiskach mogły być błędnie traktowane jako ten sam użytkownik.

Ograniczenia te nie wynikały z implementacji aplikacji mobilnej, lecz bezpośrednio z niepełnych danych udostępnionych przez [GAKKO](#), co uniemożliwiło pełną integrację z [SCD](#).

Moduł raportowy

Brak kontraktu zwracającego informacje o przedmiotach oraz liczbie godzin przewidzianych w sylabusie spowodował, że [Ekey](#) może jedynie zliczać liczbę godzin wynikającą z rezerwacji widniejących w planie zajęć. W praktyce oznacza to, że system nie jest w stanie określić, czy rzeczywista liczba odbytych zajęć odpowiada liczbie godzin przewidzianych dla danego przedmiotu.

Weryfikacja uprawnień do sal

Podstawowy mechanizm sprawdzania, czy użytkownik posiada rezerwację na daną salę w określonym czasie, działa poprawnie. Jedynym istotnym ograniczeniem pozostaje brak możliwości rozróżnienia osób o identycznych imionach i nazwiskach, co wynika bezpośrednio z niepełnych danych dostarczonych przez [GAKKO](#).

13.3 Doświadczenia i wyzwania projektu

13.3.1 Kontynuowanie projektu inżynierskiego innego zespołu

Ze względu na dostępność dwóch z trzech członków zespołu odpowiedzialnego za system [Cyfrowe Klucze](#), podjęto decyzję, aby nie tworzyć nowej wersji systemu od podstaw, lecz rozwinąć istniejące rozwiązanie poprzez dodanie brakujących funkcjonalności oraz dostosowanie go do potrzeb [Systemu Cyfrowego Dostępu PJATK](#).

Praca inżynierska oryginalnego projektu została nam przekazana, jednak ze względu na brak zgody ostatniego członka zespołu nie otrzymaliśmy dostępu do kodu źródłowego [CK](#). Wszelkie zmiany wymagane do połączenia obu systemów miały być kierowane bezpośrednio do tej osoby. W praktyce okazało się to znaczącym utrudnieniem: kontakt był bardzo ograniczony, a brak aktywnego udziału sprawił, że kluczowe elementy nie zostały zmodyfikowane lub były wprowadzane z dużym opóźnieniem. Wymuszało to wielokrotne zmiany w założeniach projektu oraz dostosowywanie harmonogramu prac.

Pozostali członkowie zespołu [CK](#) aktywnie z nami współpracowali, co znacząco ułatwiło stworzenie nowej wersji modułów sprzętowych. Dzięki temu udało się uniknąć wielu problemów, które napotkał oryginalny zespół podczas wcześniejszych etapów projektu.

Doświadczenia te pokazują, że kontynuowanie projektu inżynierskiego innego zespołu ma zarówno zalety, jak i wady. Kluczowym czynnikiem decydującym o powodzeniu takiej współpracy jest jasne ustalenie zakresu i formy udziału pierwotnych autorów. Jeżeli nie ma pewności co do ich dostępności lub gotowości do współpracy, należy rozważyć, czy kontynuacja projektu jest optymalnym rozwiązaniem, czy też lepiej rozpocząć prace od nowa, zachowując jedynie koncepcyjne elementy poprzedniego rozwiązania.

13.3.2 Współpraca z zewnętrznymi zespołami

Współpraca z wieloma niezależnymi zespołami była kluczowym elementem projektu [EkeyPJATK](#) i miała bezpośredni wpływ na jego przebieg. Taki model pracy wiązał się jednak z licznymi ryzykami organizacyjnymi i technicznymi, które ujawniły się na różnych etapach realizacji [Systemu Cyfrowego Dostępu PJATK](#).

Współpraca z zespołem odpowiedzialnym za [Cyfrowe Klucze Mobile \(CKM\)](#) wymagała od nas na samym początku stworzenia rozwiązań pośredniczących w komunikacji pomiędzy [CKM](#) a [CK](#) oraz pomiędzy [CKM](#) a [GAKKO](#). Był to intensywny sprint, którego celem było przygotowanie funkcjonalności potrzebnych do obrony pracy inżynierskiej osoby odpowiedzialnej za [CKM](#). Ostatecznie nie udało się ukończyć tego rozwiązania w wymaganym terminie.

Współpraca z [GAKKO](#) była dla nas najważniejszym elementem integracyjnym i rozpoczęła się właściwie dopiero po zakończeniu sprintu związanego z [CKM](#). Kontakt z [GAKKO](#) był jednak bardzo ograniczony, co przez długie miesiące pozostawiało nas w niepewności co do dalszych kroków. Otrzymaliśmy dostęp do harmonogramu zajęć na dany semestr, lecz udostępniony kontrakt zawierał jedynie minimalny zestaw danych niezbędnych do działania systemu [Ekey](#). Brak jednoznacznych identyfikatorów właścicieli rezerwacji tworzył istotną lukę bezpieczeństwa, potencjalnie umożliwiając dostęp do sal osobom nieuprawnionym, które przypadkowo posiadają takie samo imię i nazwisko jak wykładowca.

Współpraca z zespołem [Cyfrowych Kluczy \(CK\)](#) była stałym elementem tworzenia [EkeyPJATK](#). Pozwalała nam skupić się na konkretnych fragmentach oryginalnego systemu, które wymagały modyfikacji, oraz umożliwiła skoncentrowanie wysiłków na integracji z [GAKKO](#). Największym problemem był jednak brak współpracy ze strony członka zespołu [CK](#) odpowiedzialnego za backend, co w praktyce eliminowało większość korzyści wynikających z rozwijania istniejącego

systemu zamiast tworzenia nowego rozwiązania od podstaw. Ze względu na bliskość obu systemów, spotkania z oryginalnym zespołem [CK](#) były organizowane regularnie, często z taką samą częstotliwością jak spotkania z promotorem.

Doświadczenia te pokazują, że współpraca z wieloma zewnętrznymi zespołami może znacząco przyspieszyć rozwój projektu, ale jednocześnie wprowadza dodatkowe ryzyka związane z komunikacją, dostępnością kluczowych osób oraz zależnościami między systemami. W naszym przypadku czynniki te miały istotny wpływ na harmonogram i zakres realizowanych funkcjonalności.

13.3.3 Wyzwania i doświadczenia z wytworzeniem własnych modułów sprzętowych

13.4 Plany na przyszłość

Wraz z zakończeniem obecnego etapu prac nad [EkeyPJATK](#) możliwe jest wskazanie kierunków dalszego rozwoju, które pozwolą na pełne wykorzystanie potencjału [Systemu Cyfrowego Dostępu PJATK](#) oraz jego stopniowe wdrażanie w środowisku produkcyjnym. Poniższe zadania obejmują zarówno rozwój funkcjonalny, jak i działania organizacyjne oraz techniczne, które mogą znacząco podnieść wartość i skalowalność [SCD](#).

- **Pełna integracja z systemem [Cyfrowe Klucze](#) lub połączenie obu systemów w jeden spójny projekt.** Celem jest wyeliminowanie obecnych obejść oraz stworzenie jednolitej architektury, w której logika dostępu, komunikacja i obsługa [modułów sprzętowych](#) będą realizowane w ramach jednego systemu. Pozwoli to uprościć utrzymanie, zwiększyć niezawodność oraz umożliwić dalszy rozwój bez zależności od zewnętrznych zespołów.
- **Dodanie możliwości wyświetlania materiałów promocyjnych [PJATK](#) na ekranach modułów sprzętowych.** Plan zakłada rozszerzenie oprogramowania [modułów sprzętowych](#) o funkcję prezentacji treści multimedialnych, takich jak ogłoszenia, wydarzenia uczelniane czy komunikaty administracyjne. Wymaga to stworzenia systemu dystrybucji treści oraz mechanizmu ich zdalnej aktualizacji.
- **Przystosowanie [modułów sprzętowych](#) do warunków produkcyjnych.** Celem jest dopracowanie konstrukcji modułów tak, aby były odporne na intensywne użytkowanie, warunki środowiskowe oraz różne scenariusze

awaryjne. Obejmuje to poprawę obudowy, stabilności zasilania, odporności na uszkodzenia oraz uproszczenie procesu instalacji i serwisowania.

- **Wdrożenie systemu SCD na terenie uczelni w środowisku produkcyjnym.** Obejmuje to instalację [modułów sprzętowych](#), konfigurację infrastruktury sieciowej, testy obciążeniowe, szkolenia dla administracji oraz przygotowanie procedur wsparcia technicznego. Jest to kluczowy krok w przejściu z fazy prototypowej do realnego użytkowania.
- **Udostępnienie studentom możliwości otwierania sal, w których mają zajęcia, przy użyciu kart studenckich.** Zadanie obejmuje rozszerzenie istniejącego mechanizmu autoryzacji opartego na danych z harmonogramu zajęć, który obecnie umożliwia otwieranie sal wyłącznie właścicielowi rezerwacji. Planowana zmiana polega na dodaniu możliwości autoryzacji również dla grupy studenckiej przypisanej do danej rezerwacji. Realizacja tego zadania wymaga SCD o dodanie mechanizmu powiązania identyfikatorów kart studenckich z danymi o uczestnikach zajęć, co pozwoli studentom na samodzielne otwieranie sal, w których mają zaplanowane zajęcia.
- **Wprowadzenie systemu SCD w innych uczelniach lub instytucjach edukacyjnych.** Plan zakłada przygotowanie systemu do wdrożeń zewnętrznych poprzez standaryzację konfiguracji, stworzenie dokumentacji wdrożeniowej oraz dostosowanie architektury do różnych modeli organizacyjnych. Pozwoli to skalować projekt poza [PJATK](#).
- **Dodanie w raportach możliwości generowania statystyk zajętości sal.** Rozbudowa modułu raportowania o funkcję analizy wykorzystania sal na podstawie danych z harmonogramu oraz logów dostępu umożliwi administracji uczelni optymalizację planowania zajęć i zarządzania przestrzenią.

Realizacja powyższych zadań pozwoli na dalsze dojrzewanie systemu, zwiększenie jego niezawodności oraz przygotowanie go do pełnego wdrożenia zarówno na [PJATK](#), jak i w innych instytucjach. Stanowią one naturalny kierunek rozwoju [EkeyPJATK](#) jak i [Systemu Cyfrowego Dostępu](#), umożliwiając stopniowe przejście od prototypu do stabilnego, skalowalnego rozwiązania produkcyjnego.

13.5 Podsumowanie końcowe

Projekt [EkeyPJATK](#) pozwolił na stworzenie prototypu systemu, który łączy w sobie nowe moduły sprzętowe, mechanizmy uwierzytelniania oraz narzędzia admini-

nistracyjne, jednocześnie integrując się z istniejącą infrastrukturą uczelni. Pomimo licznych ograniczeń technicznych i organizacyjnych udało się zrealizować kluczowe elementy funkcjonalne, które stanowią solidną podstawę do dalszego rozwoju.

Doświadczenia zdobyte podczas prac - zarówno w zakresie projektowania urządzeń elektronicznych, jak i współpracy z wieloma zespołami zewnętrznymi - pokazały, jak złożonym przedsięwzięciem jest budowa systemu opartego na integracji wielu niezależnych komponentów. Jednocześnie potwierdziły, że przy odpowiedniej architekturze i konsekwentnym podejściu możliwe jest stworzenie rozwiązania, które może być rozwijane i skalowane w przyszłości.

Przedstawione w tym rozdziale plany rozwojowe wyznaczają naturalny kierunek dalszych prac, prowadzący od prototypu do stabilnego systemu produkcyjnego. Stanowią one fundament, na którym [System Cyfrowego Dostępu](#) może być stopniowo udoskonalany i wdrażany zarówno na [PJATK](#), jak i w innych instytucjach edukacyjnych.

Rozdział 14

Bibliografia i materiały dodatkowe

14.1 Tabele

4.1	Dziekanat	30
4.2	Administracja Uczelni	31
4.3	Promotor	31
4.4	Recenzent	32
4.5	Zespół Ekey	32
4.6	BSS	33
4.7	Zespół Cyfrowe Klucze Mobile	33
4.8	Zespół Cyfrowe Klucze	34
4.9	Personel techniczny	34
4.10	GAKKO	35
5.1	Podział ról i odpowiedzialności w zespole projektowym	48
5.2	Tabela ryzyk i ograniczeń dla Systemu Cyfrowego Dostępu z uwzględnieniem zależności.	51
6.1	Integracja z planem zajęć	56
6.2	Modularność systemu	57
6.3	Obsługa dynamicznych zmian planu	57
6.4	Zgodność z politykami bezpieczeństwa uczelni	57
6.5	Tryb awaryjny i degradacja działania	58
6.6	Cykliczna synchronizacja danych z GAKKO	59
6.7	Aktualizacja rekordów planu zajęć	59
6.8	Obsługa awarii komunikacji z GAKKO	60
6.9	Podstawowa decyzja o dostępie	60
6.10	Obsługa okna czasowego weryfikacji	61
6.11	Powiązanie decyzji z salą	61

6.12	Przekazywanie decyzji do systemu CK	61
6.13	Rejestrowanie decyzji o dostępie	62
6.14	Odczyt identyfikatora karty magnetycznej	62
6.15	Przekazywanie identyfikatora do backendu	62
6.16	Obsługa kart tymczasowych	63
6.17	Obsługa błędów odczytu karty magnetycznej	63
6.18	Rejestrowanie prób wejścia kartą magnetyczną	63
6.19	Zgodność z przepisami bezpieczeństwa (BHP/ppož.)	64
6.20	Wymagania dotyczące ochrony danych osobowych	65
6.21	Wymagania dotyczące infrastruktury sieciowej	65
6.22	Zewnętrzne wymagania integracyjne GAKKO	66
6.23	Wymagania zewnętrzne dotyczące integracji z Cyfrowe Klucze	66
6.24	Wymagania bezpieczeństwa dla systemu EkeyPJATK	67
6.25	Interfejs: moduł sprzętowy → system EkeyPJATK	68
6.26	Interfejs: EkeyPJATK → Cyfrowe Klucze	69
6.27	Interfejs: GAKKO → EkeyPJATK	70
6.28	Interfejs: Panel dziekanatu → Backend	71
6.29	Interfejs: urządzenie → generowanie kodów QR	72
10.1	Podsumowanie wyników testów penetracyjnych	153
10.2	Test manualny TM-01: Standardowe przejście z wykorzystaniem karty magnetycznej	154
10.3	Test manualny TM-02: Przejęcie odpowiedzialności za salę	155
10.4	Test manualny TM-03: Weryfikacja wyświetlania informacji na module	156
10.5	Test manualny TM-04: Logowanie i autoryzacja w panelu dziekanatu	157
10.6	Test manualny TM-05: Generowanie raportów odbytych zajęć	158
10.7	Test manualny TM-06: Synchronizacja z systemem GAKKO	159
10.8	Test manualny TM-07: Obsługa kart tymczasowych i gości	160
10.9	Test manualny TM-08: Tryb awaryjny i obsługa błędów	161
10.10	Test manualny TM-09: Weryfikacja integracji z Cyfrowe Klucze	162

14.2 Rysunki

3.1	Rich Picture ilustrujący obecny system zarządzania dostępem do pomieszczeń w budynku PJATK w Gdańsku.	17
3.2	Strona główna firmy Tritech [1]	20
3.3	Logitech Tap Scheduler [2].	22
3.4	Vidikom System Informacji o Salach [3].	23

5.1	Kategorie ryzyk w Cyfrowe Klucze (CK) [CK, rozdział 4.4].	49
5.2	Ryzyka i ograniczenia w Cyfrowe Klucze Mobile (CKM) [CKM, rozdział 6.2].	50
6.1	Diagram przypadków użycia.	73
6.2	Diagram przypadków użycia dla podstawowych użytkowników. . . .	74
6.3	Diagram przypadków użycia dla użytkowników z uprawnieniami do sal.	75
6.4	Diagram przypadków użycia dla dziekanatu.	76
6.5	Diagram przypadków użycia dla ochrony.	77
6.6	Diagram przypadków użycia dla IT/BSS.	78
6.7	Diagram przypadków użycia dla administratorów.	79
7.1	Architektura Systemu Cyfrowego Dostępu z wyróżnionymi komponentami EkeyPJATK i ich powiązaniem.	80
7.2	Architektura Cyfrowe Klucze przed integracją w Systemie Cyfrowego Dostępu PJATK [CK, rozdział 8.1.1].	81
7.3	Interakcja systemów podczas logowania w Cyfrowe Klucze Mobile [CKM, rozdział 8.2.1].	82
7.4	Diagram kontraktów od GAKKO z mapowaniem do bazy danych. . . .	83
7.5	Diagram kontraktu rezerwacji od GAKKO z mapowaniem do bazy danych.	84
7.6	Diagram kontraktu semestralnego od GAKKO z mapowaniem do bazy danych.	85
7.7	Diagram kontraktu grup uczelnianych od GAKKO z mapowaniem do bazy danych.	87
7.8	Diagram głównej bazy danych EkeyPJATK ze zaznaczonymi połączeniami z kontraktami.	88
7.9	Diagram bazy danych raportów EkeyPJATK.	89
7.10	Projekt Lo-Fi widoku zarządzania rezerwacjami.	93
7.11	Koncepcja widoku obsługi zdarzeń z filtrowaniem.	94
7.12	Projekt interfejsu przeglądarki logów.	95
7.13	Koncepcja modułu generowania raportów.	96
7.14	Przykładowy ekran obsługi błędu 503.	98
7.15	Diagram połączeń modułu sprzętowego.	100
7.16	Przykładowy widok przedmiotów na wyświetlaczu	100
9.1	Porównanie motywów kolorystycznych interfejsu na przykładzie widoku raportów.	129

9.2	Ekran logowania do systemu EkeyPJATK.	129
9.3	Widok główny z informacją o synchronizacji i przeglądem raportów.	130
9.4	Widok listy raportów z filtrami i opcjami zarządzania.	131
10.1	Wynik skanowania Nmap ujawniający otwarte porty baz danych i RabbitMQ.	145
10.2	Zrzut ekranu z narzędzia Hydra potwierdzający błędną interpretację (False Positive).	146
10.3	Poprawna konfiguracja RabbitMQ blokująca zdalne logowanie na konto domyślne.	147
10.4	Wyniki testu obciążeniowego wykrywającego podatność na race condition.	149
10.5	Skuteczny atak słownikowy na bazę danych i wykrycie słabego hasła deweloperskiego.	151
10.6	Raport narzędzia sqlmap potwierdzający brak podatności SQL Injection w API.	152

14.3 Kod

7.1	Przykład konfiguracji kontraktu rezerwacji	84
7.2	Przykład konfiguracji kontraktu semestralnego	85
7.3	Przykład konfiguracji kontraktu grup uczelnianych	87
7.4	Przykładowy kontrakt autoryzujący moduł do komunikacji	101
9.1	Encje domenowe snapshotu (schedule_snapshot.go)	110
9.2	Algorytm synchronizacji snapshotów (sync_service.go)	111
9.3	Algorytm weryfikacji dostępu (service.go)	112
9.4	Interfejs przechowywania własności sali (ownership_store.go)	113
9.5	Logika ustalania własności sali (service.go)	113
9.6	Serwis pushowania planu do czytników (ck_schedule_push_service.go)	115
9.7	Klient WebSocket do komunikacji z czytnikami (ws.go)	115
9.8	Kontroler HTTP do obsługi zapytań o plan zajęć zwracający odpowiedź w formacie MobileScheduleDTO (timetable_controller.go)	116
9.9	Repozytorium snapshotów (snapshot_repo.go)	117
9.10	Bezpieczne pobieranie zadania z kolejki z wykorzystaniem blokowania wierszy	120
9.11	Struktury danych obsługujące logikę ciągów (matching.go)	121
9.12	Weryfikacja ciągłości uprawnień (matching.go)	122
9.13	Wyszukiwanie logu w oknie czasowym	122

9.14 Klient API pobierający logi wejść (log_client.go)	124
9.15 Konfiguracja stylów i renderowanie komórek tabeli (pdf_renderer.go)	126
9.16 Rejestracja definicji raportów i konfiguracja batchingu (router.go) .	126
9.17 Klient RPC do pobierania planu zajęć (rabbitmq_client.go)	132
9.18 Generowanie zaszyfrowanego kodu QR (qr_service.go)	134
9.19 Implementacja szyfrowania AES-GCM (encryption.go)	135
9.20 Weryfikacja tokena dostępowego (qr_service.go)	137
10.1 Skanowanie portów narzędziem Nmap	144
10.2 Narzędzie Hydra błędnie raportujące sukces ataku	146
10.3 Poprawny wynik Hydry po zmianie celu na endpoint API	147
10.4 Skrypt testujący współbieżność (race_test.py)	148
10.5 Wyniki testu obciążeniowego	149
10.6 Logi serwera wskazujące na Panic (Nil Pointer Dereference)	150
10.7 Atak słownikowy na bazę danych MySQL	150
10.8 Uruchomienie skanera sqlmap z konfiguracją agresywną	152

14.4 Bibliografia

- [CK] Marek Kudła, Kinga Marszałkowska, and Antoni Ulenberg. „Dokumentacja projektu "Cyfrowe Klucze"”. Praca dyplomowa. PJATK, 2024.
- [CKM] Magda Megan Kokornacka. „Dokumentacja projektu "Cyfrowe Klucze Mobile"”. Praca dyplomowa. PJATK, 2025.
- [1] Tritech New Technologies Sp. z o.o. *Strona główna firmy Tritech*. Dostęp: 1 listopada 2025. 2025. URL: <https://tritech.pl/> (visited on 11/01/2025).
- [2] Logitech. *Logitech Tap Scheduler - panel rezerwacji sal*. Dostęp: 1 listopada 2025. 2025. URL: <https://www.logitech.com/pl-pl/products/video-conferencing/room-solutions/tap-scheduler.952-000091.html> (visited on 11/01/2025).
- [3] Vidikom. *System Informacji o Salach (SIS)*. Dostęp: 1 listopada 2025. 2025. URL: <https://vidikom.com.pl/multimedialne-systemy-informacyjne/> (visited on 11/01/2025).
- [XP] *Programowanie ekstremalne*. Dostęp: 02.12.2025. URL: <http://www.extremeprogramming.org/>.